

CRUD operations with CBVs

While working with Django models, one of the most common patterns we run into involves creating, reading, updating, and deleting objects that are stored inside our database. The Django admin interface allows us to achieve these CRUD operations easily, but what if we wanted to build custom views to give us the same capability?

As it turns out, Django's CBVs allow us to achieve this quite easily. All we need to do is to write our custom CBVs and inherit them from the built-in base classes provided by Django. Building on our existing example of book record management, let's see how we can build CRUD-based views in Django.

The Create view

To build a view that helps in object creation, we'll need to open the `view.py` file under the `book_management` directory and add the following lines of code to it:

```
from django.views.generic.edit import CreateView
from .models import Book

class BookCreateView(CreateView):
    model = Book
    fields = ['name', 'author']
    template_name = 'book_form.html'
    success_url = '/book_management/entry_success'
```

With this, we created `CreateView` for the book resource. Before we can use it, we will need to map it to a URL. To do this, we can open the `urls.py` file and add the following entry under the `urlpatterns` list:

```
urlpatterns = [
    ...,
    path('book_record_create', BookCreateView.as_view(),
        name='book_create')
]
```

Now, when we visit `http://localhost:8080/book_management/book_record_create`, we will be greeted with the following page:

CBV를 이용한 CRUD 작업

Django 모델 작업 중에 우리가 자주 마주치는 가장 일반적인 패턴 중 하나는 데이터 베이스에 저장된 객체를 생성, 읽기, 업데이트 및 삭제하는 것입니다. Django 관리 인터페이스를 사용하면 이러한 CRUD 작업을 쉽게 수행할 수 있지만, 동일한 기능을 제공하는 사용자 정의 뷰를 구축하고 싶다면 어떻게 해야 할까요?

알고 보니, Django의 CBV를 사용하면 이를 매우 쉽게 달성할 수 있습니다. 우리가 해야 할 일은 사용자 정의 CBV를 작성하고 Django에서 제공하는 내장 기본 클래스에서 상속하는 것입니다. 기존의 도서 기록 관리 예제를 바탕으로, Django에서 CRUD 기반 뷰를 어떻게 구축할 수 있는지 살펴보겠습니다.

생성 뷰

객체 생성을 돕는 뷰를 구축하기 위해, `book_management` 디렉토리 아래의 `view.py` 파일을 열고 다음 코드 줄을 추가해야 합니다:

```
from django.views.generic.edit import CreateView
from .models import Book

class BookCreateView(CreateView):
    model = Book
    fields = ['name', 'author']
    template_name = 'book_form.html'
    success_url = '/book_management/entry_success'
```

이로써 우리는 책 리소스를 위한 `CreateView`를 만들었습니다. 사용하기 전에 URL에 매핑해야 합니다. 이를 위해 `urls.py` 파일을 열고 `urlpatterns` 목록 아래에 다음 항목을 추가할 수 있습니다:

```
urlpatterns = [
    ...,
    path('book_record_create', BookCreateView.as_view(),
        name='book_create')
]
```

이제 `http://localhost:8080/book_management/book_record_create`를 방문하면 다음 페이지가 표시됩니다:

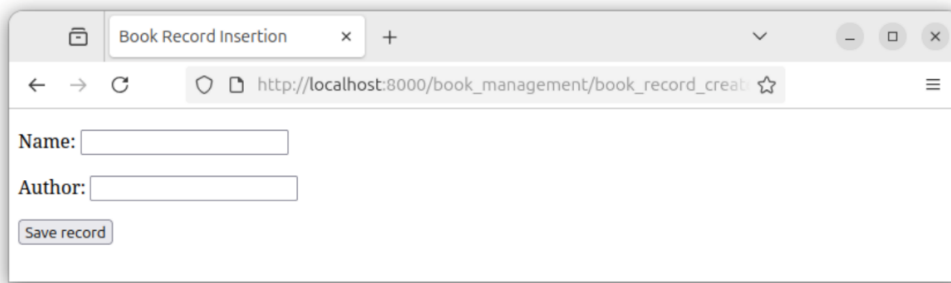


Figure 11.6 – A view to insert a new book record based on the Create view

This looks similar to the one we got when using the form view. On filling in the data and clicking **Save record**, Django will save the data to the database.

The Read view

In this view, we want to see a list of records that our database stores for the books. To achieve this, we will build a view named `DetailView`, which will render details about the book we are requesting. To build this view, we can add the following lines of code to our `views.py` file under the `book_management` directory:

```
from django.views.generic import DetailView

class BookRecordDetailView(DetailView):
    model = Book
    template_name = 'book_detail.html'
```

In the preceding code snippet, we created `DetailView`, which will help us to render the details pertaining to the book ID we are asking for. `DetailView` internally queries our database model with the book ID we provide to it and, if a record is found, renders the template with the data stored inside the record by passing it as an object variable inside the template context.

Once this is done, the next step is to create the template for our book details. For this, we'll need to create a new template file named `book_detail.html` under our `templates` directory inside the `book_management` application with the following contents:

```
<html>
<head>
  <title>Book List</title>
</head>
```

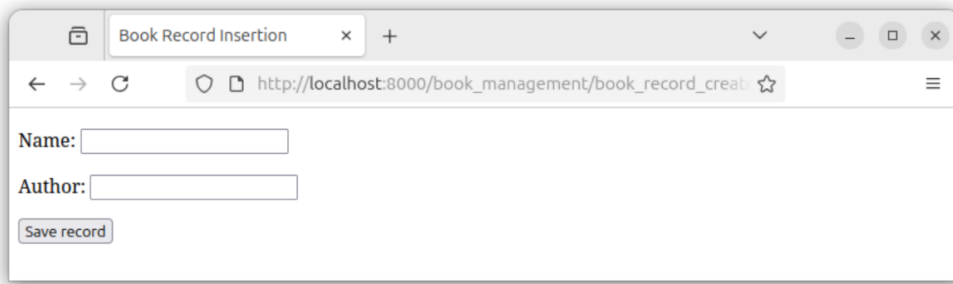


그림 11.6 - 생성 뷰를 기반으로 새 책 레코드를 삽입하는 보기

이것은 양식 보기를 사용할 때 얻은 oneidx_36d6dcdf와 유사해 보입니다. 데이터를 입력하고 레코드 저장을 클릭하면 Django가 데이터를 데이터베이스에 저장합니다.

읽기 보기

이 보기에서는 idx_74d9acd2가 우리 데이터베이스가 책에 대해 저장하는 레코드 목록을 보고 싶어합니다. 이를 달성하기 위해 `DetailView`라는 보기를 구축할 것이며, 이는 우리가 요청하는 책에 대한 세부 정보를 렌더링합니다. 이 보기를 구축하기 위해 `book_management` 디렉토리의 `views.py` 파일에 다음 코드 줄을 추가할 수 있습니다:

```
from django.views.generic import DetailView

class BookRecordDetailView(DetailView):
    model = Book
    template_name = 'book_detail.html'
```

앞서 언급한 코드 조각에서는 `DetailView`를 생성했습니다. 이는 우리가 요청하는 책 ID와 관련된 세부 정보를 렌더링하는 데 도움이 됩니다. `DetailView`는 내부적으로 제공된 책 ID로 데이터베이스 모델을 쿼리하고, 레코드가 발견되면 해당 레코드에 저장된 데이터를 객체 변수로 템플릿 컨텍스트에 전달하여 템플릿을 렌더링합니다.

이 작업이 완료되면 다음 단계 idx_4d146ca1는 책 세부 정보를 위한 템플릿을 만드는 것입니다. 이를 위해 `book_management` 애플리케이션의 템플릿 디렉토리 아래에 `book_detail.html`이라는 새 템플릿 파일을 생성해야 하며, 다음 내용을 포함해야 합니다:

```
<html>
<head>
  <title>책 목록</title>
</head>
```

```
<body>
  <span>Book Name: {{ object.name }}</span><br />
  <span>Author: {{ object.author }}</span>
</body>
</html>
```

Now, with the template created, the last thing we need to do is to add a URL mapping for the detail view. This can be done by appending the following to the `urlpatterns` list under the `urls.py` file of the `book_management` application:

```
path('book_record_detail/<int:pk>',
      BookRecordDetailView.as_view(), name='book_detail')
```

Now, with all of this configured, if we now open `http://localhost:8080/book_management/book_record_detail/2`, we will get to see the details about our book, as shown here:

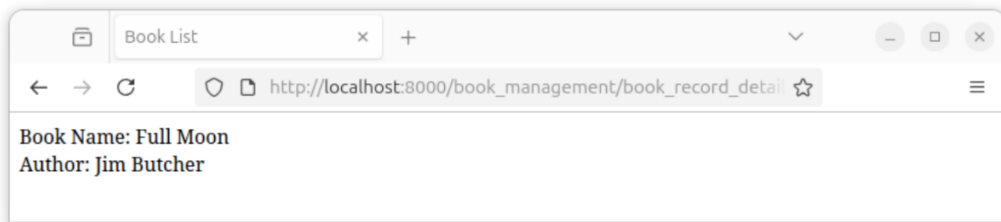


Figure 11.7 – The view rendered when we try to access a previously stored book record

With the preceding examples, we just enabled CRUD operations for our `Book` model, all while using CBVs.

The Update view

In this view, we want to update the data for a given record. To do this, we need to open the `view.py` file under the `book_management` directory and add the following lines of code to it:

```
from django.views.generic.edit import UpdateView

from .models import Book

class BookUpdateView(UpdateView):
    model = Book
    fields = ['name', 'author']
```

```
<body>
  <span>책 이름: {{ object.name }}</span><br />
  <span>저자: {{ object.author }}</span>
</body>
</html>
```

이제 템플릿이 생성되었으므로, 마지막으로 해야 할 일은 detail 뷰에 대한 URL 매핑을 추가하는 것입니다. 이는 book_management 애플리케이션의 urls.py 파일 아래의 urlpatterns 목록에 다음을 추가하여 수행할 수 있습니다:

```
path('book_record_detail/<int:pk>',
     BookRecordDetailView.as_view(), name='book_detail')
```

이제 모든 설정이 완료되었으므로, http://localhost:8080/book_management/book_record_detail/2를 열면, 여기에서 보여지는 것처럼 우리의 책에 대한 세부 정보를 볼 수 있습니다:

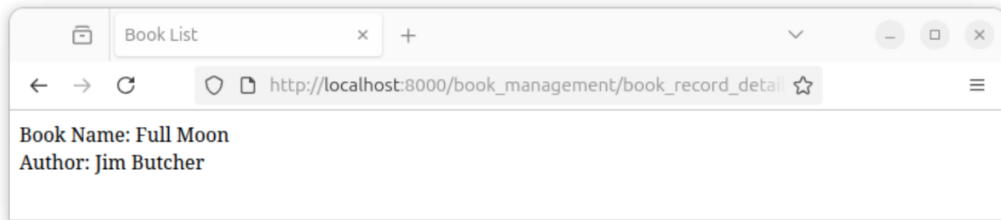


그림 11.7 - 이전에 저장된 책 기록에 접근하려고 할 때 렌더링된 뷰

앞선 예제들을 통해 우리는 CBV를 사용하여 우리의 Book 모델에 대한 CRUD 작업을 활성화했습니다.

업데이트 뷰

이 뷰에서는 주어진 기록에 대한 데이터를 업데이트하고자 합니다. 이를 위해 book_management 디렉토리 아래의 view.py 파일을 열고 다음 코드 줄을 추가해야 합니다:

```
from django.views.generic.edit import UpdateView

from .models import Book

class BookUpdateView(UpdateView):
    model = Book
    fields = ['name', 'author']
```

```
template_name = 'book_form.html'  
success_url = '/book_management/entry_success'
```

In the preceding code snippet, we used the built-in `UpdateView` template, which allows us to update the stored records. Therefore, the `fields` attribute here should take in the name of the fields that we would like to allow the user to update.

Once the view is created, the next step is to add the URL mapping. To do this, open the `urls.py` file under the `book_management` directory and add the following lines of code:

```
urlpatterns = [  
    path('book_record_update/<int:pk>',  
         BookUpdateView.as_view(), name='book_update')  
]
```

In this example, we appended `<int:pk>` to the URL field. This signifies the field input we will have to retrieve the record for. Inside Django models, Django inserts a primary key of the integer type, which uniquely identifies the records. Inside the URL mapping, this is the field that we asked to insert.

Now, when we try to open `http://localhost:8080/book_management/book_record_update/1`, it should show us a record of the first record that we inserted into our database and allow us to edit it:

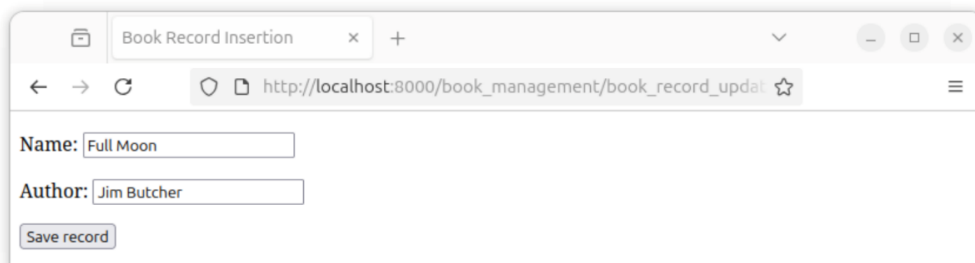


Figure 11.8 – The view displaying the book record update template based on the `UpdateView`

```
template_name = 'book_form.html'
success_url = '/book_management/entry_success'
```

앞서 언급한 코드 조각에서는 저장된 레코드를 업데이트할 수 있는 내장 UpdateView 템플릿을 사용했습니다. 따라서 여기서 fields 속성은 사용자가 업데이트할 수 있도록 허용할 필드의 이름을 받아야 합니다.

뷰가 생성되면 다음 단계는 URL 매핑을 추가하는 것입니다. 이를 위해 book_management 디렉토리 아래의 urls.py 파일을 열고 다음 코드 줄을 추가합니다:

```
urlpatterns = [
    path('book_record_update/<int:pk>',
        BookUpdateView.as_view(), name='book_update')
]
```

이 예제에서는 URL 필드에 <int:pk>를 추가했습니다. 이는 우리가 레코드를 검색하기 위해 입력해야 하는 필드를 나타냅니다. Django 모델 내부에서 Django는 레코드를 고유하게 식별하는 정수형 기본 키를 삽입합니다. URL 매핑 내부에서 이것은 우리가 삽입하라고 요청한 필드입니다.

이제 http://localhost:8080/book_management/book_record_update/1을 열어보면, 데이터베이스에 삽입한 첫 번째 레코드의 레코드를 보여주고 이를 편집할 수 있어야 합니다:

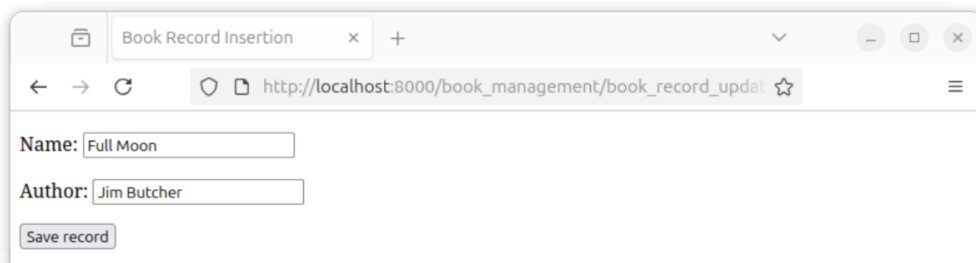


그림 11.8 - 업데이트 뷰를 기반으로 한 도서 레코드 업데이트 템플릿을 표시하는 뷰

The Delete view

The Delete view, as the name suggests, is a view that deletes the record from our database. To implement such a view for our Book model, we will need to open the `views.py` file under the `book_management` directory and add the following code snippet to it:

```
from django.views.generic.edit import DeleteView
from .models import Book

class BookDeleteView(DeleteView):
    model = Book
    template_name = 'book_delete_form.html'
    success_url = '/book_management/delete_success'
```

With this, we just created a Delete view for our book records. As we can see, this view uses a different template, where all we would like to confirm with the user is whether they really want to delete the record or not? To achieve this, you can create a new template file, `book_delete_form.html`, and add the following code to it:

```
<html>
<head>
    <title>Delete Book Record</title>
</head>
<body>
    <p>Delete Book Record</p>
    <form method="POST">
        {% csrf_token %}
        Do you want to delete the book record?
        <input type="submit" value="Delete record" />
    </form>
</body>
</html>
```

Note

For the code related to the `/delete_success` endpoint, take a look at the code files under the `Chapter 11` directory on the book's accompanying GitHub repository.

삭제 보기

삭제 보기는 이름에서 알 수 있듯이 데이터베이스에서 레코드를 삭제하는 `viewidx_1088a9d3`입니다. `Book` 모델에 대해 이러한 보기를 구현하려면 `book_management` 디렉토리 아래의 `views.py` 파일을 열고 다음 코드 스니펫을 추가해야 합니다:

```
from django.views.generic.edit import DeleteView
from .models import Book

class BookDeleteView(DeleteView):
    model = Book
    template_name = 'book_delete_form.html'
    success_url = '/book_management/delete_success'
```

이로써 우리는 책 기록을 위한 삭제 뷰 `idx_432035fd`를 방금 생성했습니다. 보시다시피, 이 뷰는 다른 템플릿을 사용하며, 사용자에게 확인하고 싶은 것은 정말로 기록을 삭제할 것인지 여부입니다. 이를 달성하기 위해, 새로운 템플릿 파일 `book_delete_form.html`을 생성하고 다음 코드를 추가할 수 있습니다:

```
<html>
<head>
  <title>책 기록 삭제</title>
</head>
<body>
  <p>책 기록 삭제</p><form
    method="POST">

    {% csrf_token %}
    책 기록을 삭제하시겠습니까?<input type="submit"
      value="기록 삭제" />

  </form>
</body>
</html>
```

메모

`/delete_success` 엔드포인트와 관련된 코드는 책의 GitHub 저장소의 Chapter 11 디렉토리 아래에 있는 코드 파일을 확인하세요.

Then we can add a mapping for our delete view by modifying the `urlpatterns` list inside the `urls.py` file under the `book_management` directory as follows:

```
urlpatterns = [
    ...,
    path('book_record_delete/<int:pk>',
        BookDeleteView.as_view(), name='book_delete')
]
```

Now, when visiting `http://localhost:8080/book_management/book_record_delete/1`, we should be greeted with the following page:

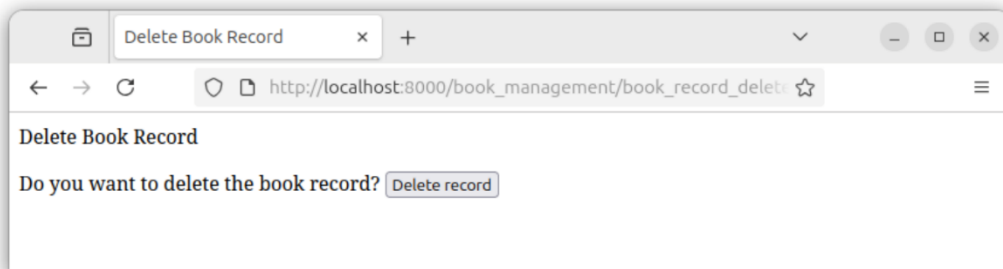


Figure 11.9 – The Delete Book Record view based on the `DeleteView` class

Upon clicking the **Delete record** button, the record will be deleted from the database, and the **Deletion success** page will be rendered.

Summary

In this chapter, we learned about the advanced templating concepts in Django and understood how we can create custom template tags, filters, and partials to fit a myriad of use cases and support the reusability of components across the application. We then examined how Django provides us with the flexibility to implement FBVs and CBVs to render our responses.

While exploring CBVs, we learned how they can help us avoid code duplication and leverage the built-in CBVs to render forms that save data, help us update existing records, and implement CRUD operations on our database resources.

As we move on to the next chapter, we will now utilize our knowledge of building CBVs to work on implementing REST APIs in Django. This will allow us to perform well-defined HTTP operations on our data inside our Bookr application without maintaining any state inside the application.

그런 다음 book_management 디렉토리의 urls.py 파일 내의 urlpatterns 목록을 다음과 같이 수정하여 delete view에 대한 mappingidx_5e01903d를 추가할 수 있습니다:

```
urlpatterns = [
    ...,
    path('book_record_delete/<int:pk>',
        BookDeleteView.as_view(), name='book_delete')
]
```

이제 http://localhost:8080/book_management/book_record_delete/1을 방문하면 다음 페이지가 표시됩니다:

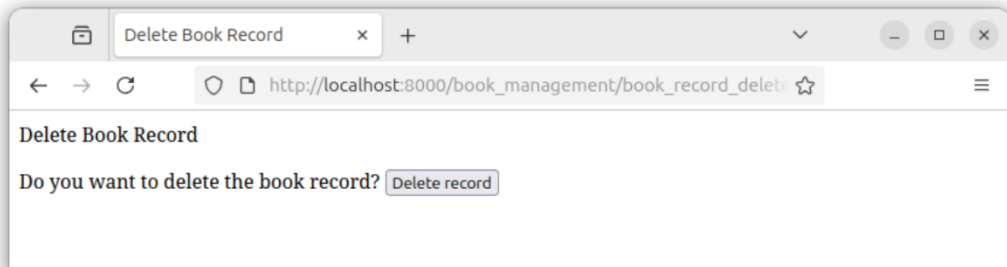


그림 11.9 - DeleteView 클래스를 기반으로 한 도서 기록 삭제 보기

삭제 기록 버튼을 클릭하면 기록이 데이터베이스에서 삭제되고 삭제 성공 페이지가 렌더링됩니다.

요약

이번 장에서는 Django의 고급 템플릿 개념에 대해 배우고, 다양한 사용 사례에 맞게 사용자 정의 템플릿 태그, 필터 및 부분을 생성하여 애플리케이션 전반에 걸쳐 구성 요소의 재사용성을 지원하는 방법을 이해했습니다. 그런 다음 Django가 응답을 렌더링하기 위해 FBV와 CBV를 구현할 수 있는 유연성을 제공하는 방법을 살펴보았습니다.

CBV를 탐색하면서 코드 중복을 피하고 내장된 CBV를 활용하여 데이터를 저장하고 기존 기록을 업데이트하며 데이터베이스 리소스에서 CRUD 작업을 수행하는 양식을 렌더링하는 데 어떻게 도움이 되는지 배웠습니다.

다음 장으로 넘어가면서 이제 CBV 구축에 대한 지식을 활용하여 Django에서 REST API를 구현하는 작업을 진행할 것입니다. 이를 통해 애플리케이션 내에서 상태를 유지하지 않고도 Bookr 애플리케이션 내의 데이터에 대해 잘 정의된 HTTP 작업을 수행할 수 있습니다.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하세요 (또는 packtpub.com/unlock으로 가세요). 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

12

Building a REST API

In the previous chapter, we learned about templates and class-based views. These concepts greatly help expand the range of functionalities we can provide to the user on the frontend (in their web browser). However, that is not sufficient to build a modern web application. Web apps typically have the frontend built with an entirely separate library, such as ReactJS or AngularJS. These libraries provide powerful tools for building dynamic user interfaces but do not communicate directly with our backend Django code or database. The frontend code simply runs in the web browser and does not have direct access to any data on our backend server. Therefore, we need to create a way for these applications to "talk" to our backend code. One of the best ways to do this in Django is by using REST APIs.

Application programming interfaces (APIs) are used to facilitate interaction between different pieces of software, and they communicate using **Hypertext Transfer Protocol (HTTP)**. This is the standard protocol for communication between servers and clients and is fundamental to information transfer on the web. APIs receive requests and send responses in HTTP format.

This chapter introduces **REST APIs** and **Django REST framework (DRF)**. You will start by implementing a simple API for the bookr project. Next, you will learn about the serialization of model instances, which is a key step in delivering data to the frontend side of Django applications. Finally, you will explore different types of API views, including both functional and class-based types.

In our use case in this chapter, an API will help facilitate interaction between our Django backend and our frontend JS code. For example, imagine that we want to create a frontend application that allows users to add new books to the bookr database. The user's web browser would send a message (an HTTP request) to our API to say that they want to create an entry for a new book and perhaps include some details about the book in that message. Our server

12

REST API 구축

이전 장에서는 템플릿과 클래스 기반 뷰에 대해 배웠습니다. 이러한 개념은 사용자에게 제공할 수 있는 기능의 범위를 크게 확장하는 데 도움이 됩니다(웹 브라우저에서). 그러나 이는 현대 웹 애플리케이션을 구축하기에는 충분하지 않습니다. 웹 애플리케이션은 일반적으로 ReactJS 또는 AngularJS와 같은 완전히 별도의 라이브러리로 프론트엔드를 구축합니다. 이러한 라이브러리는 동적 사용자 인터페이스를 구축하기 위한 강력한 도구를 제공하지만, 우리의 백엔드 Django 코드나 데이터베이스와 직접적으로 통신하지 않습니다. 프론트엔드 코드는 단순히 웹 브라우저에서 실행되며, 우리의 백엔드 서버의 데이터에 직접 접근할 수 없습니다. 따라서 이러한 애플리케이션이 우리의 백엔드 코드와 "대화"할 수 있는 방법을 만들어야 합니다. Django에서 이를 수행하는 가장 좋은 방법 중 하나는 REST API를 사용하는 것입니다.

응용 프로그램 프로그래밍 인터페이스(API)는 서로 다른 소프트웨어 조각 간의 상호 작용을 촉진하는 데 사용되며, 하이퍼텍스트 전송 프로토콜(HTTP)을 사용하여 통신합니다. 이는 서버와 클라이언트 간의 통신을 위한 표준 프로토콜이며, 웹에서 정보 전송의 기본입니다. API는 요청을 받고 HTTP 형식으로 응답을 보냅니다.

이 장에서는 REST API와 Django REST 프레임워크(DRF)를 소개합니다. 책r 프로젝트를 위한 간단한 API를 구현하는 것으로 시작할 것입니다. 다음으로, Django 애플리케이션의 프론트엔드에 데이터를 전달하는 데 중요한 단계인 모델 인스턴스의 직렬화에 대해 배울 것입니다. 마지막으로, 기능 기반 및 클래스 기반 유형을 포함한 다양한 유형의 API 뷰를 탐색할 것입니다.

이 장의 사용 사례에서 API는 우리의 Django 백엔드와 프론트엔드 JS 코드 간의 상호 작용을 촉진하는 데 도움이 될 것입니다. 예를 들어, 사용자가 책r 데이터베이스에 새 책을 추가할 수 있는 프론트엔드 애플리케이션을 만들고 싶다고 가정해 보겠습니다. 사용자의 웹 브라우저는 새 책에 대한 항목을 생성하고 싶다는 메시지(HTTP 요청)를 API에 보내고, 아마도 그 메시지에 책에 대한 세부 정보를 포함할 것입니다. 우리의 서버

would send back a response to report on whether the book was successfully added or not. The web browser would then be able to display the outcome of their action to the user.

In this chapter, we will cover the following topics:

- Understanding REST APIs
- DRF
- Understanding serializers
- Simplifying the code using ViewSets
- Implementing authentication

By the end of this chapter, you will be able to implement custom API endpoints, including token-based authentication.

Technical requirements

All the code files of this chapter can be found in the Chapter12 folder in this book's GitHub repo. To access the repository link, follow the steps in the *"Download the example code files"* section in the *Preface*.

Understanding REST APIs

Most modern web APIs can be classified as **representational state transfer (REST)** APIs. REST APIs are simply a type of API that focuses on communicating and synchronizing the *state* of objects between the database server and the frontend client.

For example, imagine that you are updating your details on a website for which you are signed in to your account. When you go to the account details page, the web server tells your browser about the various details attached to your account. When you change the values on that page, the browser sends back the updated details to the web server and tells it to update these details in the database. If the action is successful, the website will show you a confirmation message.

This is a very simple example of what is known as **decoupled** architecture between frontend and backend systems. Decoupling allows greater flexibility and makes it easier to update or change components in your architecture. So, let's say you want to create a new frontend website. In such a case, you don't have to change the backend code at all, as long as your new frontend is built to make the same API requests as the old one.

REST APIs are *stateless*, meaning neither the client nor the server stores any states in between to do the communication. Every time a request is made, the data is processed, and a response is sent back without the protocol having to store any intermediate data. What this means is that the API is processing each request in isolation. It doesn't need to store information regarding

책이 성공적으로 추가되었는지 여부에 대한 응답을 보낼 것입니다. 그런 다음 웹 브라우저는 사용자가 그들의 행동의 결과를 표시할 수 있게 됩니다.

이 장에서는 다음 주제를 다룰 것입니다:

- REST API 이해하기
- DRF
- 시리얼라이저 이해하기 ViewSets
- 를 사용하여 코드 단순화하기인
- 증 구현하기

이 장이 끝날 때쯤이면 토큰 기반 인증을 포함한 맞춤형 API 엔드포인트를 구현할 수 있게 될 것입니다.

기술 요구 사항

이 장의 모든 코드 파일은 이 책의 GitHub 저장소 내 Chapter12 폴더에서 찾을 수 있습니다. 저장소 링크에 접근하려면 서문에 있는 "예제 코드 파일 다운로드" 섹션의 단계를 따르세요.

REST API 이해하기

대부분의 최신 idx_47f10369web API는 표현 상태 전송(REST) API로 분류될 수 있습니다. RESTidx_009a03bbAPI는 데이터베이스 서버와 프론트엔드 클라이언트 간에 객체의 상태를 통신하고 동기화하는 데 중점을 둔 API 유형입니다.

예를 들어, 당신이 계정에 로그인한 상태에서 웹사이트에서 정보를 업데이트한다고 상상해 보세요. 계정 상세 페이지로 이동하면 웹 서버가 당신의 브라우저에 계정에 연결된 다양한 정보를 알려줍니다. 그 페이지에서 값을 변경하면 브라우저가 업데이트된 정보를 웹 서버로 다시 보내고 데이터베이스에서 이 정보를 업데이트하도록 지시합니다. 작업이 성공하면 웹사이트는 확인 메시지를 보여줍니다.

이것은 프론트엔드와 백엔드 시스템 간의 분리된 아키텍처로 알려진 매우 간단한 예입니다. 분리는 더 큰 유연성을 제공하며 아키텍처 내 구성 요소를 업데이트하거나 변경하는 것을 더 쉽게 만듭니다. 예를 들어, 새로운 프론트엔드 웹사이트를 만들고 싶다고 가정해 봅시다. 이런 경우, 새로운 프론트엔드가 기존과 동일한 API 요청을 하도록 구축되어 있다면 백엔드 코드를 전혀 변경할 필요가 없습니다.

REST API는 상태 비저장(stateless) 방식으로, 클라이언트나 서버가 통신 중간에 어떤 상태도 저장하지 않는다는 의미입니다. 요청이 이루어질 때마다 데이터가 처리되고, 프로토콜이 중간 데이터를 저장할 필요 없이 응답이 반환됩니다. 이것이 의미하는 바는 API가 각 요청을 독립적으로 처리한다는 것입니다. API는 관련 정보를 저장할 필요가 없습니다.

the session itself. This is in contrast to a stateful protocol (such as **Transmission Control Protocol (TCP)**), which maintains information regarding the session during its life.

So, a **RESTful web service**, as the name suggests, is a collection of REST APIs used to carry out a set of tasks. For example, if we develop a set of REST APIs for the bookr application to carry out a certain set of tasks, we can call it a RESTful web service. Next, we will start working with DRF.

DRF

DRF is an open source Python library that can be used to develop REST APIs for a Django project. DRF has most of the necessary functionality built in to help develop APIs for any Django project. Throughout this chapter, we will be using it to develop APIs for our bookr project. In the following section, we will install and configure DRF.

Installation and configuration

To install `django-rest-framework` in the `virtual env` setup along with PyCharm, follow these steps:

1. Enter the following code in your terminal app or Command Prompt to do this:

```
pip install django-rest-framework
```

2. Next, open the `settings.py` file and add `rest_framework` to `INSTALLED_APPS` as shown in the following snippet:

```
INSTALLED_APPS = [  
    'reviews.adminconfig.ReviewsAdminConfig',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'reviews',]  

```

You are now ready to start using DRF to create your first simple API.

세션 자체. 이는 세션의 생애 동안 세션에 대한 정보를 유지하는 상태 저장 idx_955b0f6d 프로토콜(예: 전송 제어 프로토콜(TCP))과 대조됩니다.

따라서 RESTful 웹 서비스는 이름에서 알 수 있듯이 일련의 작업을 수행하는 데 사용되는 REST API의 모음입니다. 예를 들어, 특정 작업을 수행하기 위해 bookr 애플리케이션을 위한 REST API 세트를 개발하면 이를 RESTful 웹 서비스라고 부를 수 있습니다. 다음으로 DRF 작업을 시작하겠습니다.

DRF

DRF는 Django 프로젝트를 위한 REST API를 개발하는 데 사용할 수 있는 오픈 소스 Python 라이브러리입니다. DRF는 모든 Django 프로젝트를 위한 API 개발을 돕기 위해 필요한 기능의 대부분이 내장되어 있습니다. 이 장에서는 bookr 프로젝트를 위한 API를 개발하는 데 이를 사용할 것입니다. 다음 섹션에서는 DRF를 설치하고 구성할 것입니다.

설치 및 구성

가상 환경 설정 idx_f8439399과 함께 PyCharm에서 djangorestframework를 설치하려면 다음 단계를 따르세요:

1. 이를 수행하기 위해 터미널 앱 또는 명령 프롬프트에 다음 코드를 입력하세요:

```
pip install djangorestframework
```

2. 다음으로, settings.py 파일을 열고 INSTALLED_APPS에 rest_framework를 추가합니다. 다음 코드 조각에서:

```
INSTALLED_APPS = [  
    'reviews.adminconfig.ReviewsAdminConfig',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'reviews',]
```

이제 DRF를 사용하여 첫 번째 간단한 API를 만들 준비가 되었습니다.

Functional API views

In *Chapter 3, URL Mapping, Views, and Templates*, we learned about simple functional views that take a request and return a response. We can write similar functional views using DRF. However, note that class-based views are more commonly used and will be covered next. A functional view is created by simply adding the following decorator to a normal view, as follows:

```
from rest_framework.decorators import api_view

@api_view
def my_view(request):
    ...
```

This decorator takes the functional view and turns it into a subclass of the `APIView` DRF. It's a quick way to include an existing view as part of your API. Using what we have learned so far, we will create a simple REST API using DRF in the next section.

Exercise 12.01 – Creating a simple REST API

In this exercise, you will create your first REST API using DRF and implement an endpoint using a functional view. You will create this endpoint to view the total number of books in the database:

Note

You'll need to have DRF installed on your system to proceed with this exercise. If you haven't already installed it, make sure you refer to the *Installation and configuration* section earlier in this chapter.

1. Create `api_views.py` in the `bookr/reviews` folder.

REST API views work like Django's conventional views. We could have added the API views, along with the other views, in the `views.py` folder. However, having our REST API views in a separate file will help us maintain a cleaner code base.

2. Add the following code in `api_views.py`:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Book
```

함수형 API 뷰

3장, URL 매핑, 뷰 및 템플릿에서 우리는 요청을 받아 응답을 반환하는 간단한 함수형 `idx_5c68aeebview`를 배웠습니다. 우리는 DRF를 사용하여 유사한 함수형 뷰를 작성할 수 있습니다. 그러나 클래스 기반 뷰가 더 일반적으로 사용되며 다음에 다룰 것입니다. 함수형 뷰는 다음과 같이 일반 뷰에 다음 데코레이터를 추가하여 생성됩니다:

```
from rest_framework.decorators import api_view

@api_view
def my_view(request):
    ...
```

이 데코레이터는 함수형 뷰를 가져와 `APIView` DRF의 하위 클래스로 변환합니다. 이는 기존 뷰를 API의 일부로 포함하는 빠른 방법입니다. 지금까지 배운 내용을 바탕으로, 다음 섹션에서는 DRF를 사용하여 간단한 REST API를 생성할 것입니다.

연습 12.01 – 간단한 REST API 만들기

이 연습에서는 DRF를 사용하여 첫 번째 REST API를 만들고 기능적 뷰를 사용하여 엔드포인트를 구현합니다. 이 엔드포인트는 데이터베이스에 있는 총 책 수를 보기 위해 생성됩니다:

참고

이 연습을 진행하려면 시스템에 DRF가 설치되어 있어야 합니다. 아직 설치하지 않았다면, 이 장의 이전 섹션에서 설치 및 구성 부분을 참조하세요.

1. `bookr/reviews` 폴더에 `api_views.py`를 만듭니다.

REST API 뷰는 Django의 전통적인 뷰처럼 작동합니다. API 뷰를 다른 뷰와 함께 `views.py` 폴더에 추가할 수도 있지만, REST API 뷰를 별도의 파일에 두면 더 깔끔한 코드 베이스를 유지하는 데 도움이 됩니다.

2. `api_views.py`에 다음 코드를 추가하세요:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

.models에서 Book을 가져옵니다
```

```
@api_view(['GET'])
def first_api_view(request):
    num_books = Book.objects.count()
    return Response({"num_books": num_books})
```

The first line imports the `api_view` decorator, which will convert our functional view into one that can be used with DRF, and the second line imports `Response`, which will be used to return a response.

The view function returns a `Response` object containing a dictionary with the number of books in our database (see the highlighted part in the previous code snippet).

3. Open `bookr/reviews/urls.py` and import the `api_views` module. Then, add a new path to the `api_views` module in the URL patterns that we have developed throughout this course, as follows:

```
from . import views, api_views

urlpatterns = [path('api/first_api_view/',
                  api_views.first_api_view),
              ...
            ]
```

4. Start the Django service with the `python manage.py runserver` command and go to `http://127.0.0.1:8000/api/first_api_view/` to make your first API request. Your screen should appear as in *Figure 12.1*:

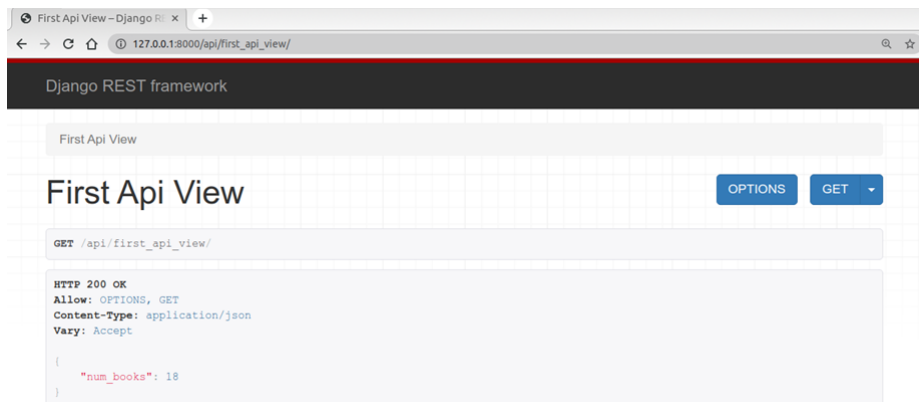


Figure 12.1: First Api View with the number of books

```
@api_view(['GET'])
def first_api_view(request):
    num_books = Book.objects.count()
    return Response({"num_books": num_books})
```

첫 번째 줄은 `api_view` 데코레이터를 가져오며, 이는 우리의 함수형 뷰를 DRF와 함께 사용할 수 있는 것으로 변환하고, 두 번째 줄은 응답을 반환하는 데 사용될 `Response`를 가져옵니다.

뷰 함수는 데이터베이스에 있는 책의 수를 포함하는 사전이 포함된 `Response` 객체를 반환합니다 (이전 코드 스니펫의 강조된 부분을 참조하십시오).

3. `bookr/reviews/urls.py`를 열고 `api_views` 모듈을 가져옵니다. 그런 다음, 이 과정에서 개발한 URL 패턴의 `api_views` 모듈에 `idx_2d06255bnewpath`를 추가합니다, 다음과 같이:

```
from . import views, api_views

urlpatterns = [path('api/first_api_view/',
                   api_views.first_api_view),
               ...
               ],
```

4. Django 서비스를 `python manage.py runserver` 명령어로 시작하고 `http://127.0.0.1:8000/api/first_api_view/`로 가서 첫 번째 API 요청을 만드세요. 당신의 화면은 그림 12.1과 같이 나타나야 합니다:

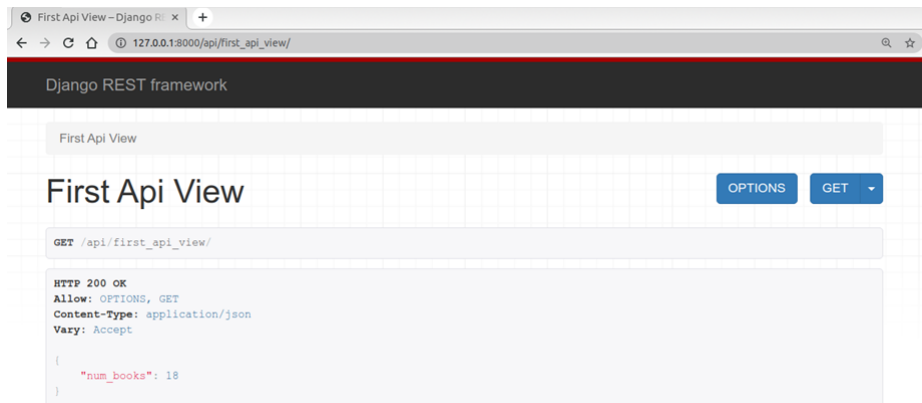


그림 12.1: 책의 수가 있는 첫 번째 API 뷰

Calling this URL endpoint made a default GET request to the API endpoint, which returned a **JavaScript Object Notation (JSON)** key-value pair (`"num_books": 0`). Also, notice how DRF provides a nice interface to view and interact with the APIs.

5. We could also use the Linux `curl` (client URL) command to send an HTTP request as follows:

```
curl http://127.0.0.1:8000/api/first_api_view/  
{ "num_books": 18 }
```

Alternatively, if you are using Windows 10, you can make an equivalent HTTP request with `curl.exe` from Command Prompt as follows:

```
curl.exe http://127.0.0.1:8000/api/first_api_view/
```

In this exercise, we learned how to create an API view using DRF and a simple functional view. We will now look at a more elegant way to convert between information stored in the database and what gets returned by our API using serializers.

Understanding serializers

By now, we are well versed in the way Django works with data in our application. Broadly, the columns of a database table are defined in a class in `models.py`, and when we access a row of the table, we are working with an instance of that class. Ideally, we often just want to pass this object to our frontend application. For example, if we wanted to build a website that displayed a list of books in our bookr app, we would want to call the `title` property of each book instance to know what string to display to the user. However, our frontend application knows nothing about Python and needs to retrieve this data through an HTTP request, which just returns a string in a specific format.

This means that any information translated between Django and the frontend (via our API) must be done by representing the information in JSON format. JSON objects look similar to a Python dictionary, except there are some extra rules that constrict the exact syntax. In our previous example in *Exercise 12.01 – Creating a simple REST API*, the API returned the following JSON object containing the number of books in our database:

```
{ "num_books": 0 }
```

But what if we wanted to return the full details about an actual book in our database with our API? DRF's serializer class helps to convert complex Python objects into formats such as

이 URL 엔드포인트를 호출하면 API 엔드포인트에 대한 기본 GET 요청이 이루어지며, JavaScript 객체 표기법(JSON) 키-값 쌍 (`"num_books": 0`)을 반환합니다. 또한, DRF가 API를 보고 상호작용할 수 있는 멋진 인터페이스를 제공하는 방법을 주목하세요.

5. 우리는 또한 Linux `curl` (클라이언트 URL) 명령을 사용하여 다음과 같이 HTTP 요청을 보낼 수 있습니다:

```
curl http://127.0.0.1:8000/api/first_api_view/
{"num_books":18}
```

또는 Windows 10을 사용하는 경우, 다음과 같이 Command Prompt에서 `curl.exe`를 사용하여 동등한 HTTP 요청을 할 수 있습니다:

```
curl.exe http://127.0.0.1:8000/api/first_api_view/
```

이번 연습에서는 DRF와 간단한 함수형 뷰를 사용하여 API 뷰를 만드는 방법을 배웠습니다. 이제 데이터베이스에 저장된 정보와 API에서 반환되는 정보를 직렬 변환기를 사용하여 변환하는 더 우아한 방법을 살펴보겠습니다.

직렬 변환기 이해하기

지금까지 우리는 Django가 애플리케이션의 데이터와 어떻게 작동하는지 잘 알고 있습니다. 대체로 데이터베이스 테이블의 열은 `models.py`의 클래스에서 정의되며, 테이블의 행에 접근할 때 우리는 해당 클래스의 인스턴스와 작업하고 있습니다. 이상적으로, 우리는 종종 이 객체를 프론트엔드 애플리케이션에 전달하고 싶습니다. 예를 들어, 우리 `bookr` 앱에서 책 목록을 표시하는 웹사이트를 만들고 싶다면, 각 책 인스턴스의 `title` 속성을 호출하여 사용자에게 어떤 문자열을 표시할지 알아야 합니다. 그러나 우리의 프론트엔드 애플리케이션은 Python에 대해 아무것도 모르며, 특정 형식의 문자열을 반환하는 HTTP 요청을 통해 이 데이터를 검색해야 합니다.

이는 Django와 프론트엔드(우리의 API를 통해) 간에 번역되는 모든 정보가 JSON 형식으로 정보를 표현해야 함을 의미합니다. JSON 객체는 Python 사전과 유사하게 보이지만, 정확한 구문을 제한하는 몇 가지 추가 규칙이 있습니다. 이전 예제인 연습 12.01 - 간단한 REST API 만들기에서 API는 데이터베이스에 있는 책의 수를 포함하는 다음 JSON 객체를 반환했습니다:

```
{"num_books": 0}
```

하지만 우리의 API로 데이터베이스에 있는 실제 책에 대한 전체 세부 정보를 반환하고 싶다면 어떻게 될까요? DRF의 직렬 변환기 클래스는 복잡한 Python 객체를 다음과 같은 형식으로 변환하는 데 도움을 줍니다.

JSON or XML so that they can be transmitted across the web using the HTTP protocol. The part of DRF that does this conversion is named `serializer`. Serializers also perform deserialization, which refers to converting serialized data back into Python objects so that the data can be processed in the application. In the following section, we will implement a way to display a list of books.

Exercise 12.02 – Creating an API view to display a list of books

In this exercise, you will use serializers to create an API that returns a list of all books present in the bookr application:

1. Create a file named `serializers.py` in the `bookr/reviews` folder. This is the file where we will place all the serializer code for the APIs.
2. Add the following code to `serializers.py`:

```
from rest_framework import serializers

class PublisherSerializer(serializers.Serializer):
    name = serializers.CharField()
    website = serializers.URLField()
    email = serializers.EmailField()

class BookSerializer(serializers.Serializer):
    title = serializers.CharField()
    publication_date = serializers.DateField()
    isbn = serializers.CharField()
    publisher = PublisherSerializer()
```

Here, the first line imports the serializers from the `rest_framework` module.

Following the imports, we have defined two classes, `PublisherSerializer` and `BookSerializer`. As the names suggest, they are serializers for the `Publisher` and `Book` models, respectively. Both these serializers are subclasses of `serializers.Serializer`, and we have defined field types for each serializer, such as `CharField`, `URLField`, `EmailField`, and so on.

Look at the `Publisher` model in the `bookr/reviews/models.py` file. The `Publisher` model has the `name`, `website`, and `email` attributes. So, to serialize a `Publisher` object,

HTTP 프로토콜을 사용하여 웹을 통해 전송될 수 있도록 JSON 또는 XML. 이 변환을 수행하는 DRF의 부분은 serializer라고 합니다. Serializer는 또한 역직렬화 (deserialization)를 수행하는데, 이는 직렬화된 데이터를 다시 Python 객체로 변환하여 애플리케이션에서 데이터를 처리할 수 있도록 하는 것을 의미합니다. 다음 섹션에서는 책 목록을 표시하는 방법을 구현할 것입니다.

연습 12.02 – 책 목록을 표시하는 API 뷰 만들기

이 idx_2ca0c186 연습에서는 serializer를 사용하여 bookr 애플리케이션에 있는 모든 책의 목록을 반환하는 API를 생성합니다:

1. bookr/reviews 폴더에 serializers.py라는 이름의 파일을 만듭니다. 이 파일은 API를 위한 모든 serializer 코드를 배치할 파일입니다.
2. serializers.py에 다음 코드를 추가하세요:

```
from rest_framework import serializers

class PublisherSerializer(serializers.Serializer):
    name = serializers.CharField()
    website = serializers.URLField()
    email = serializers.EmailField()

class BookSerializer(serializers.Serializer):
    title = serializers.CharField()
    publication_date = serializers.DateField()
    isbn = serializers.CharField()
    publisher = PublisherSerializer()
```

여기서 첫 번째 줄은 rest_framework 모듈에서 serializers를 가져옵니다.

가져온 후, 우리는 두 개의 클래스를 정의했습니다: PublisherSerializer와 BookSerializer. 이름이 암시하듯이, 이들은 각각 Publisher와 Book 모델에 대한 idx_15e8aff6 직렬 변환기입니다. 이 두 직렬 변환기는 serializers.Serializer의 하위 클래스이며, CharField, URLField, EmailField 등과 같은 각 직렬 변환기에 대한 필드 유형을 정의했습니다.

bookr/reviews/models.py 파일의 Publisher 모델을 살펴보세요. Publisher 모델은 name, website 및 email 속성을 가지고 있습니다. 따라서 Publisher 객체를 직렬화하려면,

we need the name, website, and email attributes in the serializer class, which we have defined accordingly in `PublisherSerializer`.

Similarly, for the `Book` model, we have defined `title`, `publication_date`, `isbn`, and `publisher` as the desired attributes in `BookSerializer`. Since `publisher` is a foreign key for the `Book` model, we have used `PublisherSerializer` as the serializer for the `publisher` attribute.

3. Open `bookr/reviews/api_views.py`, remove any pre-existing code, and add the following code:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

from .models import Book
from .serializers import BookSerializer

@api_view(['GET'])
def all_books(request):
    books = Book.objects.all()
    book_serializer = BookSerializer(books, many=True)
    return Response(book_serializer.data)
```

In the second line, we have imported the newly created `BookSerializer` class from the `serializers` module.

We then add a functional view, `all_books` (as in the previous exercise). This view takes a `QuerySet` containing all books and then serializes them using `BookSerializer`. The serializer class is also taking an argument, `many=True`, which indicates that the `books` object is a `QuerySet` or a list of many objects. Remember that serialization takes Python objects and returns them in a JSON serializable format, as follows:

```
[OrderedDict([('title', 'Advanced Deep Learning with Keras'),
('publication_date', '2018-10-31'), ('isbn', '9781788629416'),
('publisher', OrderedDict([('name', 'Packt Publishing'), ('website',
'https://www.packtpub.com/'), ('email', 'info@packtpub.com')]))]),
OrderedDict([('title', 'Hands-On Machine Learning for Algorithmic
Trading'), ('publication_date', '2018-12-31'), ('isbn', '9781789346411'),
```

우리는 PublisherSerializer에서 정의한 대로 직렬 변환기 클래스에 이름, 웹사이트 및 이메일 속성이 필요합니다.

마찬가지로, Book 모델에 대해 우리는 BookSerializer에서 원하는 속성으로 제목, 출판일, ISBN 및 출판사를 정의했습니다. 출판사는 Book 모델의 외래 키이므로, 우리는 출판사 속성의 직렬 변환기로 PublisherSerializer를 사용했습니다.

3. bookr/reviews/api_views.py를 열고 기존 코드를 모두 제거한 후 다음 코드를 추가하세요:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

from .models import Book
from .serializers import BookSerializer

@api_view(['GET'])
def all_books(request):
    books = Book.objects.all()
    book_serializer = BookSerializer(books, many=True)
    return Response(book_serializer.data)
```

두 번째 줄에서는 새로 생성된 BookSerializer 클래스를 theserializers 모듈에서 가져왔습니다.

그런 다음 idx_b27b2049add 기능적 뷰인 all_books를 추가했습니다(이전 연습과 같이). 이 뷰는 모든 책을 포함하는 QuerySet을 가져와서 BookSerializer를 사용하여 직렬화합니다. 직렬화 클래스는 또한 many=True라는 인수를 사용하고 있으며, 이는 books 객체가 QuerySet 또는 여러 객체의 목록임을 나타냅니다. 직렬화는 Python 객체를 가져와 JSON 직렬화 가능한 형식으로 반환한다는 점을 기억하세요. 다음과 같이:

```
[OrderedDict([('title', 'Advanced Deep Learning with Keras'),
('publication_date', '2018-10-31'), ('isbn', '9781788629416'),
('publisher', OrderedDict([('name', 'Packt Publishing'),
('website', 'https://www.packtpub.com/'), ('email', 'info@packtpub.com')]))]),
OrderedDict([('title', 'Hands-On Machine Learning for Algorithmic Trading'),
('publication_date', '2018-12-31'), ('isbn', '9781789346411'),
```

```
('publisher', OrderedDict([('name', 'Packt Publishing'), ('website',  
'https://www.packtpub.com/'), ('email', 'info@packtpub.com')])))) ...
```

4. Open `bookr/reviews/urls.py`, remove the previous example path for `first_api_view`, and add the `all_books` path, as shown in the following code:

```
from django.urls import path  
from . import views, api_views  
  
urlpatterns = [  
    path(  
        'api/all_books/',  
        api_views.all_books, name='all_books'  
    ),  
    ...  
]
```

This newly added path calls the `all_books` view function when it comes across the `api/all_books/` path in the URL.

5. Once all the code is added, run the Django server with the `python manage.py runserver` command and navigate to `http://127.0.0.1:8000/api/all_books/`. You should see something similar to *Figure 12.2*:

```
( 'publisher', OrderedDict([ ( 'name', 'Packt Publishing' ), ( 'website',
' https://www.packtpub.com/' ), ( 'email', 'info@packtpub.com' ) ] ) ] ) ...
```

4. `bookr/reviews/urls.py`를 열고, `first_api_view`에 대한 이전 예제 경로를 제거하고, 다음 코드에 표시된 대로 `all_books` 경로를 추가합니다:

```
from django.urls import path
from . import views, api_views

urlpatterns = [
    path(
        'api/all_books/', api_views.all_books,
        name='all_books'
    ),
    ...
]
```

이 새로 추가된 `idx_89adb3d7` 경로는 URL에서 `api/all_books/` 경로를 만났을 때 `all_books` 뷰 함수를 호출합니다.

5. 모든 코드가 추가되면 `python manage.py runserver` 명령어로 Django 서버를 실행하고 `http://127.0.0.1:8000/api/all_books/`로 이동합니다. Figure 12.2와 유사한 것을 볼 수 있어야 합니다:

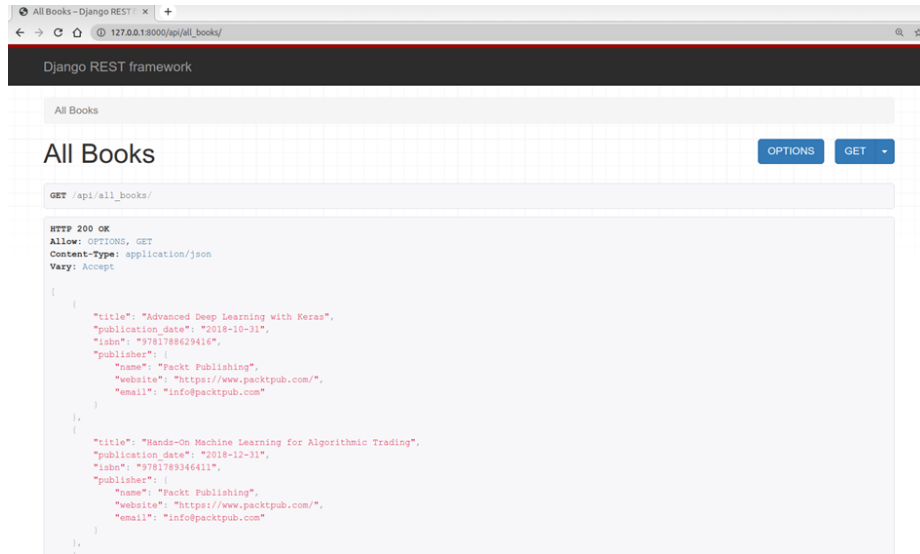


Figure 12.2: List of books shown in the `all_books` endpoint

The preceding screenshot shows that a list of all books is returned upon calling the `/api/all_books` endpoint. With that, you have successfully used a serializer to return data efficiently in your database with the help of a REST API.

Till now, we have been focusing on functional views. However, you will now learn that class-based views are more commonly used in DRF and will make your life much easier.

Class-based API views and generic views

Similar to what we learned in *Chapter 11, Advanced Templating and Class-Based Views*, we can also write class-based views for REST APIs. Class-based views are the preferred way of writing views among developers, as a lot can be achieved by writing very little code.

Just as with conventional views, DRF offers a set of generic views that make writing class-based views even simpler. Generic views are designed considering some of the most common operations needed while creating APIs. Some of the generic views offered by DRF are `ListAPIView`, `RetrieveAPIView`, and so on. In *Exercise 12.02 – Creating an API view to display a list of books*, our functional view was responsible for creating a `QuerySet` of the objects and then calling the serializer. Equivalently, we could use `ListAPIView` to do the same thing:

```
class AllBooks(ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

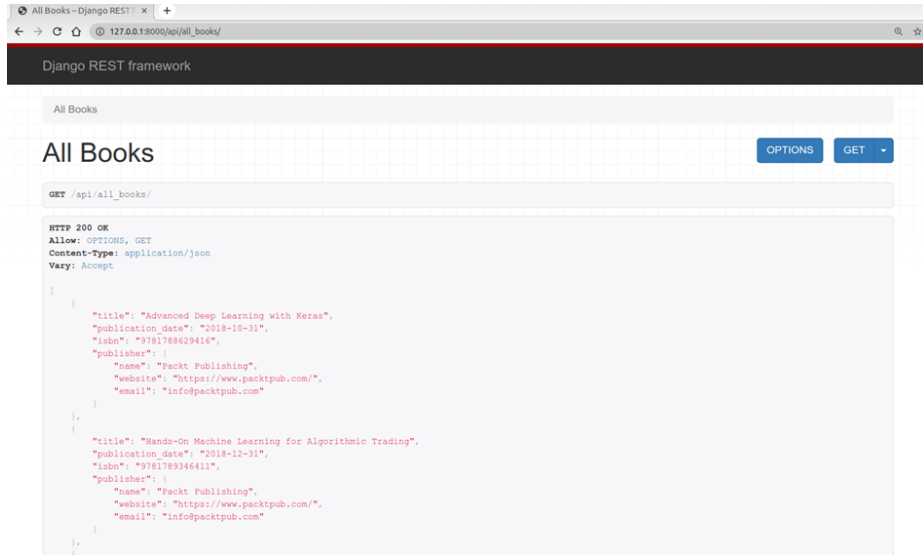


그림 12.2: `all_books` 엔드포인트에 표시된 책 목록

앞서 `idx_7ddb05cd` 스크린샷은 `/api/all_books` 엔드포인트를 호출할 때 모든 책의 목록이 반환됨을 보여줍니다. 이를 통해 REST API의 도움으로 데이터베이스에서 데이터를 효율적으로 반환하기 위해 직렬 변환기를 성공적으로 사용했습니다.

지금까지 우리는 함수 기반 뷰에 집중해 왔습니다. 그러나 이제 클래스 기반 뷰가 DRF에서 더 일반적으로 사용되며 여러분의 삶을 훨씬 더 쉽게 만들어 줄 것임을 배우게 될 것입니다.

클래스 기반 API 뷰 및 일반 뷰

11장, 고급 템플릿 및 클래스 기반 뷰에서 배운 것과 유사하게, 우리는 `idx_eaca73c1` REST API를 위한 클래스 기반 `idx_7493a550` 뷰를 작성할 수 있습니다. 클래스 기반 뷰는 개발자들 사이에서 뷰를 작성하는 선호되는 방법으로, 매우 적은 코드로 많은 것을 이룰 수 있습니다.

전통적인 뷰와 마찬가지로, DRF는 클래스 기반 뷰 작성을 더욱 간단하게 만드는 일반 뷰 세트를 제공합니다. 일반 뷰는 API를 생성할 때 필요한 가장 일반적인 몇 가지 작업을 고려하여 설계되었습니다. DRF에서 제공하는 일반 뷰 중 일부는 `ListAPIView`, `RetrieveAPIView` 등이 있습니다. 연습 12.02 - 책 목록을 표시하는 API 뷰 만들기에서 우리의 함수 기반 `idx_ffe92b8b` 뷰는 `idx_7c1f773d` 객체의 `QuerySet`을 생성하고 직렬 변환기를 호출하는 역할을 했습니다. 동등하게, 우리는 `ListAPIView`를 사용하여 동일한 작업을 수행할 수 있습니다:

```
class AllBooks(ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

Here, a `QuerySet` of objects is defined as a class attribute. Passing `queryset` through to `serializer` is handled by methods on `ListAPIView`.

Model serializers

In *Exercise 12.02 – Creating an API view to display a list of books*, our serializer was defined as follows:

```
class BookSerializer(serializers.Serializer):
    title = serializers.CharField()
    publication_date = serializers.DateField()
    isbn = serializers.CharField()
    publisher = PublisherSerializer()
```

However, our model for `Book` looks like this (note how similar the definitions of the model and serializer appear to be):

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70,
        help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        max_length=20,
        verbose_name="ISBN number of the book.")
    ...
    def __str__(self):
        return f"{self.title} ({self.isbn})"
```

We would prefer not to specify that the title must be `serializers.CharField()`. It would be easier if the serializer just looked at how `title` was defined in the model and could figure out what serializer field to use.

This is where model serializers come in. They provide shortcuts to create serializers by utilizing the definition of the fields on the model. Instead of specifying that `title` should be serialized using `CharField`, we just tell the model serializer we want to include `title`, and it uses the `CharField` serializer because the `title` field on the model is also `CharField`.

여기에서 객체의 QuerySet이 클래스 속성으로 정의됩니다. queryset을 toserializer로 전달하는 것은 ListAPIView의 메서드에 의해 처리됩니다.

모델 직렬 변환기

연습 12.02 - 책 목록을 표시하는 API 뷰 만들기에서, 우리의 직렬 변환기는 다음과 같이 정의되었습니다:

```
class BookSerializer(serializers.Serializer):
    title = serializers.CharField()
    publication_date = serializers.DateField()
    isbn = serializers.CharField()
    publisher = PublisherSerializer()
```

하지만, 우리의 Book 모델은 다음과 같습니다 (모델과 직렬화기의 정의가 얼마나 유사한지 주목하세요):

```
class Book(models.Model):
    """출판된 책입니다."""
    title = models.CharField(
        max_length=70,
        help_text="책의 제목입니다.")
    publication_date = models.DateField(
        verbose_name="책이 출판된 날짜입니다.")
    isbn = models.CharField(
        max_length=20,
        verbose_name="책의 ISBN 번호입니다.")
    ...
    def __str__(self):
        return f"{self.title} ({self.isbn})"
```

우리는 idx_930d45d8제목이 serializers.CharField()여야 한다고 명시하지 않는 것을 선호합니다. 모델에서 제목이 어떻게 정의되었는지 살펴보고 어떤 직렬화 필드를 사용할지 알아낼 수 있다면 더 쉬울 것입니다.

여기서 모델 직렬 변환기가 등장합니다. 그들은 모델의 필드 정의를 활용하여 직렬 변환기를 생성하는 단축키를 제공합니다. 제목이 CharField를 사용하여 직렬화되어야 한다고 명시하는 대신, 우리는 모델 직렬 변환기에게 제목을 포함하고 싶다고만 말하면, 모델의 제목 필드가 CharField이기 때문에 CharField 직렬 변환기를 사용합니다.

For example, suppose we wanted to create a serializer for the Contributor model in `models.py`. Instead of specifying the types of serializers that should be used for each field, we can give it a list of field names and let it figure out the rest:

```
from rest_framework import serializers

from .models import Contributor

class ContributorSerializer(serializers.ModelSerializer):

    class Meta:
        model = Contributor
        fields = ['first_names', 'last_names', 'email']
```

In the following exercise, we will see how we can use a model serializer to avoid the duplication of code in the preceding classes.

Exercise 12.03 – Creating class-based API views and model serializers

In this exercise, you will create class-based views to display a list of all books while using model serializers:

1. Open the `bookr/reviews/serializers.py` file, remove any pre-existing code, and replace it with the following code:

```
from rest_framework import serializers

from .models import Book, Publisher

class PublisherSerializer(
    serializers.ModelSerializer):

    class Meta:
        model = Publisher
        fields = ['name', 'website', 'email']

class BookSerializer(serializers.ModelSerializer):
    publisher = PublisherSerializer()
```

예를 들어, `models.py`에서 `Contributor` 모델에 대한 직렬 변환기를 만들고 싶다고 가정해 보겠습니다. 각 필드에 대해 사용해야 하는 직렬 변환기 유형을 지정하는 대신, 필드 이름 목록을 제공하고 나머지는 스스로 알아내도록 할 수 있습니다:

```
from rest_framework import serializers

from .models import Contributor

class ContributorSerializer(serializers.ModelSerializer):

    class Meta:
        model = Contributor
        fields = ['first_names', 'last_names', 'email']
```

`idx_4fdd55d8` 다음 연습에서는 모델 직렬 변환기를 사용하여 앞서 언급한 클래스에서 코드 중복을 피하는 방법을 살펴보겠습니다.

연습 12.03 – 클래스 기반 API 뷰 및 모델 직렬 변환기 만들기

이 `idx_6044c64f` 연습에서는 모델 직렬 변환기를 사용하여 모든 책의 목록을 표시하는 클래스 기반 뷰를 생성합니다:

1. `bookr/reviews/serializers.py` 파일을 열고 기존 코드를 제거한 후 다음 코드를 추가합니다:

```
from rest_framework import serializers

from .models import Book, Publisher

class PublisherSerializer(
    serializers.ModelSerializer):

    class Meta:
        model = Publisher
        fields = ['name', 'website', 'email']

class BookSerializer(serializers.ModelSerializer):
    publisher = PublisherSerializer()
```

```
class Meta:
    model = Book
    fields = ['title', 'publication_date', 'isbn',
              'publisher']
```

Here, we have included two model serializer classes, `PublisherSerializer` and `BookSerializer`. Both these classes inherit the `serializers.ModelSerializer` parent class. We do not need to specify how each field gets serialized; instead, we can simply pass a list of field names, and the field types are inferred from the definition in `models.py`.

Although mentioning the field inside `fields` is sufficient for the model serializer, in certain special cases, such as this one, we may have to customize the field since the `publisher` field is a foreign key. Hence, we must use `PublisherSerializer` to serialize the `publisher` field.

2. Next, open `bookr/reviews/api_views.py`, remove any pre-existing code, and add the following code:

```
from rest_framework import generics

from .models import Book
from .serializers import BookSerializer

class AllBooks(generics.ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

Here, we use the DRF class-based `ListAPIView` view instead of a functional view. This means that the list of books is defined as a class attribute, and we do not have to write a function that directly handles the request and calls the serializer. The book serializer from the previous step is also imported and assigned as an attribute of this class.

Open the `bookr/reviews/urls.py` file and modify the `/api/all_books` API path to include the new class-based view as follows:

```
urlpatterns = [
    path(
        'api/all_books/',
        api_views.AllBooks.as_view(),
```

```
class Meta:
    model = Book
    fields = ['title', 'publication_date', 'isbn',
              'publisher']
```

여기에서는 두 개의 모델 직렬 변환기 클래스인 `PublisherSerializer`와 `BookSerializer`를 포함했습니다. 이 두 클래스는 모두 `serializers.ModelSerializer` 부모 클래스를 상속합니다. 각 필드가 어떻게 직렬화 되는지를 명시할 필요는 없으며, 대신 필드 이름의 목록을 간단히 전달하면 필드 유형은 `models.py`의 정의에서 유추됩니다.

필드 내부에 `fields`를 언급하는 것만으로는 모델 직렬 변환기에 충분하지만, 이와 같은 특정 특수 경우에는 필드를 사용자 정의해야 할 수도 있습니다. 왜냐 하면 `publisher` 필드는 외래 키이기 때문입니다. 따라서 `publisher` 필드를 직렬화 하기 위해 `PublisherSerializer`를 사용해야 합니다.

2. 다음으로 `bookr/reviews/api_views.py`를 열고 기존 코드를 모두 제거한 후 다음 코드를 추가하세요:

```
from rest_framework import generics

from .models import Book
from .serializers import BookSerializer

class AllBooks(generics.ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

여기서 우리는 기능적 뷰 대신 DRF 클래스 기반 `ListAPIView` 뷰를 사용합니다. 이는 책 목록이 클래스 속성으로 정의되며, 요청을 직접 처리하고 직렬 변환기를 호출하는 함수를 작성할 필요가 없음을 의미합니다. 이전 단계의 책 직렬 변환기 도 가져와서 이 클래스의 속성으로 할당됩니다.

`bookr/reviews/urls.py` 파일을 열고 `/api/all_books` API 경로를 다음과 같이 새 클래스 기반 뷰를 포함하도록 수정합니다:

```
urlpatterns = [
    path(
        'api/all_books/',
        api_views.AllBooks.as_view(),
```

```

        name='all_books'
    ),
    ...
]

```

Since we are using a class-based view, we have to use the class name along with the `as_view()` method.

3. Once all the preceding modifications are completed, wait till the Django service restarts or start the server with the `python manage.py runserver` command, and then open the API at `http://127.0.0.1:8000/api/all_books/` in the web browser. You should see something like *Figure 12.3*:

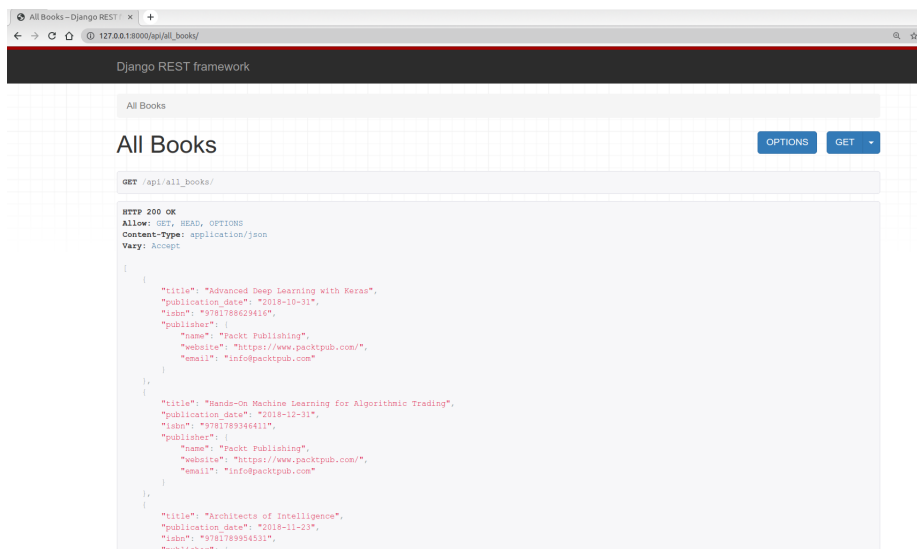


Figure 12.3: List of books shown in the `all_books` endpoint

Similar to what we saw in *Exercise 12.02 – Creating an API view to display a list of books*, this is a list of all books present in the book review application. In this exercise, we used model serializers to simplify our code and the generic class-based `ListAPIView` view to return a list of the books in our database. In the next section, you will implement an API endpoint to return a list of top contributors.

```

        name='all_books'
    ),
    ...

```

우리는 클래스 기반 뷰를 사용하고 있기 때문에, 클래스 이름과 함께 `as_view()` 메서드를 사용해야 합니다.

3. 모든 이전 수정이 완료되면 Django 서비스가 재시작될 때까지 기다리거나 `python manage.py runserver` 명령어로 서버를 시작한 후 웹 브라우저에서 `http://127.0.0.1:8000/api/all_books/` 에 API를 열어보세요. 다음과 같은 화면이 나타날 것입니다(그림 12.3 참조):

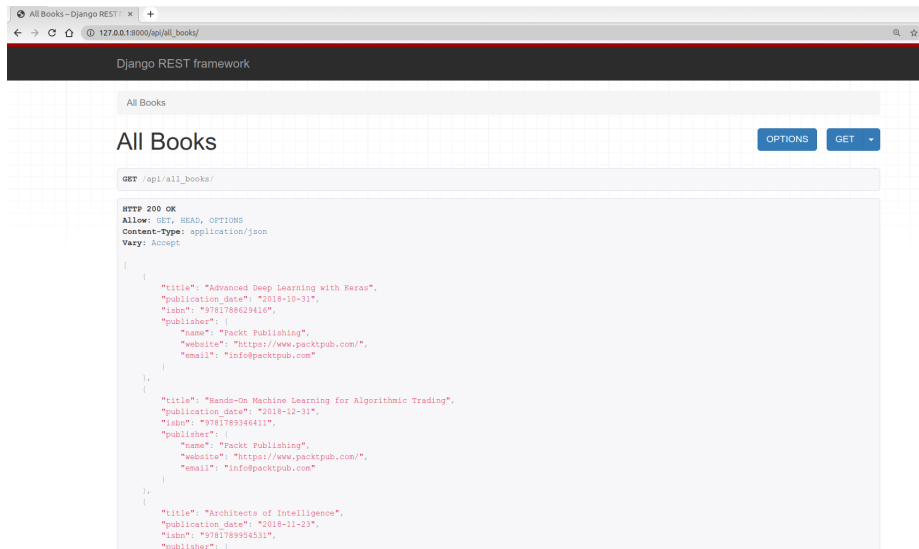


그림 12.3: `all_books` 엔드포인트에 표시된 책 목록

운동 12.02 – 책 목록을 표시하는 API 뷰 만들기에서 본 것과 유사하게, 이것은 책 리뷰 `idx_9f621908` 애플리케이션에 존재하는 모든 책의 목록입니다. 이 연습에서는 모델 시리얼라이저를 사용하여 코드를 단순화하고, 제네릭 클래스 기반 `ListAPIView` 뷰를 사용하여 데이터베이스에 있는 책 목록을 반환했습니다. 다음 섹션에서는 상위 기여자 목록을 반환하는 API 엔드포인트를 구현할 것입니다.

Activity 12.01 – Creating an API endpoint for a top contributors page

Imagine that your team decides to create a web page that displays the top contributors (authors, coauthors, and editors) in your database. They decide to enlist the services of an external developer to create an app in ReactJS. To integrate with the Django backend, the developer will need an endpoint that provides the following:

- A list of all contributors in the database
- For each contributor, a list of all books they contributed to
- For each contributor, the number of books they contributed to
- For each book they contributed to, their role in the book

The final API view should look like this:

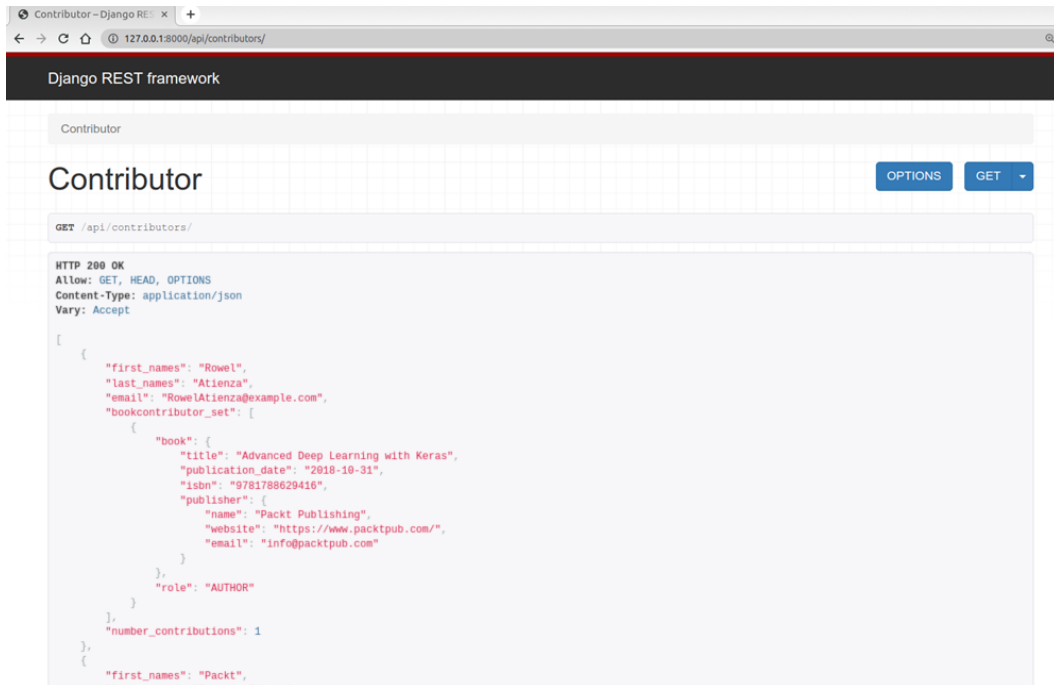


Figure 12.4: The top contributors endpoint

To perform this task, execute the following steps:

1. Add a method to the Contributor class that returns the number of contributions made.
2. Add ContributionSerializer, which serializes the BookContribution model.

활동 12.01 – 상위 기여자 페이지를 위한 API 엔드포인트 생성하기

당신의 팀이 데이터베이스에 있는 상위 기여자(저자, 공동 저자 및 편집자)를 표시하는 웹 페이지를 만들기로 결정했다고 상상해 보십시오. 그들은 ReactJS로 앱을 만들기 위해 외부 개발자의 서비스를 요청하기로 결정했습니다. Django 백엔드와 통합하기 위해 개발자는 다음을 제공하는 엔드포인트가 필요합니다:

- 데이터베이스에 있는 모든 기여자의 목록
- 각 기여자에 대해 그들이 기여한 모든 책의 목록 각 기여
- 자가 기여한 책의 수
- 그들이 기여한 각 책에 대해 책에서의 역할 최종

API 뷰는 다음과 같아야 합니다:

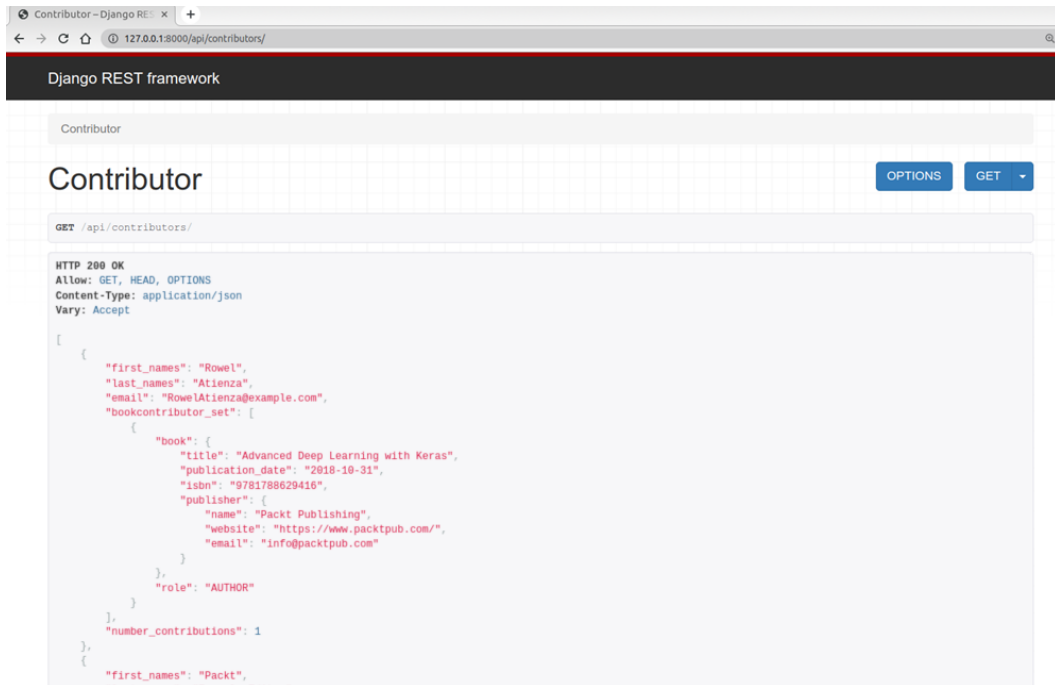


그림 12.4: 주요 기여자 엔드포인트

idx_01cf4d37이 작업을 수행하려면, idx_ba79d14f 다음 단계를 실행하십시오:

1. 기여자 클래스에 기여 수를 반환하는 메서드를 추가합니다. BookContribution 모
2. 델을 직렬화하는 ContributionSerializer를 추가합니다.

3. Add `ContributorSerializer`, which serializes the `Contributor` model.
4. Add `ContributorView`, which uses `ContributorSerializer`.
5. Add a pattern to `urls.py` to enable access to `ContributorView`.

Simplifying the code using ViewSets

We have seen how we can optimize our code and make it more concise using class-based generic views. **ViewSets** and **routers** help us further simplify our code. As the name indicates, ViewSets are a set of views represented in a single class. For example, we used the `AllBooks` view to return a list of all books in the application and the `BookDetail` view to return the details of a single book. Using ViewSets, we could combine both these classes into a single class.

DRF also provides a class named `ModelViewSet`. This class not only combines the two views mentioned in the preceding discussion (list and detail) but also allows you to create, update, and delete model instances. The code needed to implement all this functionality could be as simple as specifying the serializer and `QuerySet`. For example, a view that allows you to manage all these actions for your user model could be defined as tersely as the following:

```
class UserViewSet(viewsets.ModelViewSet):
    serializer_class = UserSerializer
    queryset = User.objects.all()
```

Lastly, DRF provides a `ReadOnlyModelViewSet` class. This is a simpler version of the preceding `ModelViewSet` class. It is identical, except that it only allows you to list and retrieve specific users. You cannot create, update, or delete records. In the following section, we will learn about routers.

URL configuration using routers and ViewSets

Routers, when used along with a `ViewSet`, automatically create the required URL endpoints for the `ViewSet`. This is because a single `ViewSet` is accessed at different URLs. For example, in the preceding `UserViewSet` class, you would access a list of users at the `/api/users/` URL, and a specific user record at the `/api/users/123` URL, where 123 is the primary key of that user record. Here is a simple example of how you might use a router in the context of the previously defined `UserViewSet` class:

```
from rest_framework import routers
router = routers.SimpleRouter()
```

3. Contributor 모델을 직렬화하는 ContributorSerializer를 추가합니다.
4. ContributorSerializer를 사용하는 ContributorView를 추가합니다.
5. ContributorView에 대한 접근을 가능하게 하기 위해 urls.py에 패턴을 추가합니다.

ViewSets를 사용하여 코드를 단순화하기

우리는 idx_dc97a614코드를 최적화하고 idx_f9675c2e클래스 기반의 일반 뷰를 사용하여 더 간결하게 만드는 방법을 보았습니다. ViewSets와 라우터는 idx_f4305c2d코드를 더 단순화하는 데 도움을 줍니다. 이름에서 알 수 있듯이 idx_29e587c3ViewSets는 단일 클래스에 표현된 뷰의 집합입니다. 예를 들어, 우리는 AllBooks 뷰를 사용하여 애플리케이션의 모든 책 목록을 반환하고 BookDetail 뷰를 사용하여 단일 책의 세부 정보를 반환했습니다. ViewSets를 사용하면 이 두 클래스를 단일 클래스로 결합할 수 있습니다.

DRF는 ModelViewSet이라는 클래스를 제공합니다. 이 클래스는 앞서 언급한 두 뷰(목록 및 세부 정보)를 결합할 뿐만 아니라 모델 인스턴스를 생성, 업데이트 및 삭제할 수 있도록 합니다. 이러한 모든 기능을 구현하는 데 필요한 코드는 직렬화기와 QuerySet을 지정하는 것만큼 간단할 수 있습니다. 예를 들어, 사용자 모델에 대한 모든 작업을 관리할 수 있는 뷰는 다음과 같이 간결하게 정의될 수 있습니다:

```
class UserViewSet
    (viewsets.ModelViewSet):serializer_class = UserSerializerqueryset = User.objects.all()
```

마지막으로, DRF idx_67987f47는 ReadOnlyModelViewSet 클래스를 제공합니다. 이 idx_940885b2는 이전 ModelViewSet 클래스의 더 간단한 버전입니다. 특정 사용자를 나열하고 검색하는 것만 허용되며, 레코드를 생성, 업데이트 또는 삭제할 수 없습니다. 다음 섹션에서는 라우터에 대해 배울 것입니다.

라우터 및 ViewSets를 사용한 URL 구성

라우터는 ViewSet과 함께 사용될 때 자동으로 ViewSet에 필요한 URL 엔드포인트를 idx_ad836fdf 생성합니다. 이는 단일 ViewSet이 서로 다른 URL에서 idx_bf6a2d1a 접근되기 때문입니다. 예를 들어, 이전 UserViewSet 클래스에서는 /api/users/ URL에서 사용자 목록에 접근하고, /api/users/123 URL에서 특정 사용자 레코드에 접근할 수 있습니다. 여기 이전에 정의된 UserViewSet 클래스의 맥락에서 라우터를 사용하는 간단한 예가 있습니다:

```
from rest_framework import routers
router = routers.SimpleRouter()
```

```
router.register(r'users', UserViewSet)
urlpatterns = router.urls
```

Now, let's try combining the concepts of routers and ViewSets in a simple exercise.

Exercise 12.04 – Using ViewSets and routers

In this exercise, we will combine the existing views to create a ViewSet and create the required routing for the ViewSet:

1. Open the `bookr/reviews/serializers.py` file, remove the pre-existing code, and add the following code snippet:

```
from django.contrib.auth.models import User
from django.utils import timezone
from rest_framework import serializers
from rest_framework.exceptions import
    NotAuthenticated, PermissionDenied
from .models import Book, Publisher, Review
from .utils import average_rating
class PublisherSerializer(serializers.ModelSerializer):
```

You can find the complete code snippet in the `Chapter12` folder in this book's GitHub repo. To access the repository link, follow the steps in the *"Download the example code files"* section in the *Preface*.

Here, we added two new fields to `BookSerializer`, namely `reviews` and `rating`. The interesting thing about these fields is that the logic behind them is defined as a method on the serializer itself. This is why we use the `serializers.SerializerMethodField` type to set the serializer class attributes.

2. Open the `bookr/reviews/api_views.py` file, remove the pre-existing code, and add the following:

```
from rest_framework import viewsets
from rest_framework.pagination import LimitOffsetPagination

from .models import Book, Review
from .serializers import (BookSerializer, ReviewSerializer)

class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
```

```
router.register(r'users', UserViewSet)
urlpatterns = router.urls
```

이제 라우터와 ViewSet의 개념을 간단한 연습에서 결합해 보겠습니다.

연습 12.04 – ViewSets와 라우터 사용하기

이 idx_90cd676cexercise에서는 기존 뷰를 결합하여 ViewSet을 만들고 idx_db7a82a1ViewSet에 필요한 라우팅을 생성합니다:

1. bookr/reviews/serializers.py 파일을 열고 기존 코드를 제거한 후 다음 코드 스니펫을 추가합니다:

```
from django.contrib.auth.models import User
from django.utils import timezone
from rest_framework import serializers
from rest_framework.exceptions import NotAuthenticated,
    PermissionDenied
from .models import Book, Publisher, Review
from .utils import average_rating
class PublisherSerializer(serializers.ModelSerializer):
```

이 책의 GitHub 저장소의 Chapter12 폴더에서 전체 코드 스니펫을 찾을 수 있습니다. 저장소 링크에 접근하려면 서문에 있는 "예제 코드 파일 다운로드" 섹션의 단계를 따르세요.

여기에서 우리는 BookSerializer에 reviews와 rating이라는 두 개의 새로운 필드를 추가했습니다. 이 필드의 흥미로운 점은 그 뒤에 있는 로직이 직렬 변환기 자체의 메서드로 정의된다는 것입니다. 그래서 우리는 직렬 변환기 클래스 속성을 설정하기 위해 serializers.SerializerMethodField 유형을 사용합니다.

2. bookr/reviews/api_views.py 파일을 열고 기존 코드를 제거한 후 다음을 추가하세요:

```
from rest_framework import viewsets
from rest_framework.pagination import LimitOffsetPagination

from .models import Book, Review
from .serializers import (BookSerializer, ReviewSerializer)

class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
```

```
serializer_class = BookSerializer

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.order_by(
        '-date_created')
    serializer_class = ReviewSerializer
    pagination_class = LimitOffsetPagination
    authentication_classes = []
```

Here, we have removed the AllBook and BookDetail views and replaced them with BookViewSet and ReviewViewSet. In the first line, we import the ViewSets module from rest_framework. The BookViewSet class is a subclass of ReadOnlyModelViewSet, which ensures that the views are only used for the GET operation.

3. Next, open the bookr/reviews/urls.py file, remove the first two URL patterns starting with api/, and then add the following (highlighted) code:

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter

from . import views, api_views

router = DefaultRouter()
router.register(r'books', api_views.BookViewSet)
router.register(r'reviews', api_views.ReviewViewSet)

urlpatterns = [
    path(
        'api/', include((router.urls, 'api'))
    ),
    path(
        'books/', views.book_list, name='book_list'),
    ...
    path(
        'publishers/new/', views.publisher_edit,
        name='publisher_create'
    )
]
```

```

        serializer_class = BookSerializer

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.order_by(
        '-date_created')
    serializer_class = ReviewSerializer
    pagination_class = LimitOffsetPagination
    authentication_classes = []

```

여기에서 우리는 idx_470e84ac의 AllBook 및 BookDetail 뷰를 제거하고 idx_4a535f7d를 BookViewSet 및 ReviewViewSet으로 교체했습니다. 첫 번째 줄에서는 rest_framework에서 ViewSets 모듈을 가져옵니다. BookViewSet 클래스는 ReadOnlyModelViewSet의 하위 클래스이며, 이는 뷰가 GET 작업에만 사용되도록 보장합니다.

3. 다음으로, bookr/reviews/urls.py 파일을 열고 api/로 시작하는 처음 두 개의 URL 패턴을 제거한 다음 다음(강조된) 코드를 추가합니다:

```

from django.urls import path, include
from rest_framework.routers import DefaultRouter

from . import views, api_views

router = DefaultRouter()
router.register(r'books', api_views.BookViewSet)
router.register(r'reviews', api_views.ReviewViewSet)

urlpatterns = [
    path(
        'api/', include((router.urls, 'api'))
    ),
    path(
        'books/', views.book_list, name='book_list'),
    ...
    path(
        'publishers/new/', views.publisher_edit,
        name='publisher_create'
    )
]

```

Here, we have combined the `all_books` and `book_detail` paths into a single path called `books`. We have also added a new endpoint under the `reviews` path, which we will need in a later chapter.

We start by importing the `DefaultRouter` class from `rest_framework.routers`. Then, we create a router object using the `DefaultRouter` class and then register the newly created `BookViewSet` and `ReviewViewSet` classes, as can be seen from the highlighted code. This ensures that `BookViewSet` is invoked whenever the API has the `/api/books/` path.

4. Save all the files, and once the Django service restarts (or you start it manually with the `python manage.py runserver` command), go to the `http://127.0.0.1:8000/api/books/` URL to get a list of all the books. You should see the following view in the API explorer:

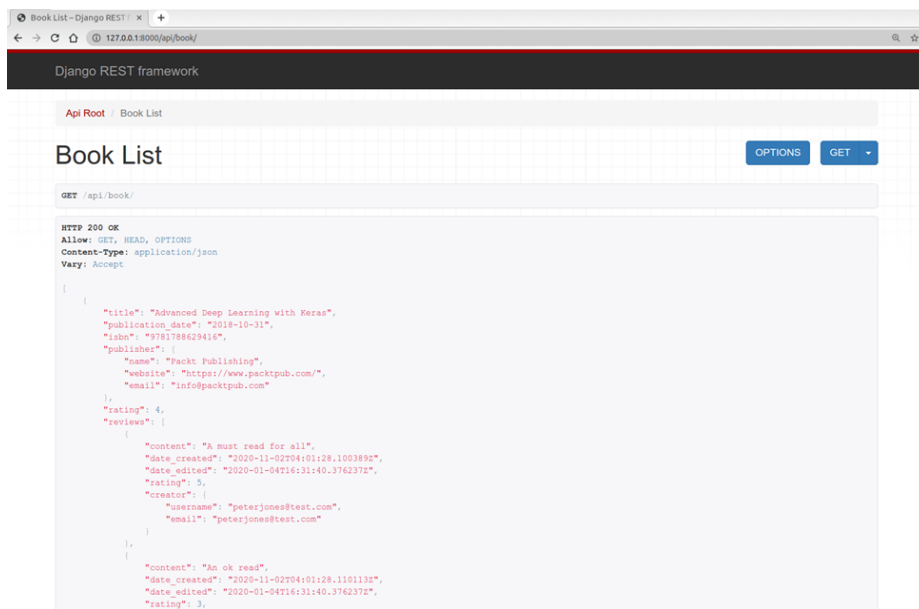


Figure 12.5: Book list at the `/api/books` path

5. You can also access the details for a specific book using the `http://127.0.0.1:8000/api/books/1/` URL. In this case, it will return details for the book with a primary key of 1 (if it exists in your database):

여기에서 우리는 all_books와 book_detail 경로를 하나의 경로인 books로 결합했습니다. 또한, 나중 장에서 필요할 reviews 경로 아래에 새로운 엔드포인트를 추가했습니다.

우리는 rest_framework.routers에서 DefaultRouter 클래스를 가져오는 것으로 시작합니다. 그런 다음 DefaultRouter 클래스를 사용하여 라우터 객체를 생성하고, 새로 생성된 BookViewSet 및 ReviewViewSet 클래스를 등록합니다. 이는 강조된 코드에서 볼 수 있습니다. 이렇게 하면 API가 /api/bookspi 경로를 가질 때마다 BookViewSet이 호출됩니다.

4. 모든 파일을 저장하고 Django 서비스가 재시작되면(또는 python manage.py runserver 명령으로 수동으로 시작하면) `http://127.0.0.1:8000/api/books/` URL로 이동하여 모든 책의 목록을 가져옵니다. API 탐색기에서 다음 뷰를 볼 수 있어야 합니다:

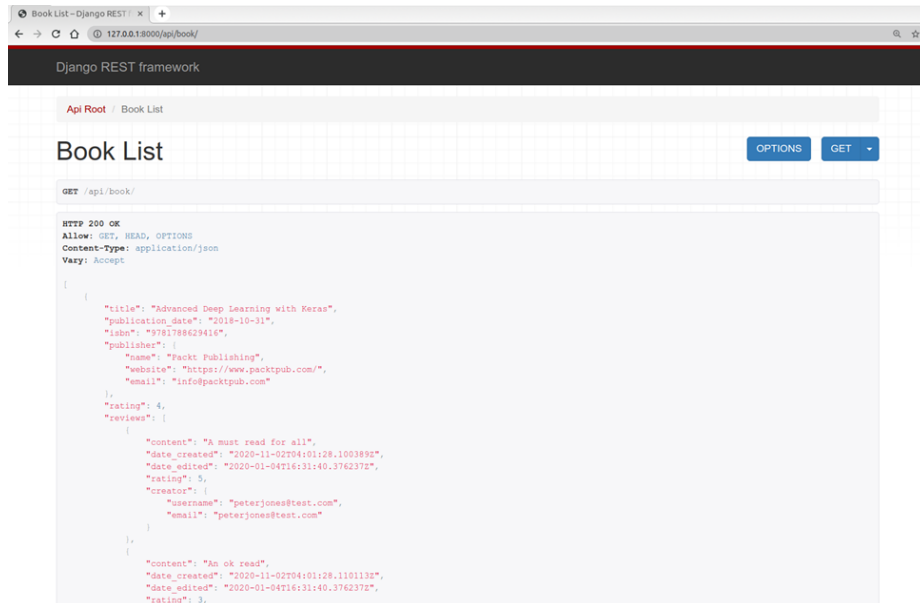


그림 12.5: /api/books 경로의 책 목록

5. `http://127.0.0.1:8000/api/books/1/` URL을 사용하여 특정 책의 세부정보에도 접근할 수 있습니다. 이 경우, 데이터베이스에 존재하는 경우 기본 키가 1인 책의 세부정보를 반환합니다:

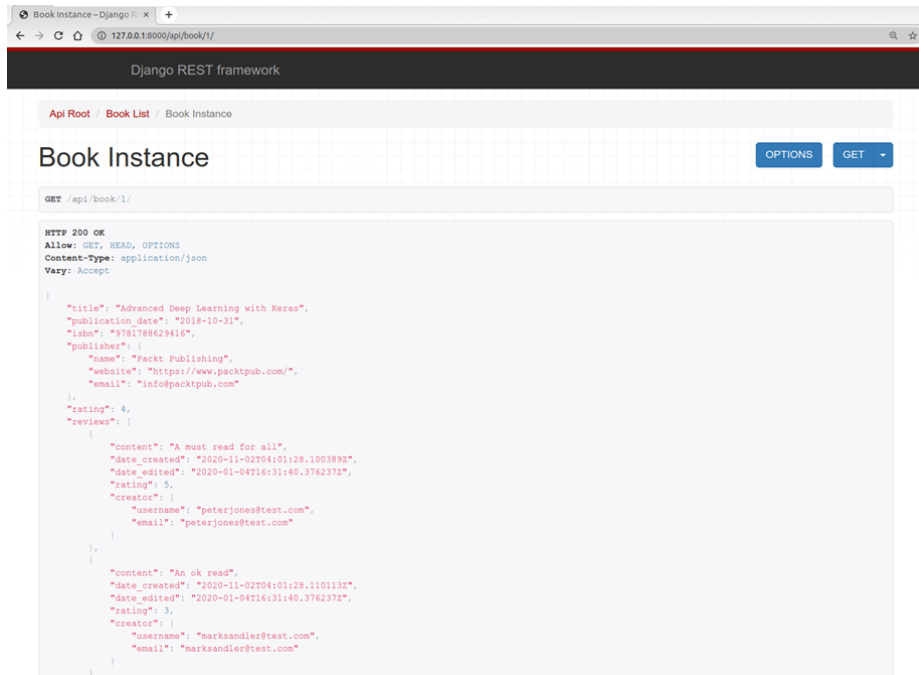


Figure 12.6: Book details for "Advanced Deep Learning with Keras"

In this exercise, we saw how we can use ViewSets and routers to combine list and detail views into a single ViewSet. Using ViewSets makes our code more consistent and idiomatic, making it easier to collaborate with other developers. This becomes particularly important when integrating with a separate frontend application. In the next section, we will learn in brief about different types of authentication and implement token-based authentication for the book review application.

Implementing authentication

As we learned in *Chapter 9, Sessions and Authentication*, it is important to authenticate the users of our application. It is good practice to only allow those users who have registered in the application to log in and access information from the application. Similarly, for REST APIs, we also need to design a way to authenticate and authorize users before any information is passed on. For example, suppose Facebook's website makes an API request to get a list of all comments for a post. If they did not have authentication on this endpoint, you could use it to get comments for any post you want programmatically. They obviously don't want to allow this, so some sort of authentication needs to be implemented.

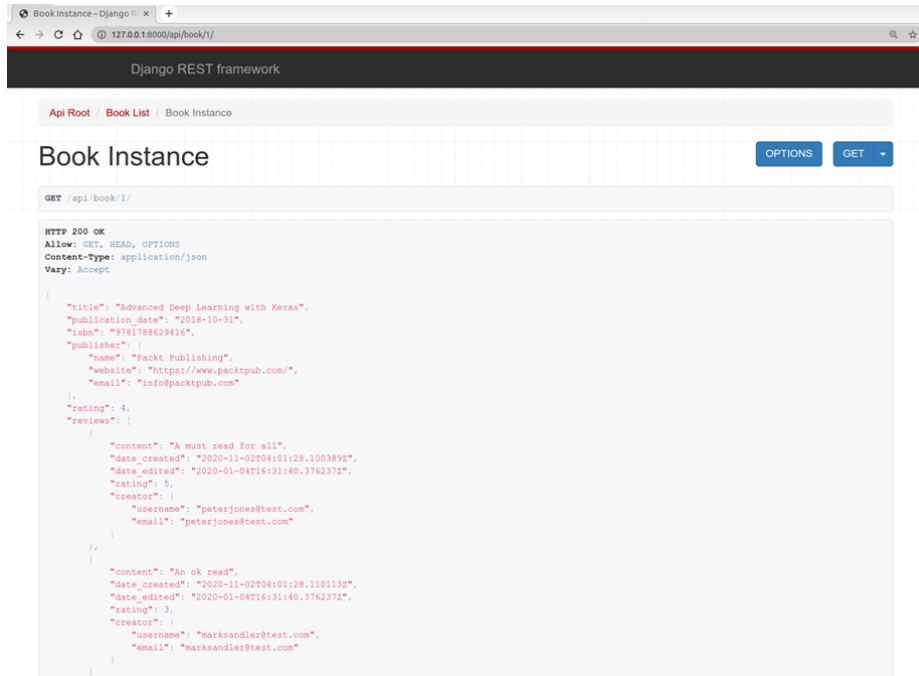


그림 12.6: "Keras로 배우는 고급 딥러닝"의 책 세부정보

이번 연습에서는 ViewSets와 라우터를 사용하여 목록 및 세부 정보 보기를 단일 ViewSet으로 결합하는 방법을 보았습니다. ViewSets를 사용하면 코드가 더 일관되고 관용적이 되어 다른 개발자와 협업하기가 더 쉬워집니다. 이는 별도의 프론트엔드 애플리케이션과 통합할 때 특히 중요해집니다. 다음 섹션에서는 다양한 유형의 인증에 대해 간략히 배우고 책 리뷰 애플리케이션을 위한 토큰 기반 인증을 구현할 것입니다.

인증 구현하기

9장 세션 및 인증에서 배운 바와 같이, 우리 애플리케이션의 사용자를 인증하는 것이 중요합니다. 애플리케이션에 등록된 사용자만 로그인하고 애플리케이션의 정보에 접근할 수 있도록 허용하는 것이 좋은 관행입니다. 마찬가지로 REST API의 경우, 정보를 전달하기 전에 사용자를 인증하고 권한을 부여하는 방법을 설계해야 합니다. 예를 들어, Facebook 웹사이트가 게시물에 대한 모든 댓글 목록을 가져오기 위해 API 요청을 한다고 가정해 보겠습니다. 이 엔드포인트에 인증이 없다면, 원하는 게시물에 대한 댓글을 프로그래밍 방식으로 가져오는 데 사용할 수 있습니다. 그들은 분명히 이를 허용하고 싶지 않으므로 어떤 형태의 인증이 구현되어야 합니다.

There are different authentication schemes, such as **basic authentication**, **session authentication**, **token authentication**, **remote user authentication**, and various third-party authentication solutions. For the scope of this chapter and for our bookr application, we will use token authentication.

Note

For further reading on all the authentication schemes, please refer to the official documentation at <https://www.django-rest-framework.org/api-guide/authentication>.

Token-based authentication

Token-based authentication works by generating a unique token for a user in exchange for the user's username and password. Once the token is generated, it will be stored in the database for further reference and will be returned to the user upon every login.

This token is unique to the user, and they can then use it to authorize every API request they make. Token-based authentication eliminates the need to pass the username and password on every request. It is much safer and is best suited to client-server communication, such as a JavaScript-based web client interacting with the backend application via REST APIs.

An example of this would be a ReactJS or AngularJS application interacting with a Django backend via REST APIs.

The same architecture can be used if you are developing a mobile application to interact with the backend server via REST APIs; for instance, an Android or iOS application interacting with a Django backend via REST APIs.

Exercise 12.05 – Implementing token-based authentication for bookr APIs

In this exercise, you will implement token-based authentication for the bookr application's APIs:

1. Open the bookr/settings.py file and add `rest_framework.authtoken` to `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',
```

다양한 idx_572dd604 인증 idx_d5016340 방식이 있으며, 기본 인증, 세션 인증, 토큰 인증, 원격 사용자 인증 및 다양한 idx_e5058f6f 제3자 인증 솔루션이 있습니다. 이 장의 범위와 우리의 bookr 애플리케이션을 위해, 우리는 토큰 인증을 사용할 것입니다.

참고

모든 인증 방식에 대한 추가 idx_b9717ea4 읽기를 원하시면 공식 문서를 참조하시기 바랍니다.

<https://www.django-rest-framework.org/api-guide/authentication>에서 문서 확인.

토큰 기반 인증

토큰 기반 인증은 사용자의 사용자 이름과 비밀번호를 교환하여 사용자에게 고유한 토큰을 생성하는 방식으로 작동합니다. 토큰이 생성되면, 데이터베이스에 저장되어 이후 참조를 위해 사용되며, 매 로그인 시 사용자에게 반환됩니다.

이 토큰은 사용자에게 고유하며, 사용자는 이를 사용하여 자신이 만드는 모든 API 요청을 승인할 수 있습니다. 토큰 기반 인증은 매 요청마다 사용자 이름과 비밀번호를 전달할 필요를 없애줍니다. 이는 훨씬 안전하며, REST API를 통해 백엔드 애플리케이션과 상호작용하는 JavaScript 기반 웹 클라이언트와 같은 클라이언트-서버 통신에 가장 적합합니다.

이의 예로는 ReactJS 또는 AngularJS 애플리케이션이 REST API를 통해 Django 백엔드와 상호작용하는 경우가 있습니다.

같은 idx_84b3ef27architecture는 REST API를 통해 백엔드 서버와 상호작용하는 모바일 애플리케이션을 개발하는 경우 사용할 수 있습니다. 예를 들어, Django 백엔드와 REST API를 통해 상호작용하는 Android 또는 iOS 애플리케이션입니다.

연습 12.05 – bookr API에 대한 토큰 기반 인증 구현

이 idx_824c069b연습에서는 bookr 애플리케이션의 API에 대한 토큰 기반 인증을 구현합니다:

1. bookr/settings.py 파일을 열고 INSTALLED_APPS에 rest_framework.authtoken을 추가합니다:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
```

```
'rest_framework.authtoken',  
'reviews']
```

2. Since the authtoken app has associated database changes, run the migrate command in the command line/terminal as follows:

```
python manage.py migrate
```

3. Open the bookr/reviews/api_views.py file, remove any pre-existing code, and replace it with the following:

```
from django.contrib.auth import authenticate  
from rest_framework import viewsets  
from rest_framework.authentication import (  
    TokenAuthentication)  
from rest_framework.authtoken.models import Token  
from rest_framework.pagination import (  
    LimitOffsetPagination)  
from rest_framework.permissions import IsAuthenticated  
from rest_framework.response import Response  
from rest_framework.status import (  
    HTTP_404_NOT_FOUND, HTTP_200_OK)  
from rest_framework.views import APIView
```

You can find the complete code for this file You can find the complete code snippet in the Chapter12 folder in this book's GitHub repo.

Here, we have defined a view called Login. The purpose of this view is to allow a user to get (or create if it does not already exist) a token that they can use to authenticate with the API.

We override the post method of this view because we want to customize the behavior when a user sends us data (that is, their login details). First, we use the authenticate method from Django's auth library to check whether the username and password are correct. If they are correct, then we will have a user object. If not, we return an HTTP 404 error. If we do have a valid user object, then we simply get or create a token and return it to the user.

4. Next, let's add an authentication class to BookViewSet. This means that when a user tries to access this ViewSet, it will require them to authenticate using token-based

```
'rest_framework.authtoken',
리뷰들
```

2. authtoken 앱에 관련된 데이터베이스 변경 사항이 있으므로 migrate 명령어를 실행하세요. 명령줄/터미널에서 다음과 같이:

```
python manage.py migrate
```

3. bookr/reviews/api_views.py 파일을 열고, 기존 코드를 모두 제거한 후 다음 내용으로 교체하세요:

```
from django.contrib.auth import authenticate
from rest_framework import viewsets
from rest_framework.authentication import
    (TokenAuthentication)
from rest_framework.authtoken.models import Token
from rest_framework.pagination import
    (LimitOffsetPagination)
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.status import
    (HTTP_404_NOT_FOUND,
    HTTP_200_OK)
from rest_framework.views import APIView
```

이 파일의 전체 코드를 찾을 수 있습니다. 이 책의 GitHub 저장소의 Chapter12 폴더에서 전체 코드 스니펫을 찾을 수 있습니다.

여기에서 우리는 Login이라는 뷰를 정의했습니다. 이 뷰의 목적은 사용자가 API와 인증하는 데 사용할 수 있는 토큰을 얻거나(존재하지 않는 경우 생성) 할 수 있도록 하는 것입니다.

우리는 사용자가 데이터를 보낼 때(즉, 로그인 세부정보) 동작을 사용자 정의하고 싶기 때문에 이 뷰의 post 메서드를 재정의합니다. 먼저, Django의 auth 라이브러리에서 authenticate 메서드를 사용하여 사용자 이름과 비밀번호가 올바른지 확인합니다. 올바르다면 사용자 객체를 가지게 됩니다. 그렇지 않으면 HTTP 404 오류를 반환합니다. 유효한 사용자 객체가 있다면, 우리는 단순히 토큰을 가져오거나 생성하여 사용자에게 반환합니다.

4. 다음으로, BookViewSet에 인증 클래스를 추가합니다. 이는 사용자가 이 ViewSet에 접근하려고 할 때, idx_a7a22e37 토큰 기반 인증을 요구한다는 것을 의미합니다.

authentication. Note that it's possible to include a list of different accepted authentication methods, not just one. We also add the `permissions_classes` attribute, which just uses DRF's built-in class that checks to see whether the given user has permission to view the data in this model:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
```

Note

The preceding code (highlighted) won't match the code you see on GitHub as we'll be modifying it later in *step 9*.

5. Open the `bookr/reviews/urls.py` file and add the following path into the URL patterns:

```
path(
    'api/login', api_views.Login.as_view(),
    name='login'
)
```

6. Save the file and wait for the application to restart, or start the server manually with the `python manage.py runserver` command. Then, access the application using the `http://127.0.0.1:8000/api/login` URL. Your screen should appear as follows:

idx_0cb97183 인증. 여러 가지 수용 가능한 인증 방법 목록을 포함하는 것이 가능하다는 점에 유의하십시오. 하나만이 아닙니다. 또한 주어진 사용자가 이 모델의 데이터를 볼 수 있는 권한이 있는지 확인하는 DRF의 내장 클래스를 사용하는 `permissions_classes` 속성을 추가합니다:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
```

참고

앞서 언급한 코드(강조된 부분)는 나중에 9단계에서 수정할 것이므로 GitHub에서 볼 수 있는 코드와 일치하지 않습니다.

5. `bookr/reviews/urls.py` 파일을 열고 `URLpatterns`에 다음 경로를 추가하세요:

```
path(
    'api/login', api_views.Login.as_view(),
    name='login'
),
```

6. 파일을 저장하고 애플리케이션이 재시작될 때까지 기다리거나, `python manage.py runserver` 명령어로 서버를 수동으로 시작하세요. 그런 다음, `http://127.0.0.1:8000/api/login` URL을 사용하여 애플리케이션 `idx_0087f4cc`에 접근하세요. 화면은 다음과 같이 나타나야 합니다:

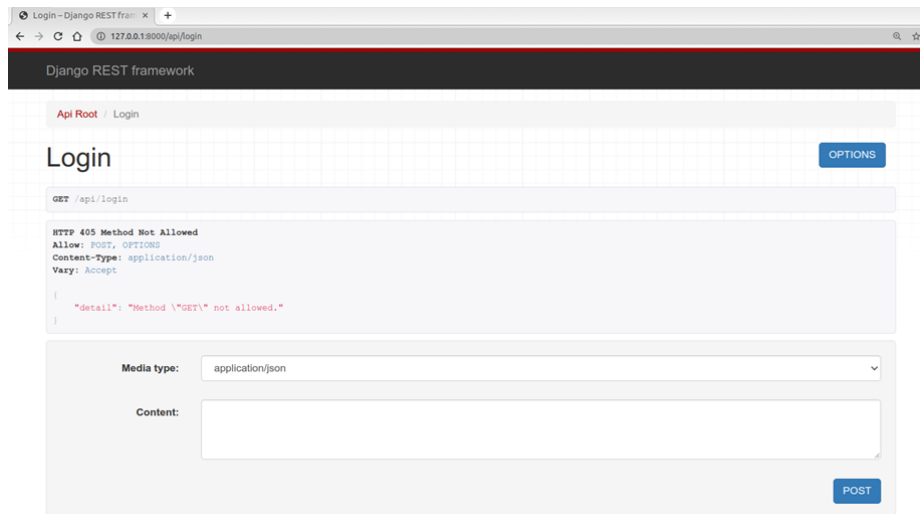


Figure 12.7: Login page

The API at `/api/login` is a POST-only message; hence, Method GET not allowed is displayed.

7. Next, enter the following snippet in the **Content** field and click on **POST**:

```
{
  "username": "Peter",
  "password": "testuserpassword"
}
```

You will need to replace this with an actual username and password for your account in the database. Now, you can see the token generated for the user. This is the token we need to use to access BookSerializer:

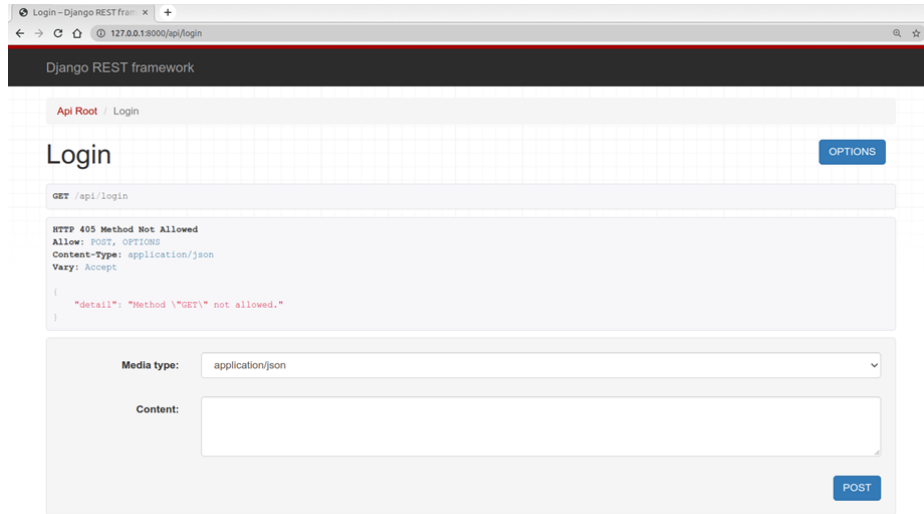


그림 12.7: 로그인 페이지

/api/login의 API는 POST 전용 메시지입니다. 따라서, Method GET not allowed가 표시됩니다.

7. 다음으로, Content 필드에 다음 스니펫을 입력하고 POST를 클릭하세요:

```
{
  "username": "Peter",
  "password": "testuserpassword"
}
```

계정의 실제 사용자 이름과 비밀번호로 이 부분을 교체해야 합니다. 이제 사용자를 위해 생성된 토큰을 볼 수 있습니다. 이것이 BookSerializer에 접근하는 데 필요한 토큰입니다:

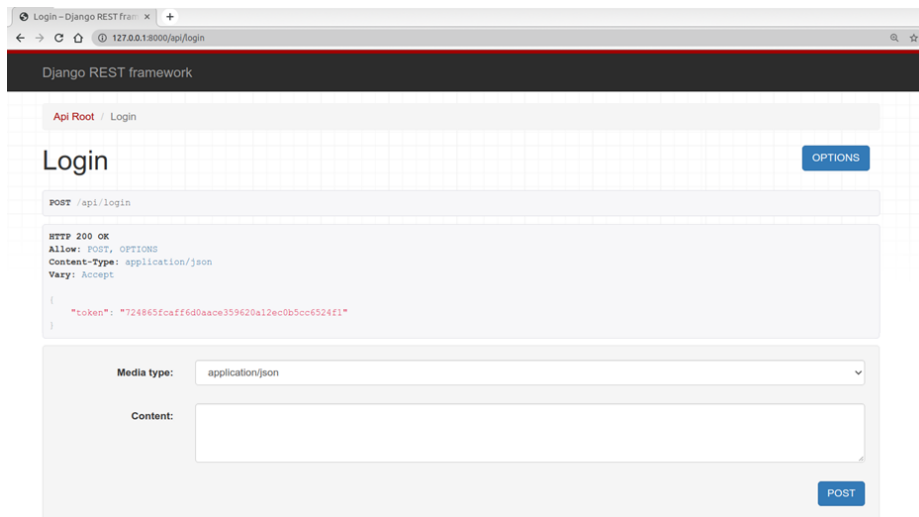


Figure 12.8: The token generated for the user

8. Try to access the list of books using the API that we previously created at `http://127.0.0.1:8000/api/books/`. Note that you are now not allowed to access it. This is because this `ViewSet` now requires you to use your token to authenticate. The same API can be accessed using `curl` on the command line:

```
curl -X GET http://127.0.0.1:8000/api/books/  
  
{"detail": "Authentication credentials were not provided."}
```

Since the token was not provided, an `Authentication credentials were not provided` message is displayed:

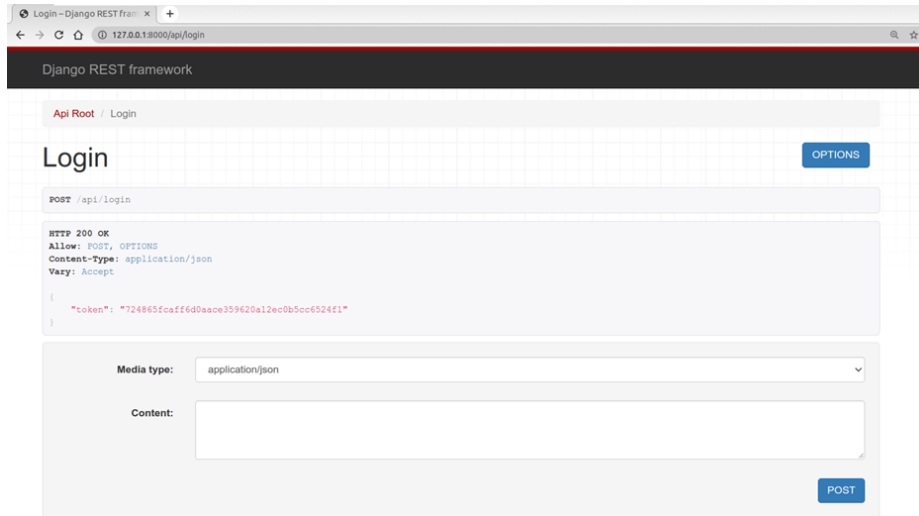


그림 12.8: 사용자에게 생성된 토큰

8. 이전에 생성한 API(<http://127.0.0.1:8000/api/books/>)를 사용하여 책 목록에 접근해 보세요. 이제 접근할 수 없다는 점에 유의하세요. 이는 이 ViewSet이 이제 토큰을 사용하여 인증할 것을 요구하기 때문입니다. 같은 API는 명령줄에서 curl을 사용하여 접근할 수 있습니다:

```
curl -X GET http://127.0.0.1:8000/api/books/
```

```
{"detail": "인증 자격 증명이 제공되지 않았습니다."}
```

idx_9b2c6c87 토큰 idx_67c213af가 제공되지 않았기 때문에 인증 자격 증명이 제공되지 않았다는 메시지가 표시됩니다:



Figure 12.9: Message saying that the authentication details weren't provided

Note that if you're using Windows 10, replace `curl` in the preceding command with `curl.exe` and execute it from Command Prompt.

To pass the Authorization token (obtained in *step 7*) as a header, you can use the following command (Windows users can replace `curl` with `curl.exe`):

```
curl -X GET http://127.0.0.1:8000/api/books/ -H "Authorization: Token  
cd5dafa1d4361fd1502652d266eed3dcdb55c64f1"
```

Note

Before pasting this command, make sure you've replaced the token (highlighted) with the one you got when you ran *step 7* of this exercise. It will be different from the one we have shown here.

The preceding command should now return a list of books:

```
[{"title": "Advanced Deep Learning with  
Keras", "publication_date": "2018-10-31", "isbn": "9781788629416", "publisher":  
{ "name": "Packt Publishing", "website": "https://  
www.packtpub.com/", "email": "info@packtpub.com" }, "rating": 4, "reviews":  
[{"content": "A must read for all", "date_created": "... (truncated)"]}]
```



그림 12.9: 인증 세부정보가 제공되지 않았다는 메시지

Windows 10을 사용하는 경우, 앞서 언급한 명령어에서 curl을 curl.exe로 바꾸고 명령 프롬프트에서 실행하세요.

Authorization 토큰(7단계에서 얻은)을 헤더로 전달하려면 다음 명령어를 사용할 수 있습니다(Windows 사용자는 curl을 curl.exe로 바꿀 수 있습니다):

```
curl -X GET http://127.0.0.1:8000/api/books/ -H "Authorization:
Tokencd5dafa1d4361fd1502652d266eed3dcdb55c64f1"
```

참고

이 명령어를 붙여넣기 전에, 이 연습의 7단계를 실행할 때 얻은 토큰(강조된 부분)을 여기에 표시된 것과 다르게 바꿨는지 확인하세요.

앞서 언급한 명령어는 이제 책 목록을 반환해야 합니다:

```
[{"title": "Advanced Deep Learning withKeras", "publication_date": "2018-10-31", "isbn": "9781788629416", "publisher": {"name": "Packt Publishing", "website": "https://www.packtpub.com/", "email": "info@packtpub.com"}, "rating": 4, "reviews": [{"content": "모두가 읽어야 할 필독서", "date_created": ...}
(생략)
```

This operation ensured that only an existing user of the application could access and fetch the collection of all books.

9. Before moving on, set the authentication and permission classes on `BookViewSet` to an empty string. Future chapters will not utilize these authentication methods, and we will assume for the sake of simplicity that an unauthenticated user can access our API:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = []
    permission_classes = []
```

In this exercise, we implemented token-based authentication in our bookr app. We created a login view that allows us to retrieve the token for a given authenticated user. This then enabled us to make API requests from the command line by passing through the token as a header in the request.

Overall, in this section, we learned about different types of authentication, and then we learned in detail about token authentication and authorization while working with REST APIs.

Summary

This chapter introduced REST APIs, a fundamental building block in most real-world web applications. These APIs facilitate communication between the backend server and the web browser, so they are central to your growth as a Django web developer. We learned how to serialize data in our database so that it can be transmitted via an HTTP request. Next, we learned about the various options DRF gives us to simplify the code we write, taking advantage of the existing definitions of the models themselves. We also covered ViewSets and routers and saw how they can be used to condense code even further by combining the functionality of multiple views. Finally, we learned about authentication and authorization and implemented token-based authentication for the book review app.

In the next chapter, we will extend bookr's functionality for its users by learning how to generate CSVs, PDFs, and other binary file types.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.

이 `idx_bd48a2c8` 작업은 `idx_c27a9f74` 애플리케이션의 기존 사용자만 모든 책의 컬렉션에 접근하고 가져올 수 있도록 보장했습니다.

9. 계속하기 전에 `BookViewSet`의 인증 및 권한 클래스를 빈 문자열로 설정하세요. 향후 장에서는 이러한 인증 방법을 사용하지 않으며, 단순화를 위해 인증되지 않은 사용자가 우리의 API에 접근할 수 있다고 가정하겠습니다:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = []
    permission_classes = []
```

이 연습에서는 `bookr` 앱에서 토큰 기반 인증을 구현했습니다. 인증된 사용자를 위해 토큰을 검색할 수 있는 로그인 뷰를 만들었습니다. 이를 통해 요청의 헤더에 토큰을 전달하여 명령줄에서 API 요청을 할 수 있게 되었습니다.

전반적으로 이 섹션에서는 다양한 유형의 인증에 대해 배웠고, REST API 작업 중에 토큰 인증 및 권한 부여에 대해 자세히 배웠습니다.

요약

이 장에서는 대부분의 실제 웹 애플리케이션에서 기본적인 구성 요소인 REST API를 소개했습니다. 이러한 API는 백엔드 서버와 웹 브라우저 간의 통신을 용이하게 하므로 Django 웹 개발자로서 성장하는 데 중심적인 역할을 합니다. 우리는 HTTP 요청을 통해 전송할 수 있도록 데이터베이스의 데이터를 직렬화하는 방법을 배웠습니다. 다음으로, 우리는 DRF가 제공하는 다양한 옵션을 통해 우리가 작성하는 코드를 단순화하는 방법을 배웠으며, 모델 자체의 기존 정의를 활용했습니다. 또한 ViewSets와 라우터에 대해 다루었고, 여러 뷰의 기능을 결합하여 코드를 더욱 간결하게 만드는 방법을 보았습니다. 마지막으로, 인증 및 권한 부여에 대해 배우고 책 리뷰 앱을 위한 토큰 기반 인증을 구현했습니다.

다음 장에서는 CSV, PDF 및 기타 이진 파일 유형을 생성하는 방법을 배워 `bookr`의 기능을 사용자에게 확장할 것입니다.

이 책의 PDF 사본, 코드 번들 및 기타 자료를 받으세요

QR 코드를 스캔하거나 packtpub.com/unlock으로 이동하세요. 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.



UNLOCK NOW

참고: 송장을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

13

Generating CSV, PDF, and Other Binary Files

So far, we have learned about the various aspects of the Django framework and explored how we can build web applications using Django with all the features and customizations we want.

Let's say that, while building a web application, we need to do some analysis and prepare some reports. We may need to analyze user demographics about how the platform is being used or generate data that can be fed into machine learning systems to find patterns. We want our website to display some of the results of our analysis in a tabular format and other results as detailed graphs and charts. Furthermore, we want to allow our users to export the reports and peruse them further in applications such as Jupyter Notebook and Excel.

As we work our way through this chapter, we will learn how to bring these ideas to fruition and implement functionality in our web application that allows us to export records into structured formats such as tables through the use of **Comma-Separated Values (CSV)** files or Excel files. We will also learn how to allow our users to generate visual representations of the data we have stored inside our web application and export it in a PDF format so that it can be distributed easily for quick reference.

Let's start our journey by learning how to work with CSV files in Python. Learning this skill will help us create functionality that allows our readers to export our data for further analysis.

In this chapter, we will be covering the following topics:

- Using the csv module to render a CSV file with a Django view
- Using the zipfile module to create a ZIP file archive with a Django view
- Generating PDF files with the WeasyPrint package
- Generating Excel files with the XlsxWriter package

13

CSV, PDF 및 기타 이진 파일 생성하기

지금까지 우리는 Django 프레임워크의 다양한 측면에 대해 배우고, 원하는 모든 기능과 맞춤 설정을 갖춘 웹 애플리케이션을 Django를 사용하여 어떻게 구축할 수 있는지 탐구했습니다.

웹 애플리케이션을 구축하는 동안 일부 분석을 수행하고 보고서를 준비해야 한다고 가정해 봅시다. 우리는 플랫폼이 어떻게 사용되고 있는지에 대한 사용자 인구 통계 분석을 하거나 패턴을 찾기 위해 기계 학습 시스템에 입력할 수 있는 데이터를 생성해야 할 수도 있습니다. 우리는 웹사이트가 분석 결과 중 일부를 표 형식으로, 다른 결과는 상세한 그래프와 차트로 표시하기를 원합니다. 또한, 사용자가 보고서를 내보내 Jupyter Notebook이나 Excel과 같은 애플리케이션에서 더 자세히 검토할 수 있도록 허용하고자 합니다.

이 장을 진행하면서 우리는 이러한 아이디어를 실현하는 방법과 웹 애플리케이션에서 기능을 구현하는 방법을 배우게 될 것입니다. 이를 통해 기록을 콤마로 구분된 값(CSV) 파일이나 엑셀 파일과 같은 구조화된 형식으로 내보낼 수 있습니다. 또한 사용자가 웹 애플리케이션에 저장된 데이터를 시각적으로 표현하고 이를 PDF 형식으로 내보내어 빠르게 참고할 수 있도록 쉽게 배포할 수 있는 방법도 배우게 될 것입니다.

파이썬에서 CSV 파일을 다루는 방법을 배우면서 우리의 여정을 시작해 봅시다. 이 기술을 배우면 독자들이 추가 분석을 위해 데이터를 내보낼 수 있는 기능을 만드는 데 도움을 주세

요. 이 장에서는 다음 주제를 다룰 것입니다:

- Django 뷰에서 csv 모듈을 사용하여 CSV 파일 렌더링하기
- zipfile 모듈을 사용하여 Django 뷰에서 ZIP 파일 아카이브 생성하기
- 기 WeasyPrint 패키지로 PDF 파일 생성하기
- XlsxWriter 패키지를 사용하여 Excel 파일 생성하기

- Generating graphics with the Python Image Library package
- Generating graphs with Plotly
- Generating graphics with Pillow

Working with CSV files in Python

There are several reasons we may need to export the data in our application. One of the reasons may involve analyzing that data – for example, we may need to understand the demographics of users registered on the application or extract patterns of application usage. We may also need to find out how our application is working for users to design future improvements. Such use cases require data to be in a format that can be easily consumed and analyzed. Here, the CSV file format comes to the rescue.

CSV is a handy file format that can be used to quickly export data from an application in a row-and-column format. CSV files usually have data separated by simple delimiters, which are used to differentiate one column from another, and newlines, which are used to indicate the start of a new record (or row) inside the table.

Python has great support for working with CSV files in its standard library, thanks to the `csv` module. This support enables the reading, parsing, and writing of CSV files. Let's take a look at how we can leverage the `csv` module provided by Python to work on CSV files and read and write data from them.

Working with Python's `csv` module

The `csv` module from Python allows us to interact with files that are in CSV format, which is nothing but a text file format – that is, the data stored inside the CSV files is human-readable.

The `csv` module requires that the file is opened before the methods supplied by the `csv` module can be applied.

First, let's look at how we can use the Python `csv` module to create new CSV files.

Writing to CSV files using Python

Now, let's learn how we can write CSV data to files. The following steps outline the process of writing data to CSV files:

1. Open the file in writing mode by running the following script:

```
csv_file = open('path to csv file', 'w')
```

- Python 이미지 라이브러리 패키지를 사용한 그래픽 생성
- 성 Plotly를 사용한 그래프 생성
- Pillow를 사용한 그래픽 생성

Python에서 CSV 파일 작업하기

우리 애플리케이션의 데이터를 내보내야 하는 여러 가지 이유가 있습니다. 그 이유 중 하나는 데이터를 분석하는 것과 관련이 있을 수 있습니다. 예를 들어, 우리는 애플리케이션에 등록된 사용자의 인구 통계를 이해하거나 애플리케이션 사용 패턴을 추출해야 할 수 있습니다. 또한, 우리는 사용자에게 애플리케이션이 어떻게 작동하는지 알아내어 향후 개선 사항을 설계해야 할 수도 있습니다. 이러한 사용 사례는 데이터가 쉽게 소비되고 분석될 수 있는 형식이어야 합니다. 여기서 CSV 파일 형식이 도움이 됩니다.

CSV는 애플리케이션에서 데이터를 행과 열 형식으로 빠르게 내보내는 데 사용할 수 있는 유용한 파일 형식입니다. CSV 파일은 일반적으로 간단한 구분 기호로 구분된 데이터를 가지고 있으며, 이는 한 열과 다른 열을 구분하는 데 사용되고, 새 레코드(또는 행)의 시작을 나타내는 데 사용되는 줄 바꿈이 있습니다.

Python은 csv 모듈 덕분에 표준 라이브러리에서 CSV 파일 작업을 위한 훌륭한 지원을 제공합니다. 이 지원은 CSV 파일의 읽기, 구문 분석 및 작성을 가능하게 합니다. 이제 Python이 제공하는 csv 모듈을 활용하여 CSV 파일에서 데이터를 읽고 쓰는 방법을 살펴보겠습니다.

Python의 csv 모듈로 작업하기

Python의 csv 모듈 `idx_4336983c`는 CSV 형식의 파일과 상호작용할 수 있게 해줍니다. CSV 형식은 텍스트 파일 형식에 불과하며, 즉 CSV 파일에 저장된 데이터는 사람이 읽을 수 있습니다.

csv 모듈은 csv 모듈에서 제공하는 메서드를 적용하기 전에 파일이 열려 있어야 합니다.

먼저, Python csv 모듈을 사용하여 새로운 CSV 파일을 만드는 방법을 살펴보겠습니다.

Python을 사용하여 CSV 파일에 쓰기

이제 CSV 데이터를 파일에 쓰는 방법 `idx_93764f22`를 배워봅시다. 다음 `idx_10594d88`단계는 CSV 파일에 데이터를 쓰는 과정을 설명합니다:

1. 다음 스크립트를 실행하여 파일을 쓰기 모드로 엽니다:

```
csv_file = open('path to csv file', 'w')
```

2. Obtain a CSV writer object, which can help us write data that is correctly formatted in CSV format. This can be done by calling the `writer()` method of the `csv` module, which returns a writer object, which can be used to write CSV format-compatible data to a CSV file:

```
csv_writer = csv.writer(csv_file)
```

3. Once the writer object is available, we can start writing the data. This is facilitated by the `write_row()` method of the writer object. The `write_row()` method takes in a list of values that it writes to the CSV file. The list itself indicates a single row and the values inside the list indicate the values of columns:

```
record = ['value1', 'value2', 'value3']  
csv_writer.writerow(record)
```

4. If you want to write multiple records in a single call, you can also use the `writerows()` method of the CSV writer. The `writerows()` method behaves similarly to the `writerow()` method but takes a list of lists and can write multiple rows in one go:

```
records = [['value11', 'value12', 'value13'],  
           ['value21', 'value22', 'value23']]  
csv_writer.writerows(records)
```

5. Once the records have been written, we can close the CSV file:

```
csv_file.close()
```

Fortunately, a Django `HttpResponse` object is a file-like object and can be used to create a CSV writer, so it is very easy to incorporate the `csv` module into a Django view in order to create dynamic spreadsheets from our models.

6. The `HttpResponse` constructor takes an optional argument, `content_type`, that can be used to specify a MIME type for the response. `DEFAULT_CHARSET` is `'UTF-8'`. If the `content_type` and `DEFAULT_CHARSET` arguments are unspecified, Django will render the `Content-Type` line of the HTML response header as follows:

```
Content-Type: text/html; charset=UTF-8
```

2. CSV 형식으로 올바르게 포맷된 데이터를 작성하는 데 도움이 되는 CSV 작성기 객체를 얻습니다. 이는 csv 모듈의 `writer()` 메서드를 호출하여 수행할 수 있으며, 이 메서드는 CSV 파일에 CSV 형식에 호환되는 데이터를 작성하는 데 사용할 수 있는 작성기 객체를 반환합니다.

```
csv_writer = csv.writer(csv_file)
```

3. ... 작가 객체가 준비되면 데이터를 쓰기 시작할 수 있습니다. 이것은 다음에 의해 용이해집니다. `writer` 객체의 `write_row()` 메서드. `write_row()` 메서드는 CSV 파일에 기록할 값들의 리스트를 입력으로 받습니다. 리스트 자체는 하나의 행을 나타내고, 리스트 안의 값들은 열의 값을 나타냅니다:

```
record = ['value1', 'value2', 'value3']
csv_writer.writerow(record)
```

4. ... 여러 레코드를 한 번의 호출로 작성하려면 `writerows()`를 사용할 수도 있습니다. CSV 작성기의 메서드입니다. `writerows()` 메서드는 `writerow()` 메서드와 유사하게 작동하지만 리스트의 리스트를 받아들이고 한 번에 여러 행을 쓸 수 있습니다:

```
records = [['value11', 'value12', 'value13'],
           ['value21', 'value22', 'value23']]
csv_writer.writerows(records)
```

5. 레코드가 작성되면 CSV 파일을 닫을 수 있습니다:

```
csv_file.close()
```

다행히도, Django `HttpResponse` 객체는 파일과 유사한 객체이며 CSV 작성기를 생성하는 데 사용할 수 있으므로, 모델에서 동적 스프레드시트를 생성하기 위해 csv 모듈을 Django 뷰에 통합하는 것이 매우 쉽습니다.

6. `HttpResponse` 생성자는 선택적 인수인 `content_type`을 사용하여 응답의 MIME 유형을 지정할 수 있습니다. `DEFAULT_CHARSET`은 'UTF-8'입니다. `content_type` 및 `DEFAULT_CHARSET` 인수가 지정되지 않으면 Django는 HTML 응답 헤더의 `Content-Type` 라인을 다음과 같이 렌더링합니다:

```
Content-Type: text/html; charset=UTF-8
```

7. The preferred MIME type for a CSV file is `text/csv`. We don't want to see the raw CSV data in the web browser, so we should set the content disposition to `attachment`, so that the CSV is downloaded with a specified filename.

```
response['Content-Disposition'] = \
    'attachment; filename="downloadfile.csv"'
```

8. Sometimes it suits our purposes to have a static filename, but in other circumstances, we might want to differentiate the filename by including a primary key or a date stamp.

```
date_stamp = datetime.datetime.now().strftime('%Y%m%d')
filename = f"download_file_{date_stamp}_{user_pk}"
response['Content-Disposition'] = f'attachment; filename="{filename}"'
```

Now let's utilize our knowledge of the `csv` module in order to create a summary report of the review data in CSV format.

Exercise 13.01 – Create a review summary CSV file

In this exercise, you will use the Python `csv` module in a Django view that returns a CSV file with a summary of book reviews.

The review summary needs to contain columns for **Book title**, **ISBN**, **Publisher name**, **Reviewer username**, and **Rating**.

We only want the book-specific information – **Book title**, **ISBN**, **Publisher name** – on the first review line of each book. Books without reviews will not be added to the report.

The column headers in the CSV file are to appear as `'title'`, `'isbn'`, `'publisher'`, `'reviewer'` and `'rating'`.

The Django view will be added to `reviews/views.py`.

Edit `reviews/views.py` and add an `import` statement for the `csv` module and make sure `HttpResponse` is being imported.

```
import csv

from django.http import HttpResponse
```

7. CSV 파일의 선호하는 MIME 유형은 text/csv입니다. 웹 브라우저에서 원시 CSV 데이터를 보고 싶지 않으므로, CSV가 지정된 파일 이름으로 다운로드 되도록 콘텐츠 배치를 attachment로 설정해야 합니다.

```
response['Content-Disposition'] = \
    'attachment; filename="downloadfile.csv"'
```

8. 때때로 정적 파일 이름이 우리의 목적에 맞지만, 다른 상황에서는 기본 키나 날짜 idx_68965774 스탬프를 포함하여 파일 이름을 구분하고 싶을 수 있습니다.

```
date_stamp = datetime.datetime.now().strftime('%Y%m%d')
filename = f"download_file_{date_stamp}_{user_pk}"
response['Content-Disposition'] = f'attachment; filename="{filename}"'
```

이제 csv 모듈에 대한 우리의 지식을 활용하여 리뷰 데이터의 요약 보고서를 CSV 형식으로 생성해 보겠습니다.

연습 13.01 – 리뷰 요약 CSV 파일 만들기

이 연습에서는 Django 뷰에서 Python csv 모듈을 사용하여 책 리뷰 요약이 포함된 CSV 파일 idx_a48918d9를 반환합니다.

리뷰 요약에는 책 제목, ISBN, 출판사 이름, 리뷰어 사용자 이름 및 평점에 대한 열이 포함되어야 합니다.

우리는 각 책의 첫 번째 리뷰 줄에 책 관련 정보 – 책 제목, ISBN, 출판사 이름 –만 원합니다. 리뷰가 없는 책은 보고서에 추가되지 않습니다.

CSV 파일의 열 헤더는 'title', 'isbn', 'publisher', 'reviewer' 및 'rating'으로 표시되어야 합니다.

Django 뷰는 reviews/views.py에 추가됩니다.

reviews/views.py를 편집하고 csv 모듈에 대한 import 문을 추가하고 HttpResponseRedirect가 가져오고 있는지 확인합니다.

```
import csv

from django.http import HttpResponseRedirect
```

The data retrieval step will involve querying the Book model and searching for books with at least one review. As this data retrieval step will be used in subsequent views in this chapter, we will create it as a private function, `_review_summary`, that can be called by various view functions.

1. The first step of the view is to retrieve all of the books from the Book model.

```
def _review_summary (request):  
    # Retrieve the data  
    books = Book.objects.all()
```

2. Define the header and include it as the first row in the data structure, rows.

```
header = ['title', 'isbn', 'publisher', 'reviewer', 'rating']  
rows = [header]
```

Iterate through the books and retrieve the current book's reviews.

```
for book in books:  
    reviews = book.review_set.all()
```

If they are present, append a row containing the book title, ISBN, publisher name, and, from the first review, the reviewer username and rating.

```
if reviews:  
    rows.append([book.title, book.isbn,  
                book.publisher.name,  
                reviews[0].creator.username,  
                reviews[0].rating])
```

On subsequent reviews, leave the book title, ISBN, and publisher name blank but include the reviewer username and rating.

```
for review in reviews[1:]:  
    rows.append(['', '', '',  
                review.creator.username,  
                review.rating])  
return rows
```

데이터 검색 단계는 Book 모델을 쿼리하고 최소한 하나의 리뷰가 있는 책을 검색하는 것을 포함합니다. 이 데이터 검색 단계는 이 장의 후속 뷰에서 사용될 것이므로, 다양한 뷰 함수에서 호출할 수 있는 비공식 함수 `_review_summary`로 생성할 것입니다.

1. 뷰의 첫 번째 단계는 Book 모델에서 모든 책을 검색하는 것입니다.

```
def _review_summary (request):
    # 데이터 검색
    books = Book.objects.all()
```

2. 헤더를 정의하고 데이터 구조인 `rows`의 첫 번째 행으로 포함시킵니다.

```
header = ['title', 'isbn', 'publisher', 'reviewer', 'rating']
rows = [header]
```

책을 반복하여 현재 책의 리뷰를 가져옵니다.

```
for book in books:
    reviews = book.review_set.all()
```

그들이 존재하면, 책 제목, ISBN, 출판사 이름, 그리고 첫 번째 리뷰에서 리뷰어 사용자 이름과 평점을 포함하는 행을 추가합니다.

```
if reviews:
    rows.append([book.title, book.isbn,
                book.publisher.name,
                reviews[0].creator.username,
                reviews[0].rating])
```

후속 리뷰에서는 책 제목, ISBN 및 출판사 이름을 비워두고 리뷰어 사용자 이름과 평점은 포함하세요.

```
for review in reviews[1:]:
    rows.append(['', '', '',
                review.creator.username,
                review.rating])

return rows
```

Now that we have created the private function, `_review_summary`, that is used to retrieve the book review data, we can code the function view, `review_summary_csv`.

After the view retrieves the data from `_review_summary`, a response object is created. It needs `content_type` of `text/csv` and an attachment disposition (since we want it to be saved as a download rather than appearing in the browser. We will give the download the filename `review_summary.csv`.

```
def review_summary_csv(request):
    rows = _review_summary()
    response = HttpResponse(content_type='text/csv')
    filename = "review_summary.csv"
    response['Content-Disposition'] = f'attachment; filename="{filename}"'
```

Create a `csv.writer` object, using the response object as a file-like object.

```
# Write the CSV response
writer = csv.writer(response)
```

Write each row in `rows` to the writer.

```
for row in rows:
    writer.writerow(row)
```

Return the response.

```
return response
```

The remaining step is to add the path of the summary report to `reviews/urls.py`.

```
urlpatterns = [
    ...
    path('reviews/csv/',
        views.review_summary_csv,
        name='review_summary_csv'),
]
```

The final set of changes should resemble the ones in the `Chapter13` folder in this book's GitHub repo.

이제 우리가 책 리뷰 데이터를 검색하는 데 사용되는 개인 함수 `_review_summary` 를 생성했으므로, 함수 `view_review_summary_csv` 를 코딩할 수 있습니다.

`view`가 `_review_summary`에서 데이터를 검색한 후, 응답 객체가 생성됩니다. 이는 `text/csv`의 `content_type`과 첨부 파일 처리가 필요합니다(브라우저에 나타나기보다는 다운로드로 저장되기를 원하기 때문입니다). 다운로드의 파일 이름은 `review_summary.csv`로 지정하겠습니다.

```
def review_summary_csv(request):
    rows = _review_summary()
    response = HttpResponse(content_type='text/csv')
    filename = "review_summary.csv"
    response['Content-Disposition'] = f'attachment; filename="{filename}"'
```

응답 객체를 파일과 유사한 객체로 사용하여 `csv.writer` 객체를 생성합니다.

```
# CSV 응답 작성하기
writer = csv.writer(response)
```

각 행을 `writer`에 작성합니다.

```
for row in rows:
    writer.writerow(row)
```

응답을 반환합니다.

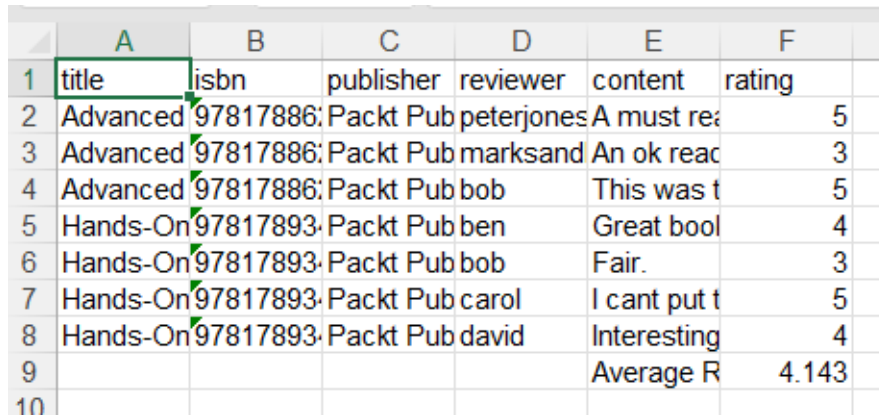
```
return response
```

`idx_57f392faremaining` 단계는 `reviews/urls.py`에 요약 보고서의 경로를 추가하는 것입니다.

```
urlpatterns = [
    ...
    path('reviews/csv/',
        views.review_summary_csv,
        name='review_summary_csv'),
    ...]
```

최종 변경 세트는 이 책의 GitHub 저장소에 있는 `Chapter13` 폴더의 변경 사항과 유사해야 합니다.

Now, when you visit the `http://localhost:8000/reviews/csv/` URL, you will be prompted to download a CSV file that looks like the following one.



	A	B	C	D	E	F
1	title	isbn	publisher	reviewer	content	rating
2	Advanced	97817886	Packt Pub	peterjones	A must read	5
3	Advanced	97817886	Packt Pub	marksand	An ok read	3
4	Advanced	97817886	Packt Pub	bob	This was t	5
5	Hands-On	97817893	Packt Pub	ben	Great book	4
6	Hands-On	97817893	Packt Pub	bob	Fair.	3
7	Hands-On	97817893	Packt Pub	carol	I cant put t	5
8	Hands-On	97817893	Packt Pub	david	Interesting	4
9					Average R	4.143
10						

Figure 13.01: The CSV file imported into Microsoft Excel

Soon, we will look at how we can make our website work with the XLSX format. But before that, let's quickly take a look at the usage of binary file formats.

Binary file formats for data exports

Until now, we have worked mainly with textual data and how we can read and write it from text files. But often, text-based formats are not enough. For example, imagine you want to export an image or a graph. How will you represent an image or a graph as text, and how will you read and write these images?

To help us in these situations, binary file formats come into the picture, which can help us read and write to and from a rich and diverse set of data. All commercial operating systems provide native support for working with both text and binary file formats, and it comes as no surprise that Python provides one of the most versatile implementations to work on binary data files. A simple example of this is the `open` command, which you can use to state the format you would like to open a file in:

```
file_handler = open('path to file', 'rb')
```

이제 `http://localhost:8000/reviews/csv/` URL을 방문하면 다음과 같은 CSV 파일을 다운로드하라는 메시지가 표시됩니다.

	A	B	C	D	E	F	
1	title	isbn	publisher	reviewer	content	rating	
2	Advanced	97817886	Packt Pub	peterjones	A must read	5	
3	Advanced	97817886	Packt Pub	marksand	An ok read	3	
4	Advanced	97817886	Packt Pub	bob	This was t	5	
5	Hands-On	97817893	Packt Pub	ben	Great book	4	
6	Hands-On	97817893	Packt Pub	bob	Fair.	3	
7	Hands-On	97817893	Packt Pub	carol	I cant put t	5	
8	Hands-On	97817893	Packt Pub	david	Interesting	4	
9					Average R	4.143	
10							

그림 13.01: Microsoft Excel로 가져온 CSV 파일

곧 XLSX 형식으로 웹사이트가 작동하도록 만드는 방법을 살펴보겠습니다. 하지만 그 전에 이진 파일 형식의 사용을 빠르게 살펴보겠습니다.

데이터 내보내기를 위한 이진 파일 형식

지금까지 우리는 주로 텍스트 데이터와 텍스트 파일에서 읽고 쓰는 방법에 대해 작업해왔습니다. 하지만 종종 텍스트 기반 형식만으로는 충분하지 않습니다. 예를 들어, 이미지를 내보내거나 그래프를 내보내고 싶다고 가정해 보십시오. 이미지를 텍스트로 어떻게 표현하고 이러한 이미지를 어떻게 읽고 쓸 것인가요?

이러한 상황에서 이진 파일 형식이 등장하여 다양한 데이터 세트에 대해 읽고 쓰는 데 도움을 줄 수 있습니다. 모든 상용 운영 체제는 텍스트 및 이진 파일 형식 작업을 위한 기본 지원을 제공하며, Python이 이진 데이터 파일 작업을 위한 가장 다재다능한 구현 중 하나를 제공하는 것은 놀라운 일이 아닙니다. 이의 간단한 예는 파일을 열고자 하는 형식을 지정하는 데 사용할 수 있는 `open` 명령입니다:

```
file_handler = open('path to file', 'rb')
```

Here, `b` indicates binary.

Starting from this section, we will now be dealing with how we can work on binary files and use them to represent and export data from our Django web application. The first of the formats we are going to look at is the XLSX file format made popular by Microsoft Excel.

So, let's dive into handling XLSX files with Python.

Working with XLSX files using the `XlsxWriter` package

In this section, we will learn more about the XLSX file format and understand how we can work with it using the **`XlsxWriter`** package.

While CSV has very wide usage, the degree of ambiguity in interpreting it between applications can make it a less desirable option for exchanging spreadsheet information – importantly, many of the features that we value in spreadsheets, such as worksheets, text formatting, color, and formulaic calculations.

Fortunately, it is possible to generate files in the native Excel format using Python. Microsoft Excel is the leading software application in the field of bookkeeping and tabular record management. Similarly, the XLSX file format that was adopted by Excel in 2007 has seen widespread support by many major product vendors.

XLSX files

XLSX files are binary files that are used to store tabular data. These files can be read by any software that implements support for this format. The XLSX format arranges data into two logical partitions:

- **Workbooks:** Each XLSX file is called a workbook and is supposed to contain datasets related to a particular domain. In *Figure 13.2*, `Examplefile.xlsx` is a workbook **(1)**:

여기서, b는 이진수를 나타냅니다.

이 섹션부터는 바이너리 파일을 다루는 방법과 이를 사용하여 Django 웹 애플리케이션에서 데이터를 표현하고 보내는 방법에 대해 다루겠습니다. 첫 번째는 우리가 살펴볼 형식은 마이크로소프트 엑셀로 유명한 XLSX 파일 형식입니다.

자, 이제 파이썬으로 XLSX 파일을 다루는 방법을 알아보시다.

XlsxWriter 패키지를 사용하여 XLSX 파일 작업하기

이 idx_c930da9b 섹션에서는 idx_eb039f99 XLSX 파일 형식에 대해 더 배우고 XlsxWriter 패키지를 사용하여 이를 다루는 방법을 이해할 것입니다.

CSV는 매우 널리 사용되지만, 애플리케이션 간에 이를 해석하는 데 있어 모호성이 크기 때문에 스프레드시트 정보를 교환하는 데 덜 바람직한 옵션이 될 수 있습니다. 특히, 워크시트, 텍스트 서식, 색상, 수식 계산과 같이 우리가 스프레드시트에서 중요하게 여기는 많은 기능들이 포함되지 않습니다.

다행히도, Python을 사용하여 네이티브 Excel 형식의 파일을 생성하는 것이 가능합니다. Microsoft Excel은 부기 및 표 형식 기록 관리 분야에서 선도적인 소프트웨어 응용 프로그램입니다. 마찬가지로, 2007년에 Excel에서 채택한 XLSX 파일 형식은 많은 주요 제품 공급업체들에 의해 널리 지원되고 있습니다.

XLSX 파일

XLSX 파일은 표 형식 데이터를 저장하는 데 사용되는 이진 idx_4052661a files입니다. 이러한 파일은 이 형식에 대한 지원을 구현하는 모든 소프트웨어에서 읽을 수 있습니다. XLSX 형식은 데이터를 두 개의 논리적 파티션으로 배열합니다:

- 워크북: 각 idx_61496be2 XLSX 파일은 워크북이라고 하며 특정 도메인과 관련된 데이터 세트를 포함해야 합니다. 그림 13.2에서 Examplefile.xlsx는 워크북(1)입니다:

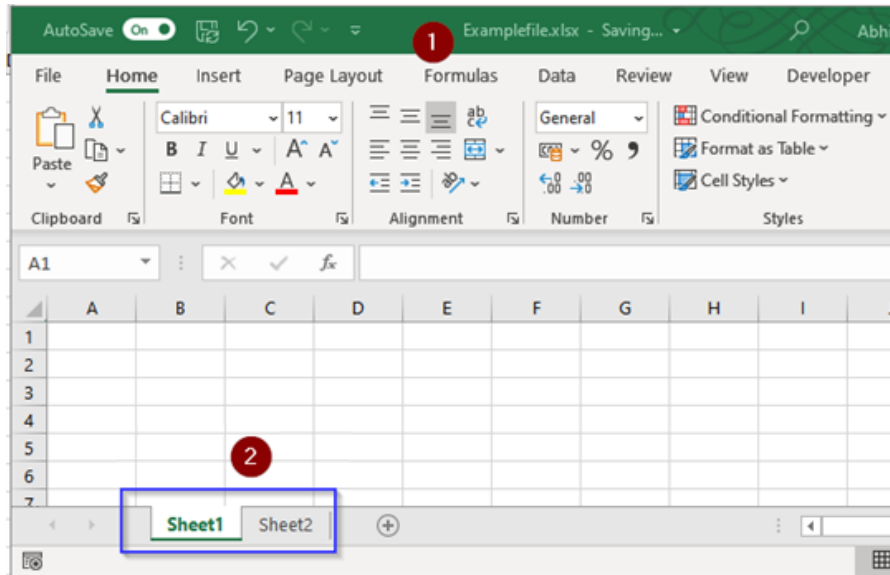


Figure 13.2: Workbooks and worksheets in Excel

- **Worksheets:** Inside each workbook, there can be one or more worksheets, which are used to store data about different but logically related datasets in a tabular format. In Figure 13.2, **Sheet1** and **Sheet2** are two worksheets (2).

When working with XLSX format, these are the two units that we generally work on. If you know about relational databases, you can think of workbooks as databases and worksheets as tables.

Now, let's try to understand how we can start working on XLSX files inside Python.

The XlsxWriter Python package

Python does not provide native support for working with XLSX files through its standard library. But thanks to the vast community of developers within the Python ecosystem, it is easy to find several packages that can help us manage our interaction with XLSX files. One popular package in this category is **XlsxWriter**.

XlsxWriter is an actively maintained package by the developer community that provides support for interacting with XLSX files. The package provides a lot of useful functionalities and supports the creation and management of workbooks, as well as worksheets in individual workbooks. You can install it by running the following command in Terminal or Command Prompt:

```
pip install XlsxWriter
```

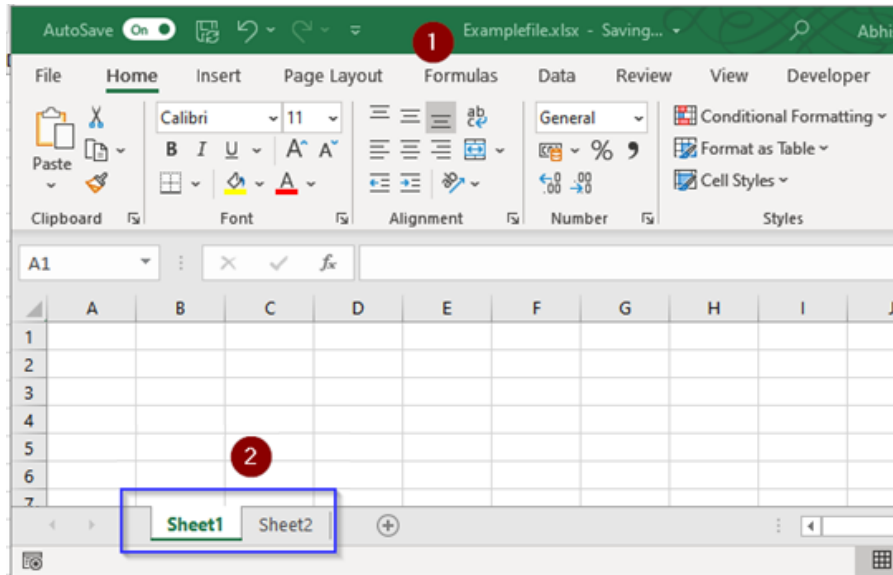


그림 13.2: Excel의 통합 문서 및 워크시트

- 워크시트: 각 통합 문서 내에는 하나 이상의 워크시트가 있을 수 있으며, 이는 서로 다르지만 논리적으로 관련된 데이터 세트에 대한 데이터를 표 형식으로 저장하는 데 사용됩니다. 그림 13.2에서 Sheet1과 Sheet2는 두 개의 워크시트(2)입니다.

XLSX 형식으로 작업할 때 일반적으로 작업하는 두 개의 단위입니다. 관계형 데이터베이스에 대해 알고 있다면, 통합 문서를 데이터베이스로, 워크시트를 테이블로 생각할 수 있습니다.

이제 Python 내에서 XLSX 파일 작업을 시작하는 방법을 이해해 봅시다.

XlsxWriter Python 패키지

Python은 표준 라이브러리를 통해 XLSX 파일 작업에 대한 기본 지원을 제공하지 않습니다. 그러나 Python 생태계 내의 방대한 개발자 커뮤니티 덕분에 XLSX 파일과의 상호 작용을 관리하는 데 도움이 되는 여러 패키지를 쉽게 찾을 수 있습니다. 이 범주에서 인기 있는 패키지는 XlsxWriter입니다.

XlsxWriter는 XLSX 파일과의 상호 작용을 지원하는 개발자 커뮤니티에 의해 적극적으로 유지 관리되는 패키지입니다. 이 패키지는 많은 유용한 기능을 제공하며, 개별 통합 문서 내에서 워크북 및 워크시트의 생성 및 관리를 지원합니다. 터미널 또는 명령 프롬프트에서 다음 명령을 실행하여 설치할 수 있습니다:

```
pip install xlsxwriter
```

Once installed, you can import the `xlsxwriter` module as follows:

```
import xlsxwriter
```

So, let's look at how we can start creating XLSX files with the support of the `XlsxWriter` package.

Creating a workbook

To start working on XLSX files, we first need to create them. An XLSX file is also known as a workbook and can be created by calling the `Workbook` class from the `xlsxwriter` package, as follows:

```
workbook = xlsxwriter.Workbook(filename)
```

The call to the `Workbook` class opens a binary file, specified with the `filename` argument, and returns an instance of `workbook` that can be used to further create worksheets and write data.

Creating a worksheet

Before we can start writing data to an XLSX file, we first need to create a worksheet. This can be done easily by calling the `add_worksheet()` method of the `workbook` object we obtained in the previous section:

```
worksheet = workbook.add_worksheet()
```

The `add_worksheet()` method creates a new worksheet, adds it to the `workbook`, and returns an object mapping the worksheet to a Python object, through which we can write data to the worksheet.

Writing data to the worksheet

Once a reference to the worksheet is available, we can start writing data to it by calling the `write` method of the `worksheet` object, as shown here:

```
worksheet.write(row_num, col_num, col_value)
```

As you can see, the `write()` method takes three parameters: a row number (`row_num`), a column number (`col_num`), and the data that belongs to the (`row_num`, `col_num`) pair, as represented by `col_value`. This call can be repeated to insert multiple data items into the worksheet.

설치가 완료되면, 다음과 같이 `xlsxwriter` 모듈을 가져올 수 있습니다:

```
import xlsxwriter
```

그럼, `XlsxWriter` 패키지의 지원을 받아 `XLSX` 파일을 생성하는 방법을 살펴보겠습니다.

워크북 생성하기

`XLSX` 파일 작업을 시작하려면, 먼저 파일을 생성해야 합니다. `XLSX` 파일은 워크북으로도 알려져 있으며, `xlsxwriter` 패키지의 `Workbook` 클래스를 호출하여 생성할 수 있습니다:

```
workbook = xlsxwriter.Workbook(filename)
```

`Workbook` 클래스 호출은 `filename` 인수로 지정된 이진 파일을 열고, 추가로 워크시트를 생성하고 데이터를 쓸 수 있는 워크북 인스턴스를 반환합니다.

워크시트 생성하기

`weidx_fdcae59e`가 `XLSX` 파일에 데이터를 쓰기 시작하기 전에, 먼저 워크시트를 생성해야 합니다. 이는 이전 섹션에서 얻은 워크북 객체의 `add_worksheet()` 메서드를 호출하여 쉽게 수행할 수 있습니다:

```
worksheet = workbook.add_worksheet()
```

`add_worksheet()` 메서드는 새 워크시트를 생성하고, 이를 워크북에 추가하며, 워크시트를 Python 객체에 매핑하는 객체를 반환합니다. 이를 통해 워크시트에 데이터를 쓸 수 있습니다.

워크시트에 데이터 쓰기

워크시트에 대한 참조 `idx_654d1cab`가 사용 가능해지면, 다음과 같이 워크시트 객체의 `write` 메서드를 호출하여 데이터를 쓰기 시작할 수 있습니다:

```
worksheet.write(row_num, col_num, col_value)
```

보시다시피, `write()` 메서드는 세 개의 매개변수를 사용합니다: 행 번호 (`row_num`), 열 번호 (`col_num`), 그리고 (`row_num`, `col_num`) 쌍에 해당하는 데이터인 `col_value`입니다. 이 호출은 워크시트에 여러 데이터 항목을 삽입하기 위해 반복될 수 있습니다.

Writing data to the workbook

Once all the data has been written, to finalize the written datasets and cleanly close the XLSX file, you must call the `close()` method on the workbook:

```
workbook.close()
```

This method writes any data that may be in the file buffer and finally closes the workbook. Now, let's use this knowledge to implement our own code, which will help us write data to an XLSX file.

Note

It's not possible to cover all the methods and features the `XlsxWriter` package provides in this chapter. For more information, you can read the official documentation: <https://xlsxwriter.readthedocs.io/contents.html>.

Formatting a spreadsheet

Let's revisit the spreadsheet from the CSV example and see how it could be implemented with additional XLSX formatting using `xlsxwriter`.

First, we need to install the `xlsxwriter` module. It is available in PyPi and can be installed quickly using `pip`, or it can be found on GitHub at <https://github.com/jmcnamara/XlsxWriter>.

```
pip install xlsxwriter
```

This section is not aimed at giving a comprehensive documentation of this module, but it gives a simple outline of the features that we are likely to encounter.

We begin by importing the `xlsxwriter` module, then creating a workbook by passing a filename or file-like object to the constructor.

```
import xlsxwriter
workbook = xlsxwriter.Workbook('output.xlsx')
```

We can then create specific worksheets in the workbook with the `add_worksheet` method.

```
worksheet = workbook.add_worksheet('Review Summary')
```

워크북에 데이터 쓰기

모든 theidx_48785feb 데이터가 작성되면, 작성된 데이터 세트를 마무리하고 XLSX 파일을 깔끔하게 닫으려면, 워크북에서 `close()` 메서드를 호출해야 합니다:

```
workbook.close()
```

이 메서드는 파일 버퍼에 있을 수 있는 모든 데이터를 작성하고 마지막으로 워크북을 닫습니다. 이제 이 지식을 사용하여 XLSX 파일에 데이터를 쓰는 데 도움이 되는 코드를 구현해 봅시다.

참고

이 장에서는 XlsxWriter 패키지가 제공하는 모든 메서드와 기능을 다룰 수 없습니다. 더 많은 정보는 공식 문서를 읽어보세요:
documentation: <https://xlsxwriter.readthedocs.io/contents.html>.

스프레드시트 서식 지정

CSV 예제에서 idx_019ff7bc 스프레드시트를 다시 살펴보고 xlsxwriter를 사용하여 추가 XLSX 형식으로 어떻게 구현할 수 있는지 살펴보겠습니다.

먼저, xlsxwriter 모듈을 설치해야 합니다. 이 모듈은 PyPi에서 사용할 수 있으며 pip를 사용하여 빠르게 설치할 수 있거나 GitHub에서 <https://github.com/jmcnamara/XlsxWriter>에서 찾을 수 있습니다.

```
pip install xlsxwriter
```

이 섹션은 이 모듈에 대한 포괄적인 문서를 제공하는 것을 목표로 하지 않지만, 우리가 마주칠 가능성이 있는 기능에 대한 간단한 개요를 제공합니다.

xlsxwriter 모듈을 가져오는 것으로 시작한 다음, 생성자에 파일 이름 또는 파일과 유사한 객체를 전달하여 워크북을 생성합니다.

```
import xlsxwriter
workbook = xlsxwriter.Workbook('output.xlsx')
```

그런 다음 `add_worksheet` 메서드를 사용하여 워크북에 특정 워크시트를 생성할 수 있습니다.

```
worksheet = workbook.add_worksheet('Review Summary')
```

The worksheet class contains a lot of methods for working with worksheets. If you are aiming to print your worksheet, or at least save it to PDF, the `set_landscape` and `set_portrait` methods come in handy.

We can also set specific column widths using the `set_column` method. The default width is approximately 8.43 characters in the default font of Calibri 11. We can specify a width based on characters for a range of columns in the spreadsheet.

So, for example, if we want the first column to have the default width, but the second to fourth columns to have a width of 20 characters, followed by a width of 50 characters for the eighth column, we can use the `worksheet.set_column` method to accomplish this.

```
worksheet.set_column(1, 3, 20)    # Columns B:D width 20
worksheet.set_column(7, 7, 50)    # Column H width 50
```

As well as numbering columns in the spreadsheet from zero and specifying two integer values for the column range, it is also possible to specify a column using Excel's A1 notation, so this could equivalently be written like this:

```
worksheet.set_column('B:D', 20)   # Columns B:D width 20
worksheet.set_column('H:H', 50)   # Column H width 50
```

As a rule, it is easier to use the A1 notation if we are hardcoding specific column details in our code, but if we are writing spreadsheets where we are dynamically formatting cells, the numeric co-ordinate approach is simpler.

Fortunately, `xlsxwriter` provides utilities functions that translate between the numeric and the A1 grid co-ordinate systems.

```
>>> xlsxwriter.utility.xl_col_to_name(4)
'E'
>>> xlsxwriter.utility.xl_rowcol_to_cell(4,6)
'G5'
>>> xlsxwriter.utility.xl_rowcol_to_cell(26,3)
'D27'
>>> xlsxwriter.utility.xl_rowcol_to_cell(3,26)
'AA4'
>>> xlsxwriter.utility.xl_cell_to_rowcol('A1')
(0, 0)
>>> xlsxwriter.utility.xl_cell_to_rowcol('AA10')
(9, 26)
```

워크시트 클래스는 워크시트 작업을 위한 많은 메서드를 포함하고 있습니다. 워크시트를 인쇄하거나 최소한 PDF로 저장하려는 경우, `set_landscape` 및 `set_portrait` 메서드가 유용합니다.

`set_column` 메서드를 사용하여 특정 열 너비를 설정할 수도 있습니다. 기본 너비는 Calibri 11의 기본 글꼴에서 약 8.43자입니다. 스프레드시트의 열 범위에 대해 문자 기반의 너비를 지정할 수 있습니다.

예를 들어, 첫 번째 열은 기본 너비를 가지도록 하고, 두 번째에서 네 번째 열은 20자 너비를 가지며, 여덟 번째 열은 50자 너비를 가지도록 하려면 `worksheet.set_column` 메서드를 사용하여 이를 수행할 수 있습니다.

```
worksheet.set_column(1, 3, 20) # 열 B:D 너비 20
worksheet.set_column(7, 7, 50) # 열 H 너비 50
```

스프레드시트에서 열 번호를 0부터 시작하고 열 범위에 대해 두 개의 정수 값을 지정하는 것 외에도, Excel의 A1 표기법을 사용하여 열을 지정하는 것도 가능하므로, 다음과 같이 쓸 수 있습니다:

```
worksheet.set_column('B:D', 20) # 열 B:D 너비 20
worksheet.set_column('H:H', 50) # 열 H 너비 50
```

일반적으로, 특정 열 세부정보를 하드코딩하는 경우 A1 표기법을 사용하는 것이 더 쉽지만, 셀을 동적으로 서식 지정하는 스프레드시트를 작성하는 경우 숫자 좌표 접근 방식이 더 간단합니다.

다행히도, `xlsxwriter`는 숫자와 A1 그리드 좌표 시스템 간의 변환을 제공하는 유틸리티 함수를 제공합니다.

```
>>> xlsxwriter.utility.xl_col_to_name(4)
'E'
>>> xlsxwriter.utility.xl_rowcol_to_cell(4,6)
'G5'
>>> xlsxwriter.utility.xl_rowcol_to_cell(26,3)
'D27'
>>> xlsxwriter.utility.xl_rowcol_to_cell(3,26)
'AA4'
>>> xlsxwriter.utility.xl_cell_to_rowcol('A1')
(0, 0)
>>> xlsxwriter.utility.xl_cell_to_rowcol('AA10')
(9, 26)
```

```
>>> xlswriter.utility.xl_range(0, 4, 5, 4)
'E1:E6'
```

These utilities come in very handy when we are trying to specify Excel formulae that require the A1 notation.

Note

Always use the `xlswriter.utility` functions for row col to A1 conversion. You may be tempted to write your own simple code to work out the cell name, but such code is error-prone, especially when the you utilize it for high-range values that you have not adequately tested.

Once we have a worksheet, we can assign specific values to cells, rows, and columns in the worksheet using the `worksheet.write`, `worksheet.write_row`, and `worksheet.write_column` methods. All of these methods use either the A1 notation or the coordinate notation for writing cells, as we can see in the following example, where it is easier to use the A1 notation to explicitly specify rows or columns, but if we need to dynamically determine the cell location, then it is easier to specify by (row, column) co-ordinates.

The Python code uses these write methods to produce the Excel spreadsheet.

```
import xlswriter

workbook = xlswriter.Workbook('WriteRowsAndColumns.xlsx')
worksheet = workbook.add_worksheet('Title')

worksheet.write_column('A1', list('WEB'))
worksheet.write_row('B4', list('DEVELOPMENT'))
worksheet.write_column('A5', list('WITH'))
worksheet.write_row('B9', list('DJANGO'))

for i, letter in enumerate('PACKT'):
    worksheet.write(12, i*2, letter)

workbook.close()
```

This brings us to cell formatting. `xlswriter` includes a `format` class that implements all of the formatting that we encounter in Excel spreadsheets – fonts, color, font size, bold, italic, struck-out text, date representation, numerical formatting, and so on. We can create a `Format` object by using the `add_format` method on the `worksheet` class. We can either pre-specify the format

```
>>> xlswriter.utility.xl_range(0, 4, 5, 4)
'E1:E6'
```

이 유틸리티는 A1 표기법이 필요한 Excel 수식을 지정하려고 할 때 매우 유용합니다.

참고

항상 `xlswriter.utility` 함수를 사용하여 행 열을 A1로 변환하십시오. 셀 이름을 계산하기 위해 간단한 코드를 작성하고 싶을 수 있지만, 그러한 코드는 오류가 발생하기 쉬우며, 특히 충분히 테스트하지 않은 고급 값에 대해 사용할 때 더욱 그렇습니다.

워크시트를 갖게 되면, `worksheet.write`, `worksheet.write_row` 및 `worksheet.write_column` 메서드를 사용하여 워크시트의 셀, 행 및 열에 특정 값을 할당할 수 있습니다. 이러한 모든 메서드는 셀을 작성하기 위해 A1 표기법 또는 좌표 표기법을 사용합니다. 다음 예제에서 볼 수 있듯이, 행이나 열을 명시적으로 지정하기 위해 A1 표기법을 사용하는 것이 더 쉽지만, 셀 위치를 동적으로 결정해야 하는 경우 (행, 열) 좌표로 지정하는 것이 더 쉽습니다.

Python 코드는 `idx_5acafbba` 이러한 쓰기 메서드를 사용하여 Excel 스프레드시트를 생성합니다.

```
import xlswriter

workbook = xlswriter.Workbook('WriteRowsAndColumns.xlsx')
worksheet = workbook.add_worksheet('Title')

worksheet.write_column('A1', list('WEB'))
worksheet.write_row('B4', list('DEVELOPMENT'))
worksheet.write_column('A5', list('WITH'))
worksheet.write_row('B9', list('DJANGO'))

for i, letter in enumerate('PACKT'):
    worksheet.write(12, i*2, letter)
workbook.close()
```

이제 셀 서식으로 넘어갑니다. `xlswriter`에는 Excel 스프레드시트에서 접하는 모든 서식을 구현하는 `format` 클래스가 포함되어 있습니다 - 글꼴, 색상, 글꼴 크기, 굵게, 기울임, 취소선 텍스트, 날짜 표현, 숫자 서식 등. 우리는 `worksheet` 클래스의 `add_format` 메서드를 사용하여 `Format` 객체를 생성할 수 있습니다. 우리는 서식을 미리 지정할 수 있습니다.

by calling `add_format` with a dictionary argument that contains the formatting parameters, or we can create an empty format and then override the defaults with specific methods on the format object. The following two code snippets show equivalent ways of creating the same format.

```
format_dict = {'bold': True,          # set bold
               'font_size': 12,      # specify font size
               'bg_color': 'gray'}) # set background color
header_format = workbook.add_format(format_dict)

header_format = workbook.add_format()
header_format.set_bold()
header_format.set_font_size(12)
header_format.set_bg_color('gray')
```

Once we have a format object, we can specify it as the final argument of one of the write methods.

```
worksheet.writerow('A1', header, header_format)
```

For a better understanding of the `Format` class, you might read the comprehensive documentation at <https://xlsxwriter.readthedocs.io/format.html>. The remaining examples and exercises in this section will only use the properties dictionary to specify formatting.

We are not limited to populating spreadsheets with static content. `xlsxwriter` includes the ability to specify Excel formulae in specific worksheet cells.

```
worksheet.write_formula('F9', '=RAND()')
```

The `XlsxWriter` reference material at <https://xlsxwriter.readthedocs.io/> can help you unlock the power of the XSLX file format and make amazing spreadsheets. We have barely scratched the surface of `xlsxwriter`, but have learned enough to recreate the review CSV file as an Excel spreadsheet, augmenting it with formatting and dynamic calculations.

Now that we have surveyed the basic functionality of the **XlsxWriter** package, we can revisit the review summary report from *Exercise 13.01* and implement it as a formatted spreadsheet.

형식 매개변수를 포함하는 사전 인수로 `add_format`을 호출하거나, 빈 형식을 생성한 다음 형식 객체의 특정 메서드로 기본값을 재정의할 수 있습니다. 다음 두 코드 조각은 동일한 형식을 생성하는 동등한 방법을 보여줍니다.

```
format_dict = {'bold': True,          # 굵게 설정# 글꼴 크기 지정
               'font_size': 12,
               'bg_color': 'gray'}) # 배경색 설정
header_format = workbook.add_format(format_dict)

header_format = workbook.add_format()
header_format.set_bold()
header_format.set_font_size(12)
header_format.set_bg_color('gray')
```

형식 객체가 있으면, 이를 `writermethods` 중 하나의 마지막 인수로 지정할 수 있습니다.

```
worksheet.writerow('A1', header, header_format)
```

Format 클래스에 대한 더 나은 이해를 위해

<https://xlsxwriter.readthedocs.io/format.html>에서 포괄적인 문서를 읽어볼 수 있습니다. 이 섹션의 나머지 예제와 연습은 형식을 지정하기 위해 속성 사전만 사용할 것입니다.

우리는 정적 콘텐츠로 스프레드시트를 채우는 것에 국한되지 않습니다. `xlsxwriter`는 특정 워크시트 셀에 Excel 수식을 지정할 수 있는 기능을 포함합니다.

```
worksheet.write_formula('F9', '=RAND()')
```

`XlsxWriter` 참조 `idx_be58a600material`은 <https://xlsxwriter.readthedocs.io/>에서 XSLX 파일 형식의 힘을 활용하고 놀라운 스프레드시트를 만드는 데 도움을 줄 수 있습니다. 우리는 `xlsxwriter`의 표면을 간신히 굽어보았지만, 형식과 동적 계산으로 보강된 Excel 스프레드시트로 리뷰 CSV 파일을 재현할 만큼 충분히 배웠습니다.

이제 `XlsxWriter` 패키지의 기본 기능을 조사했으므로, 연습 13.01의 리뷰 요약 보고서를 다시 방문하고 이를 형식이 지정된 스프레드시트로 구현할 수 있습니다.

연습 13.02 – 서식이 지정된 스프레드시트 개발

연습 13.01에서 우리는 Django 모델의 쿼리에서 CSV 스프레드시트를 생성하는 Django 뷰의 메커니즘을 구축했습니다. 이번 연습에서는 XlsxWriter 패키지를 사용하여 서식이 지정된 스프레드시트를 개발할 것입니다.

우리의 접근 방식은 데이터 검색 요청과 서식 매개변수를 받아 특정 요청을 작성하는 일반 XLSX 보고서 작성기를 개발하는 것입니다.

우리는 연습 13.01에서 개발한 비공식 함수 `_review_summary`를 재사용하고 출력에 서식을 추가하여 다음 XLSX 스프레드시트가 생성되도록 할 것입니다.

Django 뷰 `review_summary_xlsx`는 `reviews/views.py`에 추가됩니다. 이 함수는 요청 객체를 인수로 받습니다. 처음에 `_review_summary` 비공식 함수를 사용하여 데이터를 검색합니다. 보고서에는 다양한 유형의 셀 서식이 포함되어 있으므로 워크시트의 `add_format` 메서드에 전달할 매개변수를 개발할 것입니다.

```
def review_summary_xlsx(request):
    rows = _review_summary()

    format_settings = dict(
        header = dict(bold=True, font_size=12,
                      bg_color= 'gray'),
        short_text= dict(align='left'),
        isbn = dict(num_format= '0', align= 'left'),
        content = dict(align= 'left', valign='top',
                      text_wrap= True),
        rating = dict(num_format= '#', align= 'right'),
        footer = dict(bold= True, font_size=12),
        average = dict(num_format= '#.###', align='right',
                      bold= True, font_size=12),
    )
```

Excel 스프레드시트에서 열 너비를 정의할 수 있으므로, 일련의 열 너비를 정의하겠습니다:

```
# 열 A 40, B 15, C-D 30, E 50, F 5
column_widths = [('A:A', 40), ('B:B', 14),
                  ('C:D', 30), ('E:E', 50), ('F:F', 5),]
```

The format settings need to be mapped to individual columns in the report. So, title, reviewer, and content will all use the short_text format, while isbn and rating will have their own specific numeric formats.

```
# title, isbn, publisher, reviewer, content, rating
col_formats = ('short_text', 'isbn', 'short_text',
               'short_text', 'content', 'rating')
```

We create a BytesIO in-memory buffer to store the spreadsheet. The buffer will be written to by `xlsxwriter.Workbook`.

```
output = BytesIO()
workbook = xlsxwriter.Workbook(output)
```

We will call a `format_workbook` utility function that takes the workbook objects, the data rows, and our formatting information as parameters.

```
workbook = format_workbook(
    workbook, rows, format_settings,
    reader.fieldnames, col_formats, column_widths,
    footer_avg=dict(label='Average Rating',
                    id='rating'))
```

At the end of the process, we close the workbook, rewind the buffer, and attach the buffer with the appropriate disposition and content type to the HTTP response.

```
workbook.close()
# Rewind the buffer.
output.seek(0)
# Create the HTTP response.
filename = 'review_details.xlsx'
ctype = ('application/vnd.openxmlformats-
         'officedocument.spreadsheetml.sheet')
response = HttpResponse(output, content_type=ctype)
response['Content-Disposition'] = \
    f'attachment; filename={filename}'
return response
```

formatidx_73423fd9 설정은 보고서의 개별 열에 매핑되어야 합니다. 따라서 제목, 검토자 및 내용은 모두 short_text 형식을 사용하고, isbn 및 평점은 각각의 특정 숫자 형식을 가집니다.

```
# 제목, isbn, 출판사, 검토자, 내용, 평점
col_formats = ('short_text', 'isbn', 'short_text',
               'short_text', 'content', 'rating')
```

스프레드시트를 저장하기 위해 메모리 내에 BytesIO 버퍼를 생성합니다. 이 버퍼는 `xlsxwriter.Workbook`에 의해 작성됩니다.

```
output = BytesIO()
workbook = xlsxwriter.Workbook(output)
```

워크북 객체, 데이터 행 및 형식 정보를 매개변수로 사용하는 `format_workbook` 유틸리티 함수를 호출할 것입니다.

```
workbook = format_workbook(
    workbook, rows, format_settings,
    reader.fieldnames, col_formats, column_widths,
    footer_avg=dict(label='평균 평점',
                    id='rating'))
```

프로세스가 끝나면, 우리는 워크북을 닫고, 버퍼를 되감고, 적절한 처분 및 콘텐츠 유형과 함께 HTTP 응답에 버퍼를 첨부합니다.

```
workbook.close()
# 버퍼를 되감습니다.
output.seek(0)
# HTTP 응답을 생성합니다.
filename = 'review_details.xlsx'
ctype = ('application/vnd.openxmlformats-
         'officedocument.spreadsheetml.sheet')
response = HttpResponse(output, content_type=ctype)
response['Content-Disposition'] = \
    f'attachment; filename={filename}'
return response
```

Now we can define the `format_workbook` method in the `reviews/utils.py` file. We begin by importing the necessary helper functions from `xlswriter.utility`, which will be used for converting from numeric co-ordinates to the A1 cell notation.

```
from xlswriter.utility import xl_rowcol_to_cell, xl_range
```

The `format_workbook` method takes arguments for workbook objects, the rows data, `format_settings`, `col_formats`, and `column_heights` that we created in the `review_summary_xlsx` function view. The optional `footer_avg` argument is used to specify whether there will be an average calculation in the footer.

```
def format_workbook(workbook, rows, format_settings,
                    header, col_formats, column_widths, footer_avg=None):
```

A worksheet called 'Review Summary' is added to the workbook.

```
worksheet = workbook.add_worksheet('Review Summary')
```

The `formats` parameters supplied by `format_settings` are used to define a dictionary of formats.

```
formats = {format_id: workbook.add_format(format_set)
           for format_id, format_set in
           format_settings.items()}
```

The column widths specified in `column_widths` are applied to the worksheet.

```
for col_range, width in column_widths:
    worksheet.set_column(col_range, width)
```

The header is written to the file and formatted.

```
worksheet.write_row('A1', header, formats['header'])
for col_num, cell_data in enumerate(header):
    worksheet.write(0, col_num, cell_data,
                   formats['header'])
```

이제 reviews/utils.py 파일에서 format_workbook 메서드를 정의할 수 있습니다. 우리는 `xlsxwriter.utility`에서 필요한 도우미 함수를 가져오는 것으로 시작합니다. 이 함수는 `idx_3bc80d2e`를 숫자 좌표에서 A1 셀 표기법으로 변환하는 데 사용됩니다.

```
from xlsxwriter.utility import xl_rowcol_to_cell, xl_range
```

`format_workbook` 메서드는 `workbook` 객체, 행 데이터, `format_settings`, `col_formats` 및 `review_summary_xlsx` 함수 보기에서 생성한 `column_heights`에 대한 인수를 받습니다. 선택적 `footer_avg` 인수는 바닥글에 평균 계산이 있을지 여부를 지정하는 데 사용됩니다.

```
def format_workbook(workbook, rows, format_settings,
                    header, col_formats, column_widths, footer_avg=None):
```

'Review Summary'라는 워크시트가 통합 문서에 추가됩니다.

```
worksheet = workbook.add_worksheet('Review Summary')
```

`format_settings`에서 제공된 `formats` 매개변수는 형식의 사전을 정의하는 데 사용됩니다.

```
formats = {format_id: workbook.add_format(format_set)
            for format_id, format_set in
            format_settings.items()}
```

`column_widths`에 지정된 열 너비가 워크시트에 적용됩니다.

```
for col_range, width in column_widths:
    worksheet.set_column(col_range, width)
```

헤더가 파일에 작성되고 형식이 지정됩니다.

```
worksheet.write_row('A1', header, formats['header'])
for col_num, cell_data in enumerate(header):
    worksheet.write(0, col_num, cell_data,
                   formats['header'])
```

Each row retrieved from `_review_summary` is added to the worksheet, with formats being applied based on their specification in `col_formats`.

```
for row_num, row in enumerate(rows):
    if not row:
        continue
    for col_num, col in enumerate(
        list(row.values())):
        worksheet.write(row_num+1, col_num, col,
                        formats[col_formats[col_num]])
```

We can include the average calculation in the worksheet before returning the formatted workbook.

```
# Add footer with average rating
if footer_avg:
    row_id = len(rows) + 1
    col_id = header.index(footer_avg['id'])
    col_label_id = (col_id - 1 if col_id > 0
                    else len(col_formats))
    worksheet.write(
        row_id,
        col_label_id,
        f"{footer_avg['label']}: ",
        formats['footer'])
    avg_cell = xl_rowcol_to_cell(len(rows)+1, col_id)
    avg_range = xl_range(1, col_id, row_id-1, col_id)
    worksheet.write_formula(avg_cell,
                            f"AVERAGE({avg_range})")

return workbook
```

Now the view has been created, we need to include the appropriate URL path in `reviews/urls.py`.

```
path('reviews/xlsx/', views.review_summary_xlsx,
     name='review_summary_xlsx'),
```

The final set of changes should resemble the ones in the `Chapter13` folder in this book's GitHub repo.

_review_summary에서 검색된 각 행은 col_formats에 지정된 형식에 따라 워크시트에 추가됩니다.

```
for row_num, row in enumerate(rows):
    if not row:
        continue
    for col_num, col in enumerate(
        list(row.values())):
        worksheet.write(row_num+1, col_num, col,
                        formats[col_formats[col_num]])
```

우리는 서식이 지정된 워크북을 반환하기 전에 워크시트에 평균 계산을 포함할 수 있습니다.

```
# 평균 등급으로 바닥글 추가
if footer_avg:
    row_id = len(rows) + 1
    col_id = header.index(footer_avg['id'])
    col_label_id = (col_id - 1 if col_id > 0
                    else len(col_formats))
    worksheet.write(
        row_id,
        col_label_id,
        f"{footer_avg['label']}: ",
        formats['footer'])
    avg_cell = xl_rowcol_to_cell(len(rows)+1, col_id)
    avg_range = xl_range(1, col_id, row_id-1, col_id)
    worksheet.write_formula(avg_cell,
                            f"AVERAGE({avg_range})")

return workbook
```

이제 view idx_a1f13c82가 생성되었습니다. reviews/urls.py에 적절한 URL 경로를 포함해야 합니다.

```
path('reviews/xlsx/', views.review_summary_xlsx,
     name='review_summary_xlsx'),
```

최종 변경 세트는 이 책의 GitHub 저장소에 있는 Chapter13 폴더의 변경 사항과 유사해야 합니다.

Now when you visit the `http://localhost:8000/reviews/xlsx/` URL, you will be prompted to download a CSV file that looks like the following one.

	A	B	C	D	E	F
1	title	isbn	publisher	reviewer	content	rating
2	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	peterjones	A must read for all	5
3	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	marksandler	An ok read	3
4	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	bob	This was the best book on Deep Learning that I have read.	
5	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	ben	Now I am familiar with Keras.	5
6	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	bob	Great book!	4
7	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	carol	Fair.	3
8	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	david	I cant put this one down..	5
9					Interesting book.	4
10					Average Rating:	4.143

Figure 13.3: The XLSX spreadsheet

We have seen in this example that Django's MVT architecture is flexible enough to allow the creation of a binary response. We will next apply this approach to create ZIP file archives from within Django.

ZIP files

The ZIP archive format has been in popular use since the release of PKZIP in 1989. ZIP files contain either single compressed files or collections and preserve directory structure.

The `zipfile` module is part of the Python standard library and it contains the resources to generate ZIP files programmatically. The module contains a `ZipFile` class that can be used to read, write, and append to ZIP files.

The `ZipFile` constructor takes a mandatory file argument and three optional arguments:

- **file:** This can either be the path of the ZIP file or a file-like object
- **mode:** Either `'r'`, `'w'`, `'a'`, or `'x'`, for read, write, append, or exclusive (the default is `'r'`)
- **compression:** The `zipfile` module defines five constants representing the type of compression that is applied to the ZIP archive. These are as follows:
 - `zipfile.ZIP_STORED` (no compression)
 - `zipfile.ZIP_DEFLATED` (zlib)
 - `zipfile.ZIP_BZIP2` (bzip2)
 - `zipfile.ZIP_LZMA` (LZMA)
 - `zipfile.ZIP_ZSTD` (compression.zstd)

이제 <http://localhost:8000/reviews/xlsx/> URL을 방문하면 다음과 같은 CSV 파일을 다운로드하라는 메시지가 표시됩니다.

	A	B	C	D	E	F
1	title	isbn	publisher	reviewer	content	rating
2	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	peterjones	A must read for all	5
3	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	marksandler	An ok read	3
4	Advanced Deep Learning with Keras	9781788629416	Packt Publishing	bob	This was the best book on Deep Learning that I have read.	5
5	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	ben	Now I am familiar with Keras.	4
6	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	bob	Great book!	3
7	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	carol	Fair.	5
8	Hands-On Machine Learning for Algorithmic Trac	9781789346411	Packt Publishing	david	I cant put this one down..	4
9					Interesting book.	4.143
10					Average Rating:	

그림 13.3: XLSX 스프레드시트

이 예제에서 우리는 Django의 MVT 아키텍처가 이진 응답 생성을 허용할 만큼 유연하다는 것을 보았습니다. 다음으로 이 접근 방식을 적용하여 Django 내에서 ZIP 파일 아카이브를 생성할 것입니다.

ZIP 파일

ZIPidX_4735bfce 아카이브 형식은 1989년 PKZIP 출시 이후 널리 사용되고 있습니다. ZIP 파일은 단일 압축 파일 또는 컬렉션을 포함하며 디렉토리 구조를 유지합니다.

zipfile 모듈은 `idx_d06d3a5d` 파이썬 표준 라이브러리의 일부이며 ZIP 파일을 프로그래밍 방식으로 생성하는 데 필요한 리소스를 포함합니다. 이 모듈에는 ZIP 파일을 읽고, 쓰고, 추가하는 데 사용할 수 있는 `ZipFile` 클래스가 포함되어 있습니다.

`ZipFile` 생성자는 필수 파일 인수와 세 개의 선택적 인수를 받습니다: `file`: 이것은 ZIP

- 파일의 경로이거나 파일과 유사한 객체일 수 있습니다.
- 모드: 읽기, 쓰기, 추가 또는 독점('r'이 기본값) 중 하나인 'r', 'w', 'a' 또는 'x'
- 압축: zipfile 모듈은 ZIP 아카이브에 적용되는 압축 유형을 나타내는 다섯 개의 상수를 정의합니다. 이들은 다음과 같습니다:
 - `zipfile.ZIP_STORED` (압축 없음)
 - `zipfile.ZIP_DEFLATED` (zlib)
 - `zipfile.ZIP_BZIP2` (bzip2)
 - `zipfile.ZIP_LZMA` (LZMA)
 - `zipfile.ZIP_ZSTD` (ZSTD)
 - `(compression.zstd)`

The default is no compression. As not all ZIP software clients use the bzip2, LZMA, and compression.zstd libraries, `zipfile.ZIP_DEFLATED` is the safest option if you intend to compress your ZIP file.

- `allowZipFile64`: This is a boolean to allow files with ZIP64 extensions. By default, this is `True`.

Consider a situation where a directory contains the following files.

```
csv/book_list.csv
csv/reviews_2010.csv
csv/reviews_2011.csv
csv/reviews_2012.csv
csv/reviews_2013.csv
...
csv/reviews_2025.csv
```

We want to programmatically create a ZIP archive that only stores the files that match the wildcard `csv/reviews_20???.csv`. Fortunately, Python has a module for this kind of wildcard matching called `glob`.

This simple program solves our problem.

```
from zipfile import ZipFile, ZIP_DEFLATED
import glob

file_paths = glob.glob('csv/reviews_20???.csv')
with ZipFile('all_reviews.zip', 'w', ZIP_DEFLATED) as z:
    for file_path in file_paths:
        z.write(file_path)
```

As in the case of the CSV data, we can easily create a Django view that will take a request and write a ZIP file to a response.

```
import datetime
import glob
from io import BytesIO
from zipfile import ZipFile

from django.http import HttpResponse
```

기본값은 압축이 없습니다. 모든 ZIP 소프트웨어 클라이언트가 bzip2, LZMA 및 compression.zstd 라이브러리를 사용하지 않기 때문에, ZIP 파일을 압축할 의도가 있다면 zipfile.ZIP_DEFLATED가 가장 안전한 옵션입니다.

- allowZipFile64: 이는 ZIP64 확장자를 가진 파일을 허용하는 불리언입니다. 기본적으로 이는 True입니다.

디렉토리에 다음 파일이 포함된 상황을 고려해 보십시오.

```
csv/book_list.csv
csv/reviews_2010.csv
csv/reviews_2011.csv
csv/reviews_2012.csv
csv/reviews_2013.csv
...
csv/reviews_2025.csv
```

우리는 와일드카드 csv/reviews_20???.csv와 일치하는 파일만 저장하는 ZIP 아카이브를 프로그래밍 방식으로 생성하고자 합니다. 다행히도, Python에는 이러한 종류의 와일드카드 매칭을 위한 모듈인 glob이 있습니다.

이 간단한 프로그램은 우리의 문제를 해결합니다.

```
from zipfile import ZipFile, ZIP_DEFLATED
import glob

file_paths = glob.glob('csv/reviews_20???.csv')
with ZipFile('all_reviews.zip', 'w', ZIP_DEFLATED) as z:
    for file_path in file_paths:
        z.write(file_path)
```

CSV 데이터의 경우 idx_d2af3a2eof와 같이, 요청을 받아 ZIP 파일을 응답으로 작성하는 Django 뷰를 쉽게 만들 수 있습니다.

```
import datetime
import glob
from io import BytesIO
from zipfile import ZipFile

from django.http import HttpResponse
```

```
def review_zip(request):
    output = BytesIO()
    file_paths = glob.glob('csv/reviews_20???.csv')
    with ZipFile(output, 'w') as z:
        for file_path in file_paths:
            z.write(file_path)

    # Create the HTTP response.
    date_stamp = datetime.datetime.now().strftime('%Y%m%d')
    filename = f'all_reviews_{date_stamp}.zip'
    ctype = 'application/zip'
    cdisposition = f'attachment; filename={filename}'
    response = HttpResponse(output, content_type=ctype)
    response['Content-Disposition'] = cdisposition
    return response
```

Working with PDF files in Python

Portable Document Format (PDF) was developed in the late 1980s by Adobe Inc. in order to make a uniform page description format for printing that would operate over a variety of operating systems, firmware, and printers.

There are numerous modules available for generating dynamic PDF content. Many of these involve learning complex APIs in order to build a PDF object programmatically. As we are in the process of learning about Django and have delved deeply into its templating mechanisms, it would be nice if we could leverage this knowledge in order to generate PDF files.

Fortunately, there are several packages that will generate PDF files based on text content such as HTML or markup. One Python package that is actively being developed at the time of writing is WeasyPrint.

Again, WeasyPrint is extensively documented and interested developers can visit <https://weasyprint.readthedocs.io/> for more information. The examples here are only a brief, simple introduction to the power of WeasyPrint.

WeasyPrint can be installed using pip (or obtained on GitHub at <https://github.com/Kozea/WeasyPrint>).

```
pip install weasyprint
```

WeasyPrint can even be used at the command line to turn an HTML page into a PDF document without any coding involved. All that is required is to specify weasyprint <url> <pdffile>.

```
def review_zip(request):
    output = BytesIO()
    file_paths = glob.glob('csv/reviews_20???.csv')
    with ZipFile(output, 'w') as z:
        for file_path in file_paths:
            z.write(file_path)

    # HTTP 응답을 생성합니다.
    date_stamp = datetime.datetime.now().strftime('%Y%m%d')
    filename = f'all_reviews_{date_stamp}.zip'
    ctype = 'application/zip'
    cdisposition = f'attachment; filename={filename}'
    response = HttpResponse(output, content_type=ctype)
    response['Content-Disposition'] = cdisposition
    return response
```

Python에서 PDF 파일 작업하기

휴대용 문서 형식(PDF)은 1980년대 후반에 Adobe Inc.에 의해 인쇄를 위한 일관된 페이지 설명 형식을 만들기 위해 개발되었습니다. 이는 다양한 운영 체제, 펌웨어 및 프린터에서 작동합니다.

동적 PDF 콘텐츠를 생성하기 위해 사용할 수 있는 수많은 모듈이 있습니다. 이들 중 많은 부분은 PDF 객체를 프로그래밍 방식으로 구축하기 위해 복잡한 API를 배우는 것을 포함합니다. 우리는 Django에 대해 배우는 과정에 있으며 그 템플릿 메커니즘에 깊이 파고들었기 때문에, 이 지식을 활용하여 PDF 파일을 생성할 수 있다면 좋을 것입니다.

다행히도 HTML 또는 마크업과 같은 텍스트 콘텐츠를 기반으로 PDF 파일을 생성하는 여러 패키지가 있습니다. 현재 작성 시점에서 활발히 개발되고 있는 Python 패키지 중 하나는 WeasyPrint입니다.

다시 말해, WeasyPrint는 `idx_af11872a` extensively documented이며 관심 있는 개발자는 <https://weasyprint.readthedocs.io/>를 방문하여 더 많은 정보를 얻을 수 있습니다. 여기의 예시는 WeasyPrint의 강력함에 대한 간단한 소개일 뿐입니다.

WeasyPrint는 pip를 사용하여 설치할 수 있으며 (또는 GitHub에서 <https://github.com/Kozea/WeasyPrint>에서 얻을 수 있습니다).

```
pip install weasyprint
```

WeasyPrint는 명령줄에서도 사용되어 HTML 페이지를 PDF 문서로 변환할 수 있으며, 코딩이 필요하지 않습니다. 필요한 것은 `weasyprint <url> <pdf file>`를 지정하는 것입니다.

Results may vary as WeasyPrint does not implement all HTML formatting exactly as you might see it in a browser, and it is unaware of JavaScript or other dynamic content inside a website.

```
weasyprint https://python.org python_org.pdf
```

If we wanted to achieve what we did on the command line with a Python script, we could use the following piece of code.

```
from weasyprint import HTML, CSS
html = HTML('https://python.org')
html.write_pdf('python_org.pdf')
```

The HTML class in weasyprint is also capable of taking any local files, as well as raw HTML string content, and using those files to generate PDFs. We will use this capability in the next exercise to produce a book detail PDF that is generated from within the bookr app.

Note

This is a very brief introduction to using WeasyPrint and covers little of its extensive functionality. For further information, please visit the WeasyPrint documentation at <https://weasyprint.readthedocs.io>.

Exercise 13.03 – Generating a PDF version of a web page in Python

In the next example, we will make use of this technique and generate a PDF file that contains information about a specific book and its reviews in the Bookr system. We are familiar with this code as it just follows the Django MVT pattern, but this time the view uses WeasyPrint to render the response as a PDF rather than HTML.

1. Add a `ratings_to_stars` method to `Review` in `model.py`.

```
def rating_to_stars(self):
    if not self.rating:
        return ''
    elif isinstance(self.rating, int):
        return '*' * self.rating
    return self.rating
```

결과는 WeasyPrint가 브라우저에서 볼 수 있는 것처럼 모든 HTML 형식을 정확하게 구현하지 않으며, 웹사이트 내의 JavaScript나 기타 동적 콘텐츠를 인식하지 못하기 때문에 다를 수 있습니다.

```
weasyprint https://python.org python_org.pdf
```

우리가 Python 스크립트로 명령줄에서 수행한 작업을 달성하고자 한다면, 다음 코드를 사용할 수 있습니다.

```
from weasyprint import HTML,
CSShtml = HTML
('https://python.org')
html.write_pdf('python_org.pdf')
```

weasyprint의 HTML 클래스는 로컬 파일과 원시 HTML 문자열 콘텐츠를 모두 처리할 수 있으며, 이러한 파일을 사용하여 PDF를 생성할 수 있습니다. 우리는 다음 연습에서 bookr 앱 내에서 생성된 책 세부 정보 PDF를 만들기 위해 이 기능을 사용할 것입니다.

참고

이것은 WeasyPrint 사용에 대한 매우 간단한 소개이며 그 방대한 기능의 일부만 다룹니다. 추가 정보는 WeasyPrint 문서를 방문해 주시기 바랍니다.

<https://weasyprint.readthedocs.io>.

연습 13.03 – Python에서 웹 페이지의 PDF 버전 생성하기

다음 예제에서는 이 기술을 사용하여 Bookr 시스템에서 특정 책과 그 리뷰에 대한 정보를 포함하는 PDF 파일을 생성할 것입니다. 이 코드는 Django MVT 패턴을 따르기 때문에 익숙하지만, 이번에는 뷰가 HTML 대신 PDF로 응답을 렌더링하기 위해 WeasyPrint를 사용합니다.

1. model.py의 Review에 ratings_to_stars 메서드를 추가합니다.

```
def rating_to_stars(self):
    if not self.rating:
        return ''
    elif isinstance(self.rating, int):
        return '*' * self.rating
    return self.rating
```

2. Create a new Django template in reviews/templates/reviews/book_detail_pdf.html.

```
<html>
<head>
  <title>Bookr - {{ book.title }} </title>
</head>
<body>
  
  <div>
    <h1>{{ book.title }}</h1>
    <p>Published: {{ book.publication_date }}</p>
    <p>ISBN: {{ book.isbn }}</p>
    <p>Publisher: {{ book.publisher.name }} </p>
    <p>{% if book.contributors.all %}By {% endif %}
      {% for contributor in book.contributors.all %}
        {{ contributor.first_names }}
        {{ contributor.last_names }} {% endfor %}</p>
  </div>
  <div ></div>
  <div>
    {% if reviews %}
    <h2>Reviews:</h2>
    {% endif %}
    {% for review in reviews %}
      <p class="review-content" >{{review.content }}</p >
      <p class="review-creator">
        {{ review.creator.username }}
        {{ review.rating_to_stars }}</p>
      <p class="review-date">{{ review.date_edited }}</p>
    {% empty %}
      <h2> No reviews </h2>
    {% endfor %}
  </div>
</body>
</html>
```

3. We need to add a view function, book_detail_pdf, to reviews/views.py.

2. reviews/templates/reviews/book_detail_pdf.html에 새 Django 템플릿을 만듭니다.

```
<html>
<head>
  <title>Bookr - {{ book.title }} </title>
</head>
<body>
  
  <div>
    <h1>{{ book.title }}</h1>
    <p>발행일: {{ book.publication_date }}</p>
    <p>ISBN: {{ book.isbn }}</p>
    <p>출판사: {{ book.publisher.name }} </p>
    <p>{% if book.contributors.all %}저자 {% endif %}
      {% for contributor in book.contributors.all %}
        {{ contributor.first_names }}
        {{ contributor.last_names }} {% endfor %}</p>
  </div>
  <div ></div>
  <div>
    {% if reviews %}
    <h2>리뷰:</h2>
    {% endif %}
    {% for review in reviews %}
      <p class="review-content" >{{review.content }}</p >
      <p class="review-creator">
        {{ review.creator.username }}
        {{ review.rating_to_stars }}</p>
      <p class="review-date">{{ review.date_edited }}</p>
    {% empty %}
    <h2> 리뷰가 없습니다 </h2>
    {% endfor %}
  </div>
</body>
</html>
```

3. 리뷰/views.py에 view 함수 book_detail_pdf를 추가해야 합니다.

4. First, we need to import the HTML and CSS classes from weasyprint.

```
from weasyprint import HTML, CSS
...
```

5. We have an existing view function called `book_detail`. All of the code in this function is applicable to our current function, except for the final line that renders the view as HTML.

```
return render(request, "reviews/book_detail.html",
               context)
```

6. So, we will refactor this existing code so that the `book_detail` function calls a private function, `_book_detail(request, pk)`, which returns the request object and the context dictionary. We will reuse `_book_detail` when we write the `book_detail_pdf` function.

```
from django.template.loader import get_template
from weasyprint import HTML, CSS
...
def _book_detail(request, pk):
    book = get_object_or_404(Book, pk=pk)
    ...
    if request.user.is_authenticated:
        ...
        request.session['viewed_books'] = viewed_books
    return request, context

def book_detail(request, pk):
    request, context = _book_detail(request, pk)
    return render(request, "reviews/book_detail.html",
                  context)
```

7. With the package now installed, create a new file named `book_detail_pdf` that will hold the PDF generation logic. Inside this file, write the following code:

```
def book_detail_pdf(request, pk):
    request, context = _book_detail(request, pk):
    # Render the HTML
    template = get_template(
```

4. 먼저, weasyprint에서 HTML과 CSS 클래스를 가져와야 합니다.

```
from weasyprint import HTML, CSS
...
```

5. 우리는 book_detail이라는 기존 뷰 함수를 가지고 있습니다. 이 함수 내의 모든 코드는 현재 함수에 적용 가능하며, 뷰를 렌더링하는 마지막 줄을 제외하고는 HTML.

```
return render(request, "reviews/book_detail.html",
              context)
```

6. 그래서 우리는 idx_f7dbd0a1 이 기존 코드를 리팩토링하여 book_detail 함수가 private 함수 _book_detail(request, pk)를 호출하도록 할 것입니다. 이 함수는 요청 객체와 context 사전을 반환합니다. 우리는 book_detail_pdf 함수를 작성할 때 _book_detail을 재사용할 것입니다.

```
from django.template.loader import get_template
from weasyprint import HTML, CSS
...
def _book_detail(request, pk):
    book = get_object_or_404(Book, pk=pk)
    ...
    if request.user.is_authenticated:
        ...
        request.session['viewed_books'] = viewed_books
    return request, context

def book_detail(request, pk):
    request, context = _book_detail(request, pk)
    return render(request, "reviews/book_detail.html",
                  context)
```

7. 패키지가 이제 설치되었으므로 PDF 생성 로직을 포함할 book_detail_pdf라는 새 파일을 만듭니다. 이 파일 안에 다음 코드를 작성하세요:

```
def book_detail_pdf(request, pk):
    request, context = _book_detail(request, pk):
    # HTML 렌더링
    template = get_template(
```

```

        'reviews/book_detail_pdf.html')
html = template.render(
    context=context, request=request)
css = CSS(string='')

# Create a response
fname=f"book_detail_{book.isbn or book.id}.pdf"
cdisposition = f"inline; filename={fname}"
response = HttpResponse(
    content_type='application/pdf')
response['Content-Disposition'] = cdisposition
response['Content-Transfer-Encoding'] = 'binary'

# Write the PDF binary to the response
return HTML(
    string=html,
    base_url=request.build_absolute_uri()
).write_pdf(response, stylesheets=[css])

```

Now, let's try to understand what this code does. In the first line, we imported the HTML class from the weasyprint package, which you installed in *Step 1*:

```
from weasyprint import HTML
```

This HTML class provides us with a mechanism through which we can read the HTML content of a website if we have its URL.

In the next step, we created a new method named `book_detail_pdf()` that takes in two parameters – the request object and the primary key of the book to generate a PDF for:

```
def book_detail_pdf(request, pk):
```

Next, we passed the rendered HTML to the HTML class object we imported earlier. This caused the HTML string to be parsed by the weasyprint library and caused its HTML content to be read. Once this was done, we called the `write_pdf()` method of the HTML class object and provided the name of the file to which the content should be written:

```
pdf_stream = HTML(string=html,
base_url=request.build_absolute_uri()).write_pdf(stylesheets=[css])
```

```

        'reviews/book_detail_pdf.html')
    html = template.render(
        context=context, request=request)
    css = CSS(string='')

    # 응답 생성
    fname=f"book_detail_{book.isbn or book.id}.pdf"
    cdisposition = f"inline; filename={fname}"
    response = HttpResponse(
        content_type='application/pdf')
    response['Content-Disposition'] = cdisposition
    response['Content-Transfer-Encoding'] = 'binary'

    # PDF 이진 데이터를 응답에 작성합니다
    return HTML(
        string=html,
        base_url=request.build_absolute_uri()
        .write_pdf(response, stylesheets=[css])

```

이제 이 코드가 무엇을 하는지 이해해 봅시다. 첫 번째 줄에서 우리는 1단계에서 설치한 weasyprint 패키지에서 HTMLclass를 가져왔습니다:

```
from weasyprint import HTML
```

이 HTML classidx_598c99f2는 우리가 URL이 있을 경우 웹사이트의 HTML 콘텐츠를 읽을 수 있는 메커니즘을 제공합니다.

다음 단계에서는 요청 객체와 PDF 생성을 위한 책의 기본 키라는 두 개의 매개변수를 받는 book_detail_pdf()라는 새로운 메서드를 만들었습니다:

```
def book_detail_pdf (request, pk):
```

다음으로, 우리는 이전에 가져온 HTML 클래스 객체에 렌더링된 HTML을 전달했습니다. 이로 인해 weasyprint 라이브러리에 의해 HTML 문자열이 구문 분석되고 HTML 콘텐츠가 읽히게 되었습니다. 이 작업이 완료되면, 우리는 HTML 클래스 객체의 write_pdf() 메서드를 호출하고 콘텐츠가 작성될 파일의 이름을 제공했습니다:

```

    pdf_stream = HTML(string=html,
        base_url=request.build_absolute_uri()).write_pdf(stylesheets=[css])

```

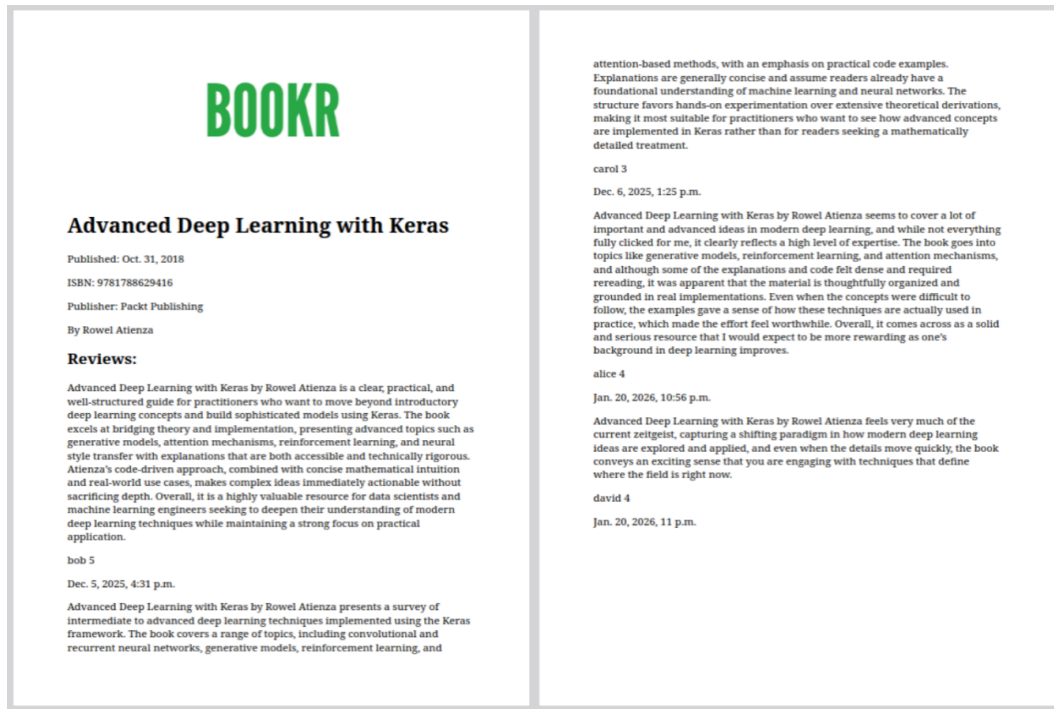


Figure 13.4: The WeasyPrint generated book detail PDF

Activity 13.01 – Adding additional functionality to a View

Including summary information in a report makes the information easier for a user to comprehend and to make comparisons with other reports.

This activity will focus on adding additional functionality to a view to include aggregated data reporting in the PDF. At the top of each Review section in the Book Detail PDF, add a simple chart that displays how many stars each movie received. This graph will be implemented as an HTML table.

The collections module in the Python Standard library provides a Counter class. You can use this class to create a function in `reviews/utils.py` called `ratings_to_histogram`. `ratings_to_histogram` is passed a list of reviews ratings and produces a list of values that will be used in the template to produce a chart that shows the distribution of ratings for the book.

```
>>> ratings_to_histogram([4, 5, 3, 4])
[('*****', 1), ('*****', 2), ('****', 1), ('***', 0), ('**', 0)]
```

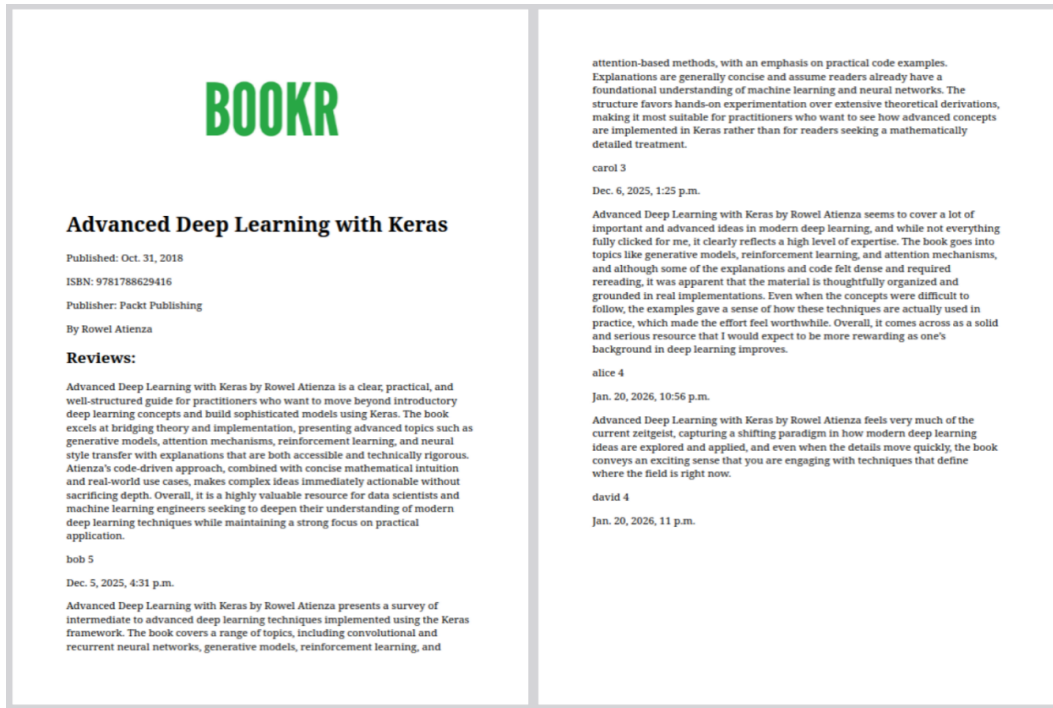


그림 13.4: WeasyPrint로 생성된 책 세부 정보 PDF

활동 13.01 – 뷰에 추가 기능 추가하기

보고서에 idx_83f8fa39 요약 정보를 포함하면 사용자가 정보를 이해하고 다른 보고서와 비교하기가 더 쉬워집니다.

이 활동은 PDF에 집계된 데이터 보고를 포함하기 위해 뷰에 추가 기능을 추가하는 데 중점을 둡니다. 책 세부 정보 PDF의 각 리뷰 섹션 상단에 각 영화가 받은 별의 수를 표시하는 간단한 차트를 추가합니다. 이 그래프는 HTML 테이블로 구현됩니다.

Python 표준 라이브러리의 collections 모듈은 Counter 클래스를 제공합니다. 이 클래스를 사용하여 reviews/utils.py에 ratings_to_histogram이라는 함수를 생성할 수 있습니다. ratings_to_histogram은 리뷰 등급 목록을 전달받아 책의 등급 분포를 보여주는 차트를 생성하는 데 사용될 값 목록을 생성합니다.

```
>>> ratings_to_histogram([4, 5, 3, 4])
[(1, 1), (2, 1), (3, 1), (4, 0), (5, 0)]
```

Include `book_rating_histogram` as a key in the context of the `book_detail_pdf` view in `reviews/views.py`. It will have a value, as follows:

```
ratings = [review.rating for review in reviews]
book_rating_histogram = ratings_to_histogram(ratings)
```

In `reviews/templates/reviews/book_detail_pdf.html`, make the following changes to incorporate the chart.

Ensure that the chart only appears on books that have received at least one review.

The histogram data can be displayed in a table with two columns inside a `div` tag with an HTML ID of `histogram`. The left column will contain the stars and the right column will contain their count. This can be achieved by iterating using a `for` loop like this:

```
{% for stars, count in book_rating_histogram %}
```

Give the existing `div` blocks of data HTML IDs of `logoheader`, `bookinfo`, `bookcover`, and `reviews`. Put the title in a `div` of its own called `title`. Add a background color to the `logoheader` `div` so that the white parts of the logo are visible.

Include some CSS so that the histogram `div` floats to the right of `bookinfo`. To contrast with the rest of the report, use white text on a dark gray background.

When you have finished this activity, you should see a simple chart like this inset in the PDF.

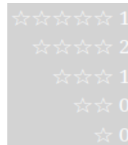


Figure 13.5: Ratings chart

The overall design will now look something like the PDF in *Figure 13.6*.

book_rating_histogram을 reviews/views.py의 book_detail_pdf 뷰의 컨텍스트에 키로 포함합니다. 값은 다음과 같습니다:

```
ratings = [review.rating for review in reviews]
book_rating_histogram = ratings_to_histogram(rating)
```

reviews/templates/reviews/book_detail_pdf.html에서 차트를 통합하기 위해 다음과 같은 변경을 합니다.

idx_850e20c4 차트는 최소한 하나의 리뷰를 받은 책에만 나타나도록 합니다.

히스토그램 데이터는 HTML ID가 histogram인 div 태그 안에 두 개의 열이 있는 테이블로 표시될 수 있습니다. 왼쪽 열에는 별이 포함되고 오른쪽 열에는 개수가 포함됩니다. 이는 다음과 같은 for 루프를 사용하여 반복함으로써 달성할 수 있습니다:

```
{% for stars, count in book_rating_histogram %}
```

기존 데이터 div 블록에 HTML ID를 logoheader, bookinfo, bookcover 및 reviews로 지정합니다. 제목을 title이라는 자체 div에 넣습니다. logoheader div에 배경색을 추가하여 로고의 흰색 부분이 보이도록 합니다.

책 정보의 오른쪽에 히스토그램 div가 떠 있도록 CSS를 포함하세요. 보고서의 나머지 부분과 대조하기 위해 어두운 회색 배경에 흰색 텍스트를 사용하세요.

이 활동을 마치면 PDF에 이렇게 삽입된 간단한 차트를 볼 수 있어야 합니다.

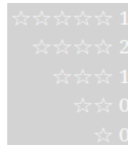


그림 13.5: 평가 차트

전체 디자인은 이제 그림 13.6의 PDF와 비슷하게 보일 것입니다.

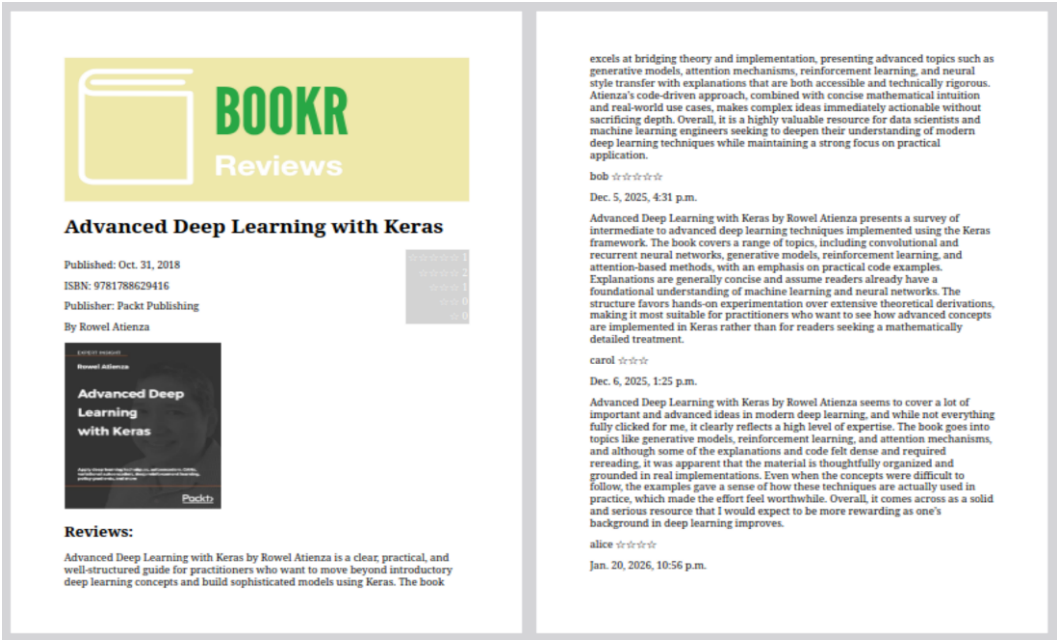


Figure 13.6: Ratings chart

So far, we have learned how we can generate different types of binary files with Python, which can help us export our data in a structured manner or help us print PDF versions of our pages. Next, we will learn how to generate graph representations of our data using Python.

Note

The solution for this activity can be found at in the Chapter13 folder in this book's GitHub repo.

Generating graphs with Plotly

Graphs are a great way to visually represent data that changes within a specific dimension. We come across graphs quite frequently in our day-to-day lives, be it weather charts for a week, stock market movements, or student performance report cards.

Plotly is a popular library for producing JavaScript enriched visualizations. It is implemented in both Python and the R programming language that is popular with statisticians and scientists. It enables the rapid visualization of data with dozens of configurable charts, including histograms, pie charts, scattergraphs, Venn diagrams, heatmaps, and many 3D plots. One of its

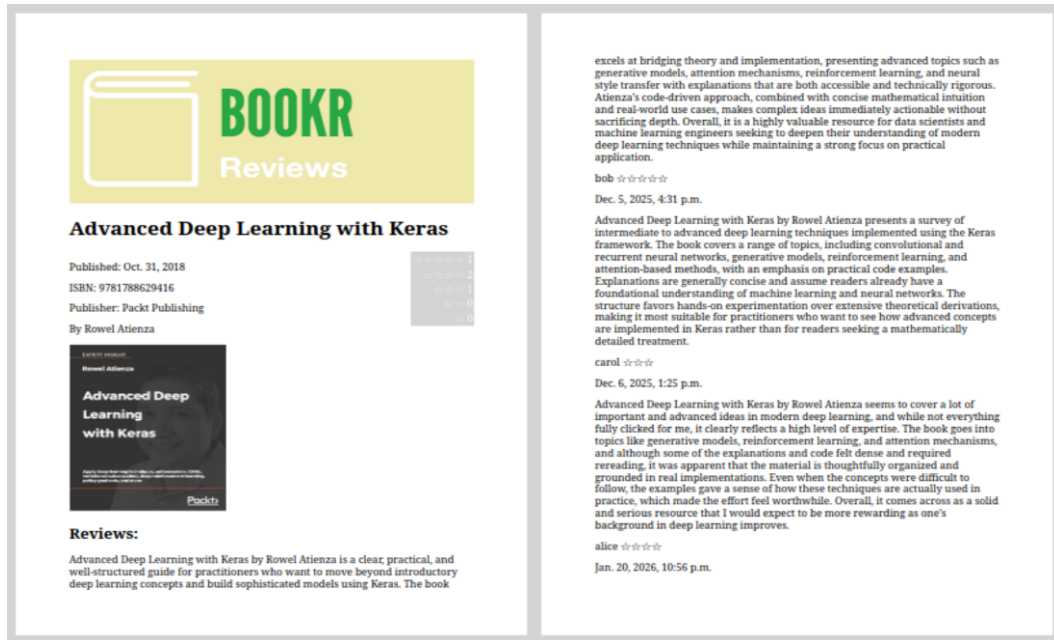


그림 13.6: 평가 차트

지금까지 우리는 Python을 사용하여 다양한 유형의 이진 파일을 생성하는 방법을 배웠습니다. 이는 데이터를 구조화된 방식으로 내보내거나 페이지의 PDF 버전을 인쇄하는 데 도움이 될 수 있습니다. 다음으로, 우리는 Python을 사용하여 데이터의 그래프 표현을 생성하는 방법을 배울 것입니다.

참고

이 활동의 솔루션은 이 책의 GitHub 리포지토리의 Chapter13 폴더에서 찾을 수 있습니다.

Plotly로 그래프 생성하기

그래프는 특정 차원 내에서 변화하는 데이터를 시각적으로 표현하는 훌륭한 방법입니다. 우리는 일상 생활에서 날씨 차트, 주식 시장 움직임 또는 학생 성적표와 같은 그래프를 자주 접합니다.

Plotly는 JavaScript로 강화된 시각화를 생성하기 위한 인기 있는 라이브러리입니다. 이는 통계학자와 과학자들 사이에서 인기 있는 Python과 R 프로그래밍 언어 모두에서 구현됩니다. 이는 히스토그램, 파이 차트, 산점도, 벤 다이어그램, 히트맵 및 많은 3D 플롯을 포함하여 수십 개의 구성 가능한 차트를 사용하여 데이터를 신속하게 시각화할 수 있게 해줍니다. 그 중 하나는

benefits is that `plotly` is particularly interesting to us due to its ease of integration with Django.

We will start with a simple pie chart example.

Suppose we have a dataset with the ratings that users have assigned to reviews in Bookr, as per *Figure 13.7*.

Rating	Count
*****	510
****	735
***	475
**	386
*	235

Figure 13.7: The star ranking dataset

Installation

To install it on your system, you can run the following command:

```
pip install plotly
```

Now that you've done that, let's take a look at how we can generate a graph visualization using `plotly`.

Setting up a figure

Before we can start generating a graph, we need to initialize a `plotly` `Figure` object, which essentially acts as a container for our graph. A `plotly` `Figure` object is quite easy to initialize; it can be done by using the following code snippet:

```
from plotly.graph_objs import graphs
figure = graphs.Figure()
```

The `Figure()` constructor from the `graph_objects` module of the `plotly` library returns an instance of the `Figure` graph container, inside which a graph can be generated. Once the `Figure` object is in place, we must generate a plot.

plotly는 Django와의 통합 용이성 덕분에 우리에게 특히 흥미롭습니다.

간단한 파이 차트 예제로 시작하겠습니다.

사용자가 Bookr의 리뷰에 부여한 평가가 포함된 데이터셋이 있다고 가정해 보겠습니다. 그림 13.7에 따라.

평가	수
*****	510
****	735
***	475
**	386
*	235

그림 13.7: 별 등급 데이터셋

설치

시스템에 idx_ce2d1370을 설치하려면 다음 명령어를 실행할 수 있습니다:

```
pip install plotly
```

이제 그 작업을 마쳤으니, plotly를 사용하여 그래프 시각화를 생성하는 방법을 살펴보겠습니다.

그림 설정

그래프 생성을 시작하기 전에, 우리는 plotly Figure 객체를 초기화해야 합니다. 이는 본질적으로 우리의 그래프를 위한 컨테이너 역할을 합니다. plotly Figure 객체는 초기화하기가 매우 쉽습니다; 다음 코드 스니펫을 사용하여 수행할 수 있습니다:

```
from plotly.graph_objs import graphs
figure = graphs.Figure()
```

plotly 라이브러리의 graph_objects 모듈에서 Figure() 생성자는 그래프를 생성할 수 있는 Figure 그래프 컨테이너의 인스턴스를 반환합니다. Figure 객체가 준비되면, 우리는 플롯을 생성해야 합니다.

Generating a plot

A plot is a visual representation of a dataset. This plot could be a pie chart, a scatter plot, a line graph, and so on. To implement a pie chart of the data in *Figure 13.7*, you can use the following code snippet:

```
labels = ['*****', '****', '***', '**', '*']
values = [510, 735, 475, 386, 235]

pie_plot = go.Pie(labels=labels, values=values, sort=False)
figure.add_trace(pie_plot)
figure.show()
```

The `add_trace()` method is responsible for adding a plotting object to the figure and generating its visualization inside the figure. By default, when the `Figure.show()` method is called, the figure will be rendered in a web browser.

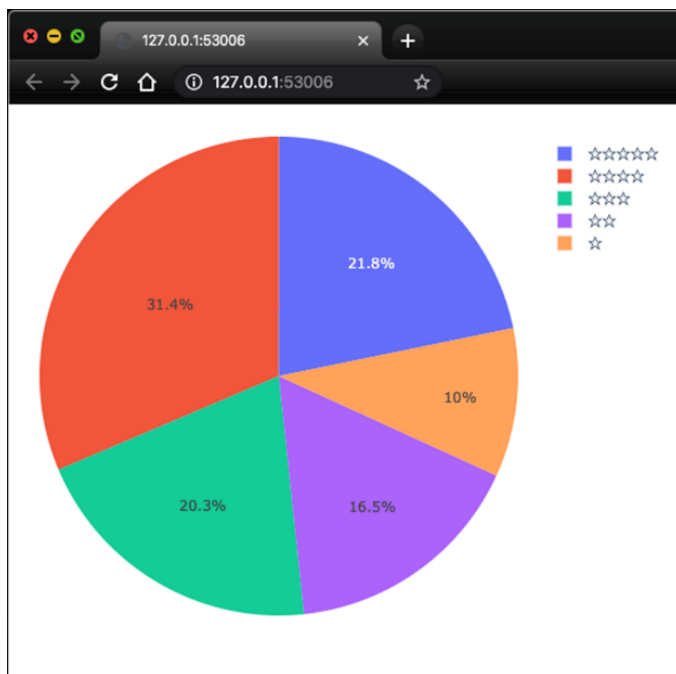


Figure 13.8: A plotly pie chart

플롯 생성하기

플롯은 데이터셋의 idx_96c2777f시각적 idx_e77bb59b 표현입니다. 이 플롯은 파이 차트, idx_0bc74a51산점도, 선 그래프 등일 수 있습니다. 그림 13.7의 데이터를 사용하여 파이 차트를 구현하려면 다음 코드 스니펫을 사용할 수 있습니다:

```
labels = ['*****', '****', '***', '**', '*']
values = [510, 735, 475, 386, 235]

pie_plot = go.Pie(labels=labels, values=values, sort=False)
figure.add_trace(scatter_plot)
figure.show()
```

add_trace() 메서드는 플로팅 객체를 그림에 추가하고 그림 안에서 그 시각화를 생성하는 역할을 합니다. 기본적으로 Figure.show() 메서드가 호출되면 idx_89c0e6dfigureidx_d0ba9497가 웹 브라우저에 렌더링됩니다.

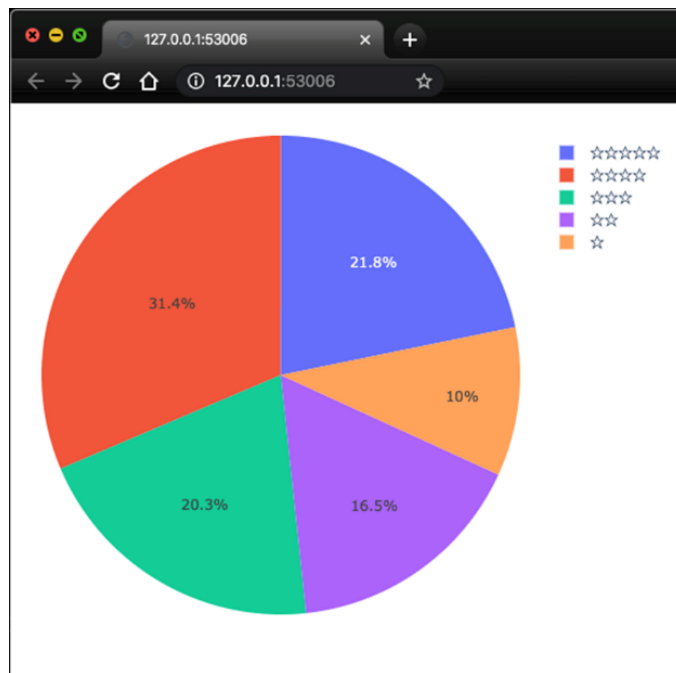


그림 13.8: 플롯리 파이 차트

Rendering a plot on a web page

Once the plot has been added to the figure, it can be rendered on a web page by calling the `plot()` method from the offline plotting module of the `plotly` library. This is shown in the following code snippet:

```
from plotly.offline import plot
visualization_html = plot(figure, output_type='div')
```

The `plot()` method takes two primary parameters: the first is the figure that needs to be rendered and the second is the HTML tag of the container inside which the figure HTML will be generated. The `plot` method returns fully integrated HTML that can be embedded in any web page or made a part of the template to render a graph.

Now, with this understanding of how graph plotting works, let's try a hands-on exercise where we will generate a graph for our sample dataset.

Advanced graphs – bubble plots

For our next example, we will develop a bubble plot. This is a scatter plot with weighted markers. In the dataset we are dealing with, we will be plotting average reviews for a variety of books and setting the size of the markers.

For our dataset, we will use this CSV file, which contains information on books and their ratings.

```
publication_year,average_rating,title,num_rating
2014,2.4285714285714284,'Book A',7
2016,2.25,'Book B',4
2018,4.0,'Book C',5
2016,2.3333333333333335,'Book D',3
2011,4.0,'Book E',1
2015,2.875,'Book F',16
2019,2.6315789473684212,'Book G',19
2020,3.375,'Book H',16
2018,5.0,'Book I',1
2011,2.6666666666666665,'Book J',9
2013,5.0,'Book K',1
2013,3.5384615384615383,'Book L',13
2016,2.5789473684210527,'Book M',19
2010,2.2941176470588234,'Book N',17
2014,3.3333333333333335,'Book O',12
2015,3.0526315789473686,'Book P',19
```

웹 페이지에 플롯 렌더링

idx_2cb95f10plot이 그림에 추가되면, plotly 라이브러리의 오프라인 플로팅 모듈에서 plot() 메서드를 호출하여 웹 페이지에 렌더링할 수 있습니다. 이는 다음 코드 스니펫에서 보여집니다:

```
from plotly.offline import plot
visualization_html = plot.figure, output_type='div')
```

plot() 메서드는 두 가지 주요 매개변수를 사용합니다: 첫 번째는 렌더링해야 하는 그림이고, 두 번째는 그림 HTML이 생성될 컨테이너의 HTML 태그입니다. plot 메서드는 어떤 웹 페이지에도 삽입할 수 있는 완전히 통합된 HTML을 반환하거나 그래프를 렌더링하기 위한 템플릿의 일부로 만들 수 있습니다.

이제 그래프 플로팅이 어떻게 작동하는지 이해했으니, 샘플 데이터 세트를 위해 그래프를 생성하는 실습을 해보겠습니다.

고급 그래프 – 버블 플롯

다음 예제에서는 버블 플롯을 개발할 것입니다. 이는 가중치가 있는 마커가 있는 산점도입니다. 우리가 다루고 있는 데이터 세트에서는 다양한 책에 대한 평균 리뷰를 플로팅하고 마커의 크기를 설정할 것입니다.

우리 데이터셋을 위해, 우리는 책과 그 평가에 대한 정보를 포함하는 이 CSV 파일을 사용할 것입니다.

```
publication_year,average_rating,title,num_rating
2014,2.4285714285714284,'Book A',7
2016,2.25,'Book B',4
2018,4.0,'Book C',5
2016,2.3333333333333335,'Book D',3
2011,4.0,'Book E',1
2015,2.875,'Book F',16
2019,2.6315789473684212,'Book G',19
2020,3.375,'Book H',16
2018,5.0,'책 I',1
2011,2.6666666666666665,'책 J',9
2013,5.0,'책 K',1
2013,3.5384615384615383,'책 L',13
2016,2.5789473684210527,'책 M',19
2010,2.2941176470588234,'책 N',17
2014,3.3333333333333335,'책 O',12
2015,3.0526315789473686,'책 P',19
```

```
2020,2.857142857142857,'Book Q',7
2018,3.0,'Book R',16
2015,2.5,'Book S',18
2020,3.230769230769231,'Book T',13
```

We need to import `plotly` and the `graph_objects` module and then read the CSV file.

```
import csv
import plotly as py
import plotly.graph_objects as go

with open('data.csv') as f:
    reader = csv.DictReader(f)
    reviews = list(reader)
```

The `Scatter` constructor takes in the values for the x axis and y axis and returns an object that can be used to build a scatter plot. In our example we are plotting the publication year on the x axis and the average rating on the y axis. We are defining the marker using the number of ratings present in the dataset. This way, books with more ratings will look more prominent than those with fewer ratings.

```
scatter_0 = go.Scatter(
    x=[review['publication_year'] for review in reviews],
    y=[review['average_rating'] for review in reviews],
    text=[review['title'] for review in reviews],
    mode='markers+text',
    textposition='middle center',
    marker=dict(
        size=[int(review['num_rating'])
              for review in reviews],
        sizemode='area',
        sizeref=0.001,
    )
)
```

Once the scatter plot object has been generated, the next step is to add this plot to `Figure`. This can be done as follows:

```
figure = go.Figure()
figure.add_trace(scatter_0)
```

```
2020,2.857142857142857, 'Book Q',7
2018,3.0, 'Book R',16
2015,2.5, 'Book S',18
2020,3.230769230769231, 'Book T',13
```

우리는 `idx_789c30cc` `plotly`와 `graph_objects` 모듈을 가져오고 CSV 파일을 읽어야 합니다.

```
import csv
import plotly as py
import plotly.graph_objects as go

with open('data.csv') as
    f:reader =
        csv.DictReader(f)
        reviews = list
            (reader)
```

Scatter 생성자는 x축과 y축의 값을 받아들이고 산점도를 구축하는 데 사용할 수 있는 객체를 반환합니다. 우리의 예에서는 x축에 출판 연도를, y축에 평균 평점을 플로팅하고 있습니다. 우리는 데이터셋에 있는 평점의 수를 사용하여 마커를 정의하고 있습니다. 이렇게 하면 평점이 더 많은 책이 평점이 적은 책보다 더 두드러져 보입니다.

```
scatter_0 = go.Scatter(
    x=[review['publication_year'] for review in reviews],
    y=[review['average_rating'] for review in reviews],
    text=[review['title'] for review in reviews],
    mode='markers+text',
    textposition='middle center',
    marker=dict(
        size=[int(review['num_rating'])
              for review in reviews],
        sizemode='area',
        sizeref=0.001,
    ),
)
```

산점도 객체가 생성되면, 다음 단계는 이 플롯을 Figure에 추가하는 것입니다. 이는 다음과 같이 할 수 있습니다:

```
figure = go.Figure()
figure.add_trace(scatter_0)
```

```
figure.update_xaxes(range=[2009, 2021])
figure.update_yaxes(range=[-0.2, 5.2])
figure.show()
```

The resulting bubble plot will appear as in *Figure 13.9*.

In the next exercise, we will use incorporate this kind of plotly object into a Django view that takes its data from a dynamic query of the database.

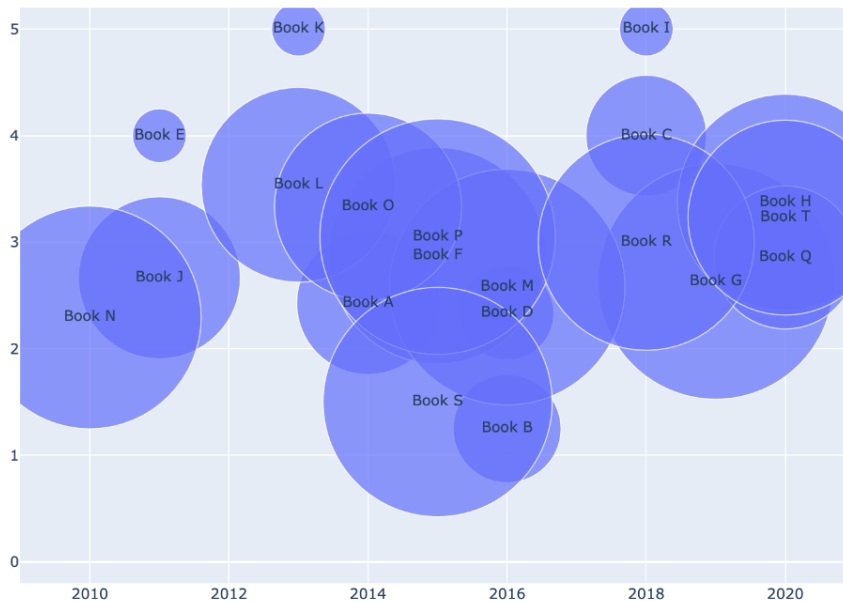


Figure 13.9: The bubble plot in Plotly

Exercise 13.4 – Generating graphs in Python

In this exercise, we will add a Django view that creates a bubble plot of the bookr app's **Review** data.

1. Edit `reviews/views.py` and include the required import statements.

```
import datetime

import plotly.graph_objects as go
from django.db.models import Avg
from plotly.offline import plot
```

```
figure.update_xaxes(range=[2009, 2021])
figure.update_yaxes(range=[-0.2, 5.2])
figure.show()
```

Theidx_779c0501resultingidx_d777a7b6 버블 플롯은 그림 13.9와 같이 나타납니다.

다음 연습에서는 이 종류의 plotly 객체를 데이터베이스의 동적 쿼리에서 데이터를 가져오는 Django 뷰에 통합할 것입니다.

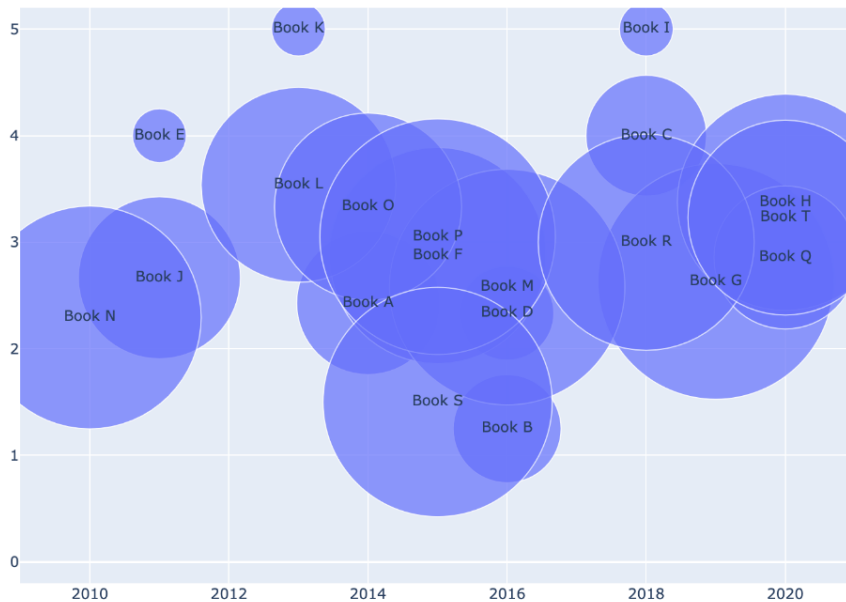


그림 13.9: Plotly의 버블 플롯

연습 13.4 – Python에서 그래프 생성

이번 연습에서는 책r 앱의 Reviewdata의 버블 플롯idx_79a590af을 생성하는 Django 뷰를 추가할 것입니다.

1. reviews/views.py를 수정하고 필요한 import 문을 포함하세요.

```
import datetime

import plotly.graph_objects as go
from django.db.models import Avg
from plotly.offline import plot
```

2. Define the `review_statistics` view with the arguments for `min_publication_year` and `max_publication_year`. Calculate the start of the first year and the end of the second year so that we can do inclusive searches on this date range. (For example, if `max_publication_year` is 2025, we want our search to include publications from up to the last second of December 31, 2025.)

```
def review_statistics(request,
    min_publication_year, max_publication_year):
    min_year = datetime.datetime(
        min_publication_year, 1, 1)
    max_year = datetime.datetime(
        max_publication_year, 12, 31, 23, 59, 59)
```

3. Now we can retrieve the reviews filtered on our date range and include aggregates of the average rating and the count of ratings.

```
reviews = Review.objects.filter(
    book__publication_date__gte=min_year,
    book__publication_date__lte=max_year).values(
    'book__id', 'book__title',
    'book__publication_date'
).annotate(average_rating=Avg('rating'))
).annotate(num_rating=Count('rating'))
```

4. We can create the scatter plot using the class declaration that we discussed in the prior section.

```
scatter_0 = go.Scatter(
    x=[review['book__publication_date'].year
        for review in reviews],
    y=[review['average_rating']
        for review in reviews],
    text=[review['book__title']
        for review in reviews],
    mode='markers+text',
    textposition='middle center',
    marker=dict(
        size=[review['num_rating']
            for review in reviews],
        sizemode='area',
```

2. `review_statistics` 뷰를 `min_publication_year`와 `max_publication_year` 인수로 정의합니다. 이 날짜 범위에서 포함 검색을 수행할 수 있도록 첫 번째 연도의 시작과 두 번째 연도의 끝을 계산합니다. (예를 들어, `max_publication_year`가 2025인 경우, 우리는 2025년 12월 31일의 마지막 초까지의 출판물을 포함하도록 검색하고자 합니다.)

```
def review_statistics(request,
    min_publication_year, max_publication_year):
    min_year = datetime.datetime(
        min_publication_year, 1, 1)
    max_year = datetime.datetime(
        max_publication_year, 12, 31, 23, 59, 59)
```

3. 이제 우리는 날짜 범위에 따라 필터링된 리뷰를 검색하고 평균 평점과 평점 수의 집계를 포함할 수 있습니다.

```
reviews = Review.objects.filter(
    book__publication_date__gte=min_year,
    book__publication_date__lte=max_year).values(
    'book__id', 'book__title',
    'book__publication_date'
).annotate(average_rating=Avg('rating'))
).annotate(num_rating=Count('rating'))
```

4. 우리는 이전 섹션에서 논의한 클래스 선언을 사용하여 산점도를 만들 수 있습니다.

```
scatter_0 = go.Scatter(
    x=[review['book__publication_date'].year
        for review in reviews],
    y=[review['average_rating']
        for review in reviews],
    text=[review['book__title']
        for review in reviews],
    mode='markers+text',
    textposition='middle center',
    marker=dict(
        size=[review['num_rating']
            for review in
            reviews],
        sizemode='area',
```

```
        sizeref=0.001,
    )
)
```

5. Now we can create the layout and figure and annotate the axes with the publication date and rating scales.

```
layout = go.Layout(autosize=True, height=900)
fig = go.Figure(data=[scatter_0], layout=layout)
fig.update_xaxes(range=[min_publication_year,
                        max_publication_year])
fig.update_yaxes(range=[-0.2, 5.2])
```

6. We will create a div element and render it using a Django template.

```
plotly_div = plot(fig, output_type='div')
return render(request, "reviews/statistics.html",
              context={'plotly_div': plotly_div})
```

7. The Django template can be defined as follows.

```
<!DOCTYPE HTML>
<html>
<head>
    <title>Review Bubbleplot</title>
</head>
<body>
    {% autoescape off %}
    {{ plotly_div }}
    {% endautoescape %}
</body>
</html>
```

8. Now we will need to add the following path to `reviews/urls.py`:

```
path('reviews/statistics/'
     '<int:min_publication_year>/'
     '<int:max_publication_year>',
```

```
        sizeref=0.001,
    )
)
```

5. 이제 `idx_dadad592we`를 사용하여 레이아웃과 도형을 만들고 `publicationdate` 및 `rating` 스케일로 축을 주석을 달 수 있습니다.

```
layout = go.Layout(autosize=True, height=900)
fig = go.Figure(data=[scatter_0], layout=layout)
fig.update_xaxes(range=[min_publication_year,
                        max_publication_year])
fig.update_yaxes(range=[-0.2, 5.2])
```

6. 우리는 `idx_88bd94fa` div 요소를 생성하고 Django 템플릿을 사용하여 렌더링할 것입니다.

```
plotly_div = plot(fig, output_type='div')
return render(request, "reviews/statistics.html",
              context={'plotly_div': plotly_div})
```

7. Django 템플릿은 다음과 같이 정의할 수 있습니다.

```
<!DOCTYPE HTML>
<html>
<head>
    <title>리뷰 버블 플롯</title>
</head>
<body>
    {% autoescape off %}
    {{ plotly_div }}
    {% endautoescape %}
</body>
</html>
```

8. 이제 `reviews/urls.py`에 다음 경로를 추가해야 합니다:

```
path('reviews/statistics/'
     '<int:min_publication_year>/'
     '<int:max_publication_year>',
```

```
views.review_statistics,  
name='review_statistics'),
```

9. By using `plotly` in a Django view, we have opened our Django applications to the sophisticated data visualizations that this library offers. Now we will learn about dynamic image generation using the Pillow library.

Generating images in Python with Pillow

We encountered Pillow in *Chapter 8, File Uploads with Django Forms*, where we used it to resize uploaded images.

While Pillow contains an extensive array of functionality for image creation and manipulation, we won't investigate these aspects in their entirety. The examples in this section look at simple image transformations such as rotation and resizing.

First, we need to import the `Image` class from the PIL module.

```
from PIL import Image
```

Now, we will define a private function that takes two image paths as arguments.

```
def _logo_transformations(logo, background):
```

We will load these images using the `Image.open` method and convert them to a common color mode.

```
bg = Image.open(background)  
bg_img = bg.convert('RGBA')  
logo_img = Image.open(logo).convert('RGBA')
```

Now we will paste the logo onto the background at position (100, 50).

```
bg_img.paste(logo_img, box=(100, 50), mask=logo_img)
```

Then we will rotate another logo image 135 degrees counterclockwise and paste it further to the right of the first logo.

```
rot = logo_img.rotate(135, Image.NEAREST, expand=1)  
bg_img.paste(rot, box=(650, 50), mask=rot)
```

```
views.review_statistics,
name='review_statistics'),
```

9. idx_59ab1c09를 사용하여 Django 뷰에서 plotly를 사용함으로써, 우리는 idx_2d0c9049 우리의 Django 애플리케이션을 이 라이브러리가 제공하는 정교한 데이터 시각화에 열었습니다. 이제 Pillow 라이브러리를 사용한 동적 이미지 생성에 대해 배워보겠습니다.

Pillow로 Python에서 이미지 생성하기

우리는 8장에서 Pillow를 만났습니다. Django Forms를 사용한 파일 업로드에서, 우리는 업로드된 이미지를 크기 조정하는 데 사용했습니다.

idx_367caa19 Pillow idx_ec46c76a에는 이미지 생성 및 조작을 위한 광범위한 기능이 포함되어 있지만, 우리는 이러한 측면을 전부 조사하지 않을 것입니다. 이 섹션의 예제는 회전 및 크기 조정과 같은 간단한 이미지 변환을 살펴봅니다.

먼저, PIL 모듈에서 Image 클래스를 가져와야 합니다.

```
from PIL import Image
```

이제 두 개의 이미지 경로를 인수로 받는 비공식 함수를 정의하겠습니다.

```
def _logo_transformations(logo, background):
```

이 이미지를 Image.open 메서드를 사용하여 로드하고 공통 색상 모드로 변환하겠습니다.

```
bg = Image.open(background)
bg_img = bg.convert('RGBA')
logo_img = Image.open(logo).convert('RGBA')
```

이제 (100, 50) 위치에 배경에 로고를 붙여넣겠습니다.

```
bg_img.paste(logo_img, box=(100, 50), mask=logo_img)
```

그런 다음 다른 로고 이미지를 135도 반시계 방향으로 회전시키고 첫 번째 로고의 오른쪽에 더 붙여넣겠습니다.

```
rot = logo_img.rotate(135, Image.NEAREST, expand=1)
bg_img.paste(rot, box=(650, 50), mask=rot)
```

Then we will resize the logo so that it has a 1:1 aspect ratio, paste it at the bottom of the background, and return the completed image.

```
resize = logo_img.resize((216, 216), Image.NEAREST)
bg_img.paste(resize, box=(100, 400), mask=resize)

return bg_img
```

Finally, we have the code to call the private method.

```
# bookr Logo: 432 x 216 pixels
logo = 'reviews/static/reviews/logo.png'
# wooden background: 1280 x 720 pixels
background = 'media/backgrounds/wood_background.jpg'
transformed_img = _logo_transformations(logo, background)
transformed_img.save('logo_transforms.png', quality=95)
```

The wood background image is available in the Chapter13 folder in this book's GitHub repo.

Running this code will create the following image.



Figure 13.10: *logo_transformation.png*

You will realize very quickly that this is not a very intuitive way to manipulate graphics, and it is very time-consuming to get even a basic effect when using this library compared to the time

그런 다음 로고의 비율을 1:1로 조정하고 배경 하단에 붙여넣은 후 완성된 이미지를 반환합니다.

```
resize = logo_img.resize((216, 216), Image.NEAREST)
bg_img.paste(resize, box=(100, 400), mask=resize)

return bg_img
```

마지막으로, 우리는 비공식 메서드를 호출하는 코드를 가지고 있습니다.

```
# bookr 로고: 432 x 216 픽셀
logo = 'reviews/static/reviews/logo.png'

# 나무 배경: 1280 x 720 픽셀
background =
'media/backgrounds/wood_background.jpg'
transformed_img =
_logo_transformations(logo, background)
transformed_img.save('logo_transforms.png', quality=95)
```

Theidx_2d080986 나무 배경 이미지는 이 책의 GitHub 저장소의 Chapter13 폴더에서 사용할 수 있습니다. Running thisidx_04eedf3b 코드는 다음 이미지를 생성합니다.



Figure 13.10: logo_transformation.png

당신은 이것이 그래픽을 조작하는 매우 직관적인 방법이 아니라는 것을 매우 빠르게 깨닫게 될 것이며, 이 라이브러리를 사용할 때 기본 효과를 얻는 데 매우 많은 시간이 소요됩니다.

taken with a standard image application such as Microsoft Paint or Adobe PhotoShop. The benefit of this approach is that it is very easy to automate this code and apply it to scripting images or dynamically rendering an image on a web server.

As we saw in the earlier examples, it is very easy to wrap this kind of binary data in a Django `HttpResponse`. Here is a view that we can add to `reviews/views.py` that uses the `_logo_transformations` private function and returns a PNG image.

```
from PIL import Image
...

def logo_transformations(request):
    logo = 'reviews/static/reviews/logo.png'
    background = 'media/backgrounds/wood_background.jpg'
    transformed_img = _logo_transformations(
        logo, background)

    response = HttpResponse(content_type="image/png;")
    transformed_img.save(response, 'PNG', quality=95)
    return response
```

Activity 13.02 – Creating dynamic graphics in Django with Pillow

The idea of this example is to generate a background image for a reviewer's profile that contains a pile of books that they have reviewed. The code is a dynamic version of our previous example.

The covers of the books that the user has reviewed are kept in a list called `covers`. Then, when we iterate through the covers, each cover is pasted onto the background at a different angle and position. This resembles a jumbled stack of books that the user has reviewed.

This activity will help consolidate your knowledge of Pillow and its use within Django. You will learn that Pillow can be used for generating JPEG images:

1. We need to include a `wood_background.jpg` file in `media/backgrounds`. You can copy the one in this activity from the Chapter13 folder in this book's GitHub repository supply your own.
2. Create a function in `reviews/views.py` called `get_review_profile_img`. It will take `image_type`, `user_id`, `background_path` as arguments and return an image.

Microsoft Paint 또는 Adobe PhotoShop과 같은 표준 이미지 애플리케이션으로 촬영한 것입니다. 이 접근 방식의 이점은 이 코드를 자동화하고 스크립팅 이미지에 적용하거나 웹 서버에서 이미지를 동적으로 렌더링하는 것이 매우 쉽다는 것입니다.

앞서 본 예제에서 알 수 있듯이, 이러한 종류의 이진 데이터를 Django HttpResponse로 감싸는 것은 매우 쉽습니다. 여기서 `idx_c493bdd5`는 `_logo_transformations` 비공식 함수인 `idx_1a309c43`을 사용하는 `reviews/views.py`에 추가할 수 있는 뷰입니다. 그리고 PNG 이미지를 반환합니다.

```
from PIL import Image
...

def logo_transformations(request):
    logo = 'reviews/static/reviews/logo.png'
    background = 'media/backgrounds/wood_background.jpg'
    transformed_img = _logo_transformations(
        로고, 배경)

    response = HttpResponse(content_type="image/png;")
    transformed_img.save(response, 'PNG', quality=95)
    return response
```

활동 13.02 – Pillow를 사용한 Django에서 동적 그래픽 만들기

이 예제의 아이디어는 리뷰어의 프로필을 위한 배경 이미지를 생성하는 것으로, 그들이 리뷰한 책들이 쌓여 있는 모습을 담고 있습니다. 이 코드는 이전 예제의 동적 버전입니다.

사용자가 리뷰한 책의 표지는 `covers`라는 리스트에 저장됩니다. 그런 다음, `covers`를 반복할 때마다 각 표지가 배경에 다른 각도와 위치로 붙여집니다. 이는 사용자가 리뷰한 책들이 뒤죽박죽 쌓인 모습과 유사합니다.

이 활동은 Pillow에 대한 지식을 강화하고 Django 내에서의 사용을 도와줄 것입니다. Pillow는 JPEG 이미지를 생성하는 데 사용될 수 있다는 것을 배울 것입니다:

1. `media/backgrounds`에 `wood_background.jpg` 파일을 포함해야 합니다. 이 활동에서 이 책의 GitHub 리포지토리의 Chapter13 폴더에서 복사하거나 직접 제공할 수 있습니다.
2. `reviews/views.py`에 `get_review_profile_img`라는 함수를 만듭니다. 이 함수는 `image_type`, `user_id`, `background_path`를 인수로 받아 이미지를 반환합니다.

3. In the function, create a list called `covers` that consists of the paths of the cover images relative to the `MEDIA_ROOT`. Get this list by filtering Review objects so that `creator_id` matches the parameter `user_id`.
4. Combine the `MEDIA_ROOT` path with `background_path` and open this image and convert it to 'RGBA'. We need to use RGBA rather than RGB because we are making use of the Alpha-Channel of the images in order to deal with transparency issues.
5. We will create a list of orientations and heights of the books. The former will be used to specify degree rotations, and the latter will be used to specify the Y-axis positioning of each book. These values give satisfying results, but consider trying your own:

```
orientations = [30, -15, 15, 0, -30, 45, -45]
heights = [125, 150, 175, 150, 125, 100, 150]
```

6. We will define a left and right displacement value for image positioning. These values account for the size of the cover image when calculating each position on the X-axis of the background image:

```
left_displacement = -10
right_displacement = 280
```

7. Now enumerate through the `covers` list and assign an orientation and height to the cover. The for loop will begin like this: `for i, cover in enumerate(covers):`
 - a. The X-axis position of the book is determined proportionately to the length of the image. We can assign a value like this:

```
x_pos = int(left_displacement + i*(
    bg_img-right_displacement)/len(covers))
```

- b. Open the cover image and convert it to RGBA.
 - c. Rotate the image according to the specific orientation and paste it at the X and Y positions derived above over the background image.
8. If the image type is 'jpg', we need to convert it back to RGB. Pillow's Image class can save JPEG files, but RGBA files first need to be converted to RGB.
9. Return the image.

Now that you have created a function that produces the review profile image, it needs to be wrapped in a view that takes a request argument and returns a response.

3. 함수에서 MEDIA_ROOT에 상대적인 커버 이미지 경로로 구성된 covers라는 목록을 만듭니다. 이 목록은 creator_id가 매개변수 user_id와 일치하도록 Review 객체를 필터링하여 가져옵니다.
4. MEDIA_ROOT 경로와 background_path를 결합하고 이 이미지를 열어 'RGBA'로 변환합니다. 투명성 문제를 처리하기 위해 이미지의 알파 채널을 사용하고 있으므로 RGB 대신 RGBA를 사용해야 합니다.
5. 우리는 책의 방향과 높이 목록을 만들 것입니다. 전자는 회전 각도를 지정하는 데 사용되고, 후자는 각 책의 Y축 위치를 지정하는 데 사용됩니다. 이러한 값은 만족스러운 결과를 제공하지만, 자신의 값을 시도해 보세요:

```
orientations = [30, -15, 15, 0, -30, 45, -45]
heights = [125, 150, 175, 150, 125, 100, 150]
```

6. 우리는 이미지 위치 지정을 위한 왼쪽 및 오른쪽 변위 값을 정의할 것입니다. 이 값들은 배경 이미지의 X축에서 각 위치를 계산할 때 커버 이미지의 크기를 고려합니다:

```
left_displacement = -10
right_displacement = 280
```

7. 이제 커버 목록을 열거하고 커버에 방향과 높이를 할당합니다. for 루프는 다음과 같이 시작됩니다: for i, cover in enumerate(covers):
 - a. 책의 X축 위치는 이미지의 길이에 비례하여 결정됩니다. 우리는 다음과 같이 값을 할당할 수 있습니다:

```
x_pos = int(left_displacement + i*(
    bg_img-right_displacement)/len(covers))
```

- b. 커버 이미지를 열고 RGBA로 변환합니다.
 - c. 특정 방향에 따라 이미지를 회전시키고 위에서 유도된 X 및 Y 위치에 배경 이미지 위에 붙여넣습니다.
8. 이미지 유형이 'jpg'인 경우, 다시 RGB로 변환해야 합니다. Pillow의 Image 클래스는 JPEG 파일을 저장할 수 있지만, RGBA 파일은 먼저 RGB로 변환해야 합니다.
9. 이미지를 반환합니다.

리뷰 프로필 이미지를 생성하는 함수를 만들었으므로, 이제 요청 인수를 받아들이고 응답을 반환하는 뷰로 감싸야 합니다.

10. Create a view in `reviews/view.py` that takes `request` and `user_id` arguments called `reviewer_profile_jpg`. It will call `get_review_profile_img`, specifying `'jpg'` for `image_type`, `user_id`, and `'backgrounds/wood_background.jpg'` as the `background_path`, returning an image.
11. Create a `BytesIO` object and save the image to it.
12. Return a `HttpResponse` with the value of the `BytesIO` object and `content_type` set to `'image/jpeg'`.
13. Add a URL pattern of `'reviews/profile/<int:user_id>/background_jpg/'` for `reviewer_profile_jpg` to the `urlpatterns` in `reviews/urls.py`. By the end of this activity, you will be able to see an image like the following one when you visit a profile background image, such as `http://localhost:8000/reviews/profile/8/background_jpg`.

You have now gained some understanding of how Pillow can be used in Django to create dynamic images.



Figure 13.11: The profile background image

Note

The solution for this activity can be found at in the Chapter13 folder in this book's GitHub repo.

10. reviews/view.py에서 request와 user_id 인수를 받는 reviewer_profile.jpg라는 뷰를 생성합니다. 'jpg'를 image_type으로, user_id를, 'backgrounds/wood_background.jpg'를 background_path로 지정하여 get_review_profile_img를 호출하고 이미지를 반환합니다.
11. BytesIO 객체를 생성하고 이미지를 그 안에 저장합니다.
12. BytesIO 객체의 값을 가진 HttpResponse를 반환하고 content_type을 'image/jpeg'로 설정합니다.
13. reviews/urls.py의 urlpatterns에 reviewer_profile.jpg를 위한 'reviews/profile/<int:user_id>/background.jpg/' URL 패턴을 추가합니다. 이 활동이 끝나면 <http://localhost:8000/reviews/profile/8/background.jpg>와 같은 프로필 배경 이미지를 방문할 때 다음과 같은 이미지를 볼 수 있습니다.

이제 Pillow가 Django에서 동적 이미지를 생성하는 데 어떻게 사용될 수 있는지에 대한 이해를 얻었습니다.



그림 13.11: 프로필 배경 이미지

메모

이 활동의 솔루션은 이 책의 GitHub 리포지토리의 Chapter13 폴더에서 찾을 수 있습니다.

Summary

In this chapter, we looked at how we can deal with binary files and how Python's standard library, which comes pre-loaded with the necessary tools, allows us to handle commonly used file formats such as CSV. We then moved on to learning how to read and write CSV files in Python using Python's `csv` module. Later, we worked with the `XlsxWriter` package, which allows us to generate Microsoft Excel-compatible files right from our Python environment without us having to worry about the internal formatting of the file.

Next, we investigated the use of `ziplib` to generate ZIP files and used the `weasyprint` library to generate PDF versions of HTML pages. This skill can come in handy when we want to provide our users with an easy option to print the HTML version of our page with any added CSS styling of our choosing. Then we discussed how we can generate interactive graphs in Python and render them as HTML pages that can be viewed inside the browser using the `plotly` library. In the last section of this chapter, we took a deep dive into `Pillow` and learnt how it can be used to create a dynamic image from a Django request.

In the next chapter, we will look at how we can test the different components we have been implementing in the previous chapters to make sure that code changes do not break our website's functionality.

요약

이 장에서는 이진 파일을 처리하는 방법과 필요한 도구가 미리 로드된 Python의 표준 라이브러리가 CSV와 같은 일반적으로 사용되는 파일 형식을 처리할 수 있도록 하는 방법을 살펴보았습니다. 그런 다음 Python의 csv 모듈을 사용하여 Python에서 CSV 파일을 읽고 쓰는 방법을 배우는 것으로 넘어갔습니다. 이후에는 XlsxWriter 패키지를 사용하여 Microsoft Excel 호환 파일을 Python 환경에서 생성하는 방법을 배웠습니다. 이 과정에서 파일의 내부 형식에 대해 걱정할 필요가 없었습니다.

다음으로 ziplib를 사용하여 ZIP 파일을 생성하는 방법을 조사하고 weasyprint 라이브러리를 사용하여 HTML 페이지의 PDF 버전을 생성했습니다. 이 기술은 사용자가 선택한 CSS 스타일을 추가하여 HTML 페이지의 인쇄 옵션을 쉽게 제공하고자 할 때 유용할 수 있습니다. 그런 다음 Python에서 대화형 그래프를 생성하고 이를 plotly 라이브러리를 사용하여 브라우저 내에서 볼 수 있는 HTML 페이지로 렌더링하는 방법에 대해 논의했습니다. 이 장의 마지막 섹션에서는 Pillow에 대해 깊이 파고들어 Django 요청에서 동적 이미지를 생성하는 데 어떻게 사용될 수 있는지 배웠습니다.

다음 장에서는 이전 장에서 구현한 다양한 구성 요소를 테스트하여 코드 변경이 웹사이트의 기능을 손상시키지 않도록 하는 방법을 살펴보겠습니다.

14

Testing Your Django Applications

In the preceding chapters, we focused on building our web application in Django by writing different components such as database models, views, and templates. We did all that to provide our users with an interactive application where they can create a profile and write reviews for the books they have read.

Apart from building and running the application, there is another important aspect of making sure that the application code works the way we expect it to work. This is ensured by a technique called **testing**. In testing, we run the different parts of the web application and check if the output of the executed component matches the output we expected. If the output matches, we say that the component was tested successfully, while if the output doesn't match, we say that the component failed to work as intended.

In this chapter, as we go through the different sections, we will learn why testing is important, the different ways to test a web application, and how we can build a strong testing strategy that will help us ensure that the web application we build is robust.

In this chapter, we will cover the following topics:

- Importance of testing
- Automation testing
- Testing in Django
- Testing Django models
- Testing Django views
- Django RequestFactory
- Test case classes in Django

Let's start our journey by first learning about the importance of testing.

14

장고 애플리케이션 테스트

앞서의 장에서는 데이터베이스 모델, 뷰 및 템플릿과 같은 다양한 구성 요소를 작성하여 장고에서 웹 애플리케이션을 구축하는 데 집중했습니다. 우리는 사용자가 프로필을 만들고 읽은 책에 대한 리뷰를 작성할 수 있는 대화형 애플리케이션을 제공하기 위해 모든 작업을 수행했습니다.

애플리케이션을 구축하고 실행하는 것 외에도 애플리케이션 코드가 우리가 기대하는 방식으로 작동하는지 확인하는 또 다른 중요한 측면이 있습니다. 이는 테스트라는 기술로 보장됩니다. 테스트에서는 웹 애플리케이션의 다양한 부분을 실행하고 실행된 구성 요소의 출력이 우리가 예상한 출력과 일치하는지 확인합니다. 출력이 일치하면 해당 구성 요소가 성공적으로 테스트되었다고 말하며, 출력이 일치하지 않으면 해당 구성 요소가 의도한 대로 작동하지 않았다고 말합니다.

이번 장에서는 다양한 섹션을 통해 테스트가 중요한 이유, 웹 애플리케이션을 테스트하는 다양한 방법, 그리고 우리가 구축하는 웹 애플리케이션이 견고하다는 것을 보장하는데 도움이 되는 강력한 테스트 전략을 구축하는 방법을 배울 것입니다.

이번 장에서는 다음 주제를 다룰 것입니다:

- 테스트의 중요성 자동
- 화 테스트장고에서의
- 테스트장고 모델 테
- 스트장고 뷰 테스트
- 장고 RequestFactory
-
- 장고의 테스트 케이스 클래스

테스트의 중요성에 대해 먼저 배우며 우리의 여정을 시작합니다.

Importance of testing

Making sure that an application works the way it was designed to work is an important aspect of development efforts because, otherwise, our users might keep on encountering weird behaviors that will usually drive away their engagement from the application.

The efforts we put into testing help us ensure that the different kinds of problems that we intend to solve are indeed being solved correctly. Imagine a case where a developer is building an online event scheduling platform. On this platform, users can schedule events on their calendars as per their local time zone. Now, what if, on this platform, users can schedule events as expected, but due to a bug, the events are scheduled in an incorrect time zone? It is issues like this that tend to drive many users away.

That's why a lot of enterprise companies spend a huge amount of money making sure that the applications they build undergo thorough testing. That way, they ensure that they do not release a buggy product or a product that is far from satisfying user requirements.

In brief, testing helps us achieve the following goals:

- Ensure that the components of the application work according to specifications
- Ensure interoperability on different infrastructure platforms, if an application can be deployed on a different operating system, such as Linux or Windows, and more
- Reduce the probability of introducing a bug while refactoring the application code

Now, a common assumption many people make about testing is that they have to test all the components manually as they are developed to make sure each component works according to its specifications, and repeat this exercise every time a change is made or a new component is added to the application. While this is true, this doesn't provide a complete picture of testing. Testing as a technique has grown very powerful with time, and as a developer, you can reduce a huge amount of testing effort by implementing **automated test cases**. So, what are these automated test cases? Or, in other words, what is **automation testing**? Let's find out.

Automation testing

Testing the whole application repeatedly when a single component is modified can turn out to be a challenging task, and even more so if that application consists of a large code base. The size of the code base could be due to the sheer number of features or the complexity of the problem it solves.

As we develop applications, it is important to make sure that the changes being made to these applications can be tested easily so that we can verify whether something is breaking. That's where the concept of automation testing comes in handy. The focus of automation testing is to

테스트의 중요성

idx_d77c8410application이 설계된 대로 작동하는지 확인하는 것은 개발 노력의 중요한 측면입니다. 그렇지 않으면 사용자가 이상한 동작을 계속 경험하게 되어 보통 애플리케이션에 대한 참여를 멀리하게 될 수 있기 때문입니다.

우리가 테스트에 기울이는 노력은 우리가 해결하려는 다양한 문제들이 실제로 올바르게 해결되고 있음을 보장하는 데 도움이 됩니다. 개발자가 온라인 이벤트 일정 관리 플랫폼을 구축하는 경우를 상상해 보십시오. 이 플랫폼에서 사용자는 자신의 현지 시간대에 따라 달력에 이벤트를 예약할 수 있습니다. 그런데 만약 이 플랫폼에서 사용자가 예상대로 이벤트를 예약할 수는 있지만, 버그로 인해 이벤트가 잘못된 시간대에 예약된다면 어떻게 될까요? 이러한 문제들이 많은 사용자를 떠나게 만드는 경향이 있습니다.

그래서 많은 대기업들이 자신들이 개발하는 애플리케이션이 철저한 테스트를 거치도록 막대한 비용을 지출합니다. 그렇게 함으로써 그들은 문제가 발생하지 않도록 보장합니다.

버그가 있는 제품이나 사용자 요구 사항을 충족하지 못하는 제품을 출시하는 것. 간단히 말해, 테스트는 다음과 같은 목표를 달성하는 데 도움이 됩니다:

- 애플리케이션의 구성 요소가 사양에 따라 작동하는지 확인하십시오. 애플리케이션이 Linux, Windows 등과 같은 다른 운영 체제에 배포될 수 있는 경우 다양한 인프라 플랫폼에서 상호 운용성을 보장하십시오. 애플리케이션 코드를 리팩토링하는 동안 버그가 발생할 확률을 줄이십시오.

이제, 많은 사람들이 테스트에 대해 일반적으로 하는 가정은 각 구성 요소가 사양에 따라 작동하는지 확인하기 위해 개발되는 모든 구성 요소를 수동으로 테스트해야 하며, 변경이 이루어지거나 애플리케이션에 새로운 구성 요소가 추가될 때마다 이 작업을 반복해야 한다는 것입니다. 이것은 사실이지만, 테스트의 전체적인 그림을 제공하지는 않습니다. 테스트는 시간이 지남에 따라 매우 강력한 기술로 발전했으며, 개발자로서 자동화된 테스트 케이스를 구현함으로써 막대한 테스트 노력을 줄일 수 있습니다. 그렇다면 이러한 자동화된 테스트 케이스는 무엇일까요? 또는 다른 말로 하면, 자동화 테스트란 무엇일까요? 알아보시다.

자동화 테스트

단일 구성 요소가 수정될 때마다 전체 애플리케이션을 반복적으로 테스트하는 것은 도전적인 작업이 될 수 있으며, 특히 그 애플리케이션이 큰 코드베이스로 구성되어 있다면 더욱 그렇습니다. 코드베이스의 크기는 기능의 수나 해결하는 문제의 복잡성 때문일 수 있습니다.

애플리케이션을 개발하면서 이러한 애플리케이션에 대한 변경 사항이 쉽게 테스트될 수 있도록 하는 것이 중요합니다. 그래야 무언가가 깨지고 있는지 확인할 수 있습니다. 자동화 테스트의 개념이 유용하게 작용하는 지점입니다. 자동화 테스트의 초점은

write tests as code so that the individual components of an application can be tested in isolation, as well as their interaction with each other.

While considering this, it becomes important for us to define the different kinds of automation tests that can be done for applications.

Automation testing can be broadly categorized into five different types:

- **Unit testing:** In this type of testing, the individual isolated units of code are tested. For example, a unit test can target a single method or a single isolated API. This kind of testing is performed to make sure the basic units of the application work according to their specification.
- **Integration testing:** In this type of testing, the individual isolated units of code are merged to form a logical grouping. Once this grouping is formed, testing is performed on this logical group to make sure that the group works in the way it is expected.
- **Functional testing:** In this kind of testing, the overall functionality of the different components of the application is tested. This may include different APIs, user interfaces, and more.
- **Smoke testing:** In this kind of testing, the stability of the deployed application is tested to make sure that the application continues to remain functional as the users interact with it, without causing a crash.
- **Regression testing:** This kind of testing is done to make sure that the changes being made to the application do not degrade the previously built functionality of the application.

As we can see, testing is a big domain that takes time to master; entire books have been written on this topic. To make sure we highlight the important aspects of testing, we are going to focus on unit testing in this chapter.

Testing in Django

Django is a feature-packed framework that aims to make web application development rapid. It is expected that it should also provide a full-featured way of testing the application.

Django provides a well-integrated module that allows application developers to write unit tests for their applications. This module is based on the Python `unittest` library, which ships with most of the Python distributions pre-packaged.

So, let's get started by understanding how we can write basic test cases in Django and how to leverage the framework-provided modules to test our application code.

코드로 테스트를 작성하여 애플리케이션의 개별 구성 요소를 독립적으로 테스트할 수 있을 뿐만 아니라 서로의 상호 작용도 테스트할 수 있습니다.

이를 고려할 때, 애플리케이션에 대해 수행할 수 있는 다양한 종류의 자동화 테스트를 정의하는 것이 중요해집니다.

자동화 테스트는 크게 다섯 가지 유형으로 분류할 수 있습니다:

- 단위 테스트: 이 유형의 테스트에서는 개별적으로 격리된 코드 단위가 테스트됩니다. 예를 들어, `unitidx_c532503a` 테스트는 단일 메서드 또는 `idx_da65155dsingle` 격리된 API를 대상으로 할 수 있습니다. 이러한 종류의 테스트는 애플리케이션의 기본 단위가 사양에 따라 작동하는지 확인하기 위해 수행됩니다.
- 통합 테스트: 이 `idx_3cf949d5` 유형의 테스트에서는 개별적으로 격리된 코드 단위가 논리적 그룹을 형성하기 위해 병합됩니다. 이 그룹이 형성되면, 이 논리적 그룹이 예상대로 작동하는지 확인하기 위해 테스트 `idx_cf93ef63`가 수행됩니다. 기능 테스트: 이 유형의 테스트에서는 `idx_827266ae` 애플리케이션의 다양한 구성 요소의 전반적인 기능이 테스트됩니다. 여기에는 다양한 `idx_2c4ec3e2` API, 사용자 인터페이스 등이 포함될 수 있습니다.
- 스모크 테스트: 이 유형의 테스트에서는 배포된 애플리케이션의 안정성이 테스트되어 사용자가 `idx_66ac106d`와 상호작용할 때 애플리케이션이 계속 기능을 유지하는지 확인합니다. 이 과정에서 충돌이 발생하지 않아야 합니다.
- 회귀 테스트: 이 유형의 테스트는 애플리케이션에 대한 변경 사항이 이전에 구현된 `idx_3105ed8d` 기능을 저하시키지 않도록 하기 위해 수행됩니다.

우리가 볼 수 있듯이, 테스트는 마스터하는 데 시간이 걸리는 큰 영역입니다; 이 주제에 대한 전체 책이 작성되었습니다. 테스트의 중요한 측면을 강조하기 위해, 우리는 이 장에서 단위 테스트에 집중할 것입니다.

장고에서의 테스트

장고는 웹 애플리케이션 개발을 신속하게 만들기 위해 설계된 기능이 풍부한 `idx_79085a93` 프레임워크입니다. 애플리케이션을 테스트하는 완전한 기능을 제공할 것으로 기대됩니다.

장고는 애플리케이션 개발자가 애플리케이션에 대한 단위 테스트를 작성할 수 있도록 하는 잘 통합된 모듈을 제공합니다. 이 모듈은 대부분의 Python 배포판에 미리 패키징되어 제공되는 Python `unittest` 라이브러리를 기반으로 합니다.

그렇다면, 장고에서 기본 테스트 케이스를 작성하는 방법과 프레임워크에서 제공하는 모듈을 활용하여 애플리케이션 코드를 테스트하는 방법을 이해하는 것부터 시작해 보겠습니다.

Implementing test cases

While working on implementing mechanisms for testing your code, the first thing you need to understand is how this implementation can be logically grouped so that modules that are closely related to each other are tested in one logical unit.

This can be simplified by implementing a **test case**. A test case is nothing more than a logical unit that groups together tests that are related to logically similar units, so that all the common logic to initialize the environment for the test cases can be combined in the same place, hence avoiding duplication of work while implementing application testing code.

Unit testing in Django

Now that we understand tests, let's take a look at how we can do unit testing inside Django. In the context of Django, a unit test consists of two major parts:

- A `TestCase` class, which wraps the different test cases that are grouped for a given module
- An actual test case that needs to be executed to test the flow of a particular component

The class implementing a unit test should inherit from the `TestCase` class provided by Django's test module. By default, Django provides a `tests.py` file in every application directory, which can be used to store the test cases for the application module.

Once these unit tests have been written, they can be executed easily by running them directly using the provided test command in `manage.py`, as follows:

```
python manage.py test
```

Now, let's look at how assertions can help us build our unit tests and check for the expected behavior.

Utilizing assertions

An important part of writing tests is validating whether the test passed or failed. Generally, to implement such decisions in a testing environment, we utilize something known as **assertions**.

Assertions are a common concept in software testing. They take in two operands and validate if the value of the operand on the **left-hand side (LHS)** matches the value of the operand on the **right-hand side (RHS)**. If the value of the LHS matches the value of the RHS, an assertion is considered to be successful, whereas if the values differ, the assertion is considered to have failed.

테스트 케이스 구현

코드 테스트를 위한 idx_8934d131 메커니즘을 구현하는 작업을 하면서, 이해해야 할 첫 번째 사항은 이 구현이 어떻게 논리적으로 그룹화될 수 있는지입니다. 서로 밀접하게 관련된 모듈이 하나의 논리적 단위로 테스트될 수 있도록 말이죠.

테스트 케이스를 구현함으로써 이를 단순화할 수 있습니다. 테스트 케이스 idx_c9a0e11b는 논리적으로 유사한 단위와 관련된 테스트를 함께 그룹화하는 논리적 단위에 불과하므로, 테스트 케이스를 위한 환경을 초기화하는 모든 공통 로직을 같은 장소에 결합할 수 있어, 애플리케이션 테스트 코드를 구현하는 동안 작업의 중복을 피할 수 있습니다.

장고의 단위 테스트

이제 테스트를 이해했으니, 장고 내에서 단위 테스트를 수행하는 방법을 살펴보겠습니다. 장고의 맥락에서 단위 테스트는 두 가지 주요 부분으로 구성됩니다:

- 주어진 모듈에 대해 그룹화된 다양한 테스트 케이스를 감싸는 TestCase 클래스

- 특정 구성 요소의 흐름을 테스트하기 위해 실행해야 하는 실제 테스트 케이스 단위 테스트를 구현하는 클래스는 장고의 테스트 모듈에서 제공하는 TestCase 클래스를 상속해야 합니다. 기본적으로 장고는 모든 애플리케이션 디렉토리에 tests.py 파일을 제공하며, 이를 사용하여 애플리케이션 모듈의 테스트 케이스를 저장할 수 있습니다.

이 단위 테스트가 작성되면, 다음과 같이 manage.py에 제공된 테스트 명령을 직접 실행하여 쉽게 실행할 수 있습니다:

```
python manage.py test
```

이제 단언이 어떻게 단위 테스트를 구축하고 예상되는 동작을 확인하는 데 도움이 되는지 살펴보겠습니다.

단언 활용하기

idx_13ed1c73 테스트를 작성하는 중요한 부분은 테스트가 통과했는지 실패했는지를 검증하는 것입니다. 일반적으로 테스트 환경에서 이러한 결정을 구현하기 위해 우리는 idx_2bd0da74라는 것을 활용합니다. 단언이라고 알려진 것입니다.

단언은 소프트웨어 테스트에서 일반적인 개념입니다. 단언은 두 개의 피연산자를 받아들이고 왼쪽 피연산자(LHS)의 값이 오른쪽 피연산자(RHS)의 값과 일치하는지 검증합니다. LHS의 값이 RHS의 값과 일치하면 단언이 성공한 것으로 간주되며, 값이 다르면 단언이 실패한 것으로 간주됩니다.

An assertion that evaluates to `False` essentially causes a test case to be evaluated as a failure, which is then reported to the user.

Assertions in Python are quite easy to implement, and they use a simple keyword called `assert`. For example, the following code snippet shows a very simple assertion:

```
assert 1 == 1
```

The preceding assertion takes in a single expression, which evaluates to `True`. If this assertion were part of a test case, the test would have succeeded.

Now, let's see how we can implement test cases using the Python `unittest` library. Doing so is quite easy and can be accomplished with some easy-to-follow steps:

1. Import the `unittest` module, which allows us to build the test cases:

```
import unittest
```

2. Once the module has been imported, we can create a class whose name starts with `Test`, and which inherits from the `TestCase` class provided by the `unittest` module:

```
class TestMyModule(unittest.TestCase):  
    def test_method_a(self):  
        self.assertTrue(<expression>)
```

Only if the `TestMyModule` class inherits the `TestCase` class will Django be able to run it automatically with full integration with the framework. Once the class has been defined, we can implement a new method inside the class named `test_method_a()`, which validates an assertion. And that's it.

An important part to note here is the naming scheme for the test cases and test functions. The test cases being implemented should be prefixed with the name `Test` so that the test execution modules can detect them as valid test cases and execute them. The same rule applies to naming testing methods.

3. Once the test case has been written, it can be simply executed by running the following command:

```
python manage.py test
```

With our basic understanding of implementing test cases clarified, let's write a very simple unit test to get a feel of how the unit testing framework behaves inside Django.

False로 평가되는 단언은 본질적으로 테스트 케이스가 실패로 평가되도록 하며, 이는 사용자에게 보고됩니다.

파이썬의 단언은 구현하기 매우 쉽고, `assert`라는 간단한 키워드를 사용합니다. 예를 들어, 다음 코드 조각은 매우 간단한 단언을 보여줍니다:

```
assert 1 == 1
```

앞선 단언은 True로 평가되는 단일 표현식을 입력받습니다. 이 단언이 테스트 케이스의 일부였다면, 테스트는 성공했을 것입니다.

이제 파이썬 `unittest` 라이브러리를 사용하여 테스트 케이스를 구현하는 방법을 살펴보겠습니다. 그렇게 하는 것은 매우 쉽고 몇 가지 따라하기 쉬운 단계로 수행할 수 있습니다:

1. 테스트 케이스를 구축할 수 있는 `unittest` 모듈을 가져옵니다:

```
import unittest
```

2. 모듈이 임포트되면, `Test`로 시작하는 이름을 가진 클래스를 만들 수 있으며, 이는 `unittest` 모듈에서 제공하는 `TestCase` 클래스를 상속합니다:

```
class TestMyModule(unittest.TestCase):
    def test_method_a(self):
        self.assertTrue(<expression>)
```

`TestMyModule` 클래스가 `TestCase` 클래스를 상속해야만 Django가 이를 자동으로 실행할 수 있으며, 프레임워크와 완전하게 통합됩니다. 클래스가 정의되면, `assertion`을 검증하는 `test_method_a()`라는 새로운 메서드를 클래스 안에 구현할 수 있습니다. 그게 전부입니다.

여기서 주목해야 할 중요한 부분은 테스트 케이스와 테스트 함수의 명명 규칙입니다. 구현되는 테스트 케이스는 `Test`라는 이름으로 접두사가 붙어야 하며, 그래야 테스트 실행 모듈이 이를 유효한 테스트 케이스로 감지하고 실행할 수 있습니다. 같은 규칙이 테스트 메서드의 명명에도 적용됩니다.

3. 테스트 케이스가 작성되면, 다음 명령어를 실행하여 간단히 실행할 수 있습니다:

```
python manage.py test
```

테스트 케이스 구현에 대한 기본 이해가 명확해졌으니, Django 내부에서 단위 테스트 프레임워크가 어떻게 작동하는지 느끼기 위해 매우 간단한 단위 테스트를 작성해 봅시다.

Exercise 14.01 – Writing a simple unit test

In this exercise, we will write a simple unit test to understand how the Django unit testing framework works and use this knowledge to implement our first test case that validates a simple expression:

1. To get started, open the `tests.py` file under the `reviews` application of the *Bookr* project. By default, this file will contain only a single line that imports Django's `TestCase` class from the `test` module:

```
from django.test import TestCase
```

2. Add the following lines of code to the `tests.py` file you just opened:

```
class TestSimpleComponent(TestCase):  
    def test_basic_sum(self):  
        assert 1+1 == 2
```

Here, we created a new class named `TestSimpleComponent`, which inherits from the `TestCase` class provided by Django's `test` module. The `assert` statement will compare the expression on the LHS (`1+1`) with the one on the RHS (`2`).

3. Once you have written the test case, navigate back to the project folder and run the following command:

```
python manage.py test
```

The following output should be generated:

```
% ./manage.py test  
Creating test database for alias 'default'...  
System check identified no issues (0 silenced).  
.  
-----  
Ran 1 test in 0.001s  
  
OK  
Destroying test database for alias 'default'...
```

The preceding output signifies that Django's test runner executed 1 test case, which successfully passed the evaluation.

연습 14.01 – 간단한 단위 테스트 작성하기

이 연습에서는 Django 단위 테스트 프레임워크가 어떻게 작동하는지 이해하기 위해 간단한 단위 테스트를 작성하고 이 지식을 사용하여 간단한 표현식을 검증하는 첫 번째 테스트 케이스를 구현할 것입니다:

1. 시작하려면 Bookr 프로젝트의 reviews 애플리케이션 아래에 있는 tests.py 파일을 엽니다. 기본적으로 이 파일은 테스트 모듈에서 Django의 TestCase 클래스를 가져오는 단일 행만 포함합니다:

```
from django.test import TestCase
```

2. 방금 연 tests.py 파일에 다음 코드 줄을 추가합니다:

```
class TestSimpleComponent(TestCase):
    def test_basic_sum(self):
        assert 1+1 == 2
```

여기에서 우리는 Django의 테스트 모듈에서 제공하는 TestCase 클래스를 상속 받는 TestSimpleComponent라는 새 클래스를 만들었습니다. assert 문은 LHS (1+1)의 표현식을 RHS(2)와 비교합니다.

3. 테스트 케이스를 작성한 후, 프로젝트 폴더로 돌아가서 다음 명령어를 실행하세요:

```
python manage.py test
```

다음 출력이 생성되어야 합니다:

```
% ./manage.py test
'default'에 대한 테스트 데이터베이스를 생성하는
중... 시스템 검사가 문제를 식별하지 못했습니다 (0
무시됨)..

-----
1개의 테스트를 0.001초에 실행했습니다

OK
'default'의 테스트 데이터베이스를 파괴하는 중...
```

앞선 출력은 Django의 테스트 러너가 1개의 테스트 케이스를 실행했으며, 성공적으로 평가를 통과했음을 의미합니다.

4. With the test case confirmed to be working and passing, we will now try to add another assertion at the end of the `test_basic_sum()` method, as shown in the following code snippet:

```
assert 1+1 == 3
```

5. With the `assert` statement added in *Step 4*, execute the test cases by running the following command from the project folder:

```
python manage.py test
```

At this point, you will notice Django reporting that the execution of the test cases has failed.

With this, you understand how test cases can be written in Django and how the assertions can be used to validate whether the output generated from your method calls under test is correct or not.

Now that we have basic knowledge of assertions, let's dive deep into the different types of assertions that are present and their usage.

Types of assertions

In *Exercise 14.01 – Writing a simple unit test*, we briefly encountered assertions when we came across the following `assert` statement:

```
assert 1+1 == 2
```

These assertion statements are simple and use the Python `assert` keyword. A few different types of assertions are possible that can be tested inside a unit test while using the `unittest` library. Let's look at those:

- `assertIsNone`: This assertion is used to check whether an expression evaluates to `None` or not. For example, this type of assertion can be used in cases where a query to a database returns `None` because no records were found for the specified filtering criteria.
- `assertIsInstance`: This assertion is used to validate if a provided object evaluates to an instance of the provided type. For example, this assertion can be used to validate if the value returned by a method indeed is of a specific type, such as `List`, `Dict`, `Tuple`, and others.

4. 테스트 케이스 idx_988b5300이 작동하고 통과하는 것이 확인되었으므로, 이제 `test_basic_sum()` 메서드의 끝에 또 다른 단언을 추가해 보겠습니다. 다음 코드 조각과 같이:

```
assert 1+1 == 3
```

5. 4단계에서 추가된 `assert` 문으로, 프로젝트 폴더에서 다음 명령을 실행하여 테스트 케이스를 실행합니다:

```
python manage.py test
```

이 시점에서, 당신은 idx_f917fa96 장고가 테스트 케이스 실행이 실패했다고 보고하는 것을 알게 될 것입니다.

이로써, 당신은 장고에서 테스트 케이스를 작성하는 방법과 단언을 사용하여 테스트 중인 메서드 호출에서 생성된 출력이 올바른지 여부를 검증하는 방법을 이해하게 됩니다.

이제 우리가 어설션에 대한 기본 지식을 갖추었으니, 다양한 유형의 어설션과 그 사용법에 대해 깊이 파고들어 보겠습니다.

어설션의 유형

연습 14.01 - 간단한 단위 테스트 작성에서, 우리는 다음의 `assert` 문을 접하면서 어설션을 간략하게 접했습니다:

```
assert 1+1 == 2
```

이 어설션 문은 간단하며 Python `assert` 키워드를 사용합니다. 단위 테스트에서 `unittest` 라이브러리를 사용할 때 테스트할 수 있는 몇 가지 다른 유형의 어설션이 가능합니다. 그것들을 살펴보겠습니다:

- `assertIsNone`: 이 어설션은 표현식이 `None`으로 평가되는지 여부를 확인하는 데 사용됩니다. 예를 들어, 이 유형의 어설션은 데이터베이스에 대한 쿼리가 지정된 필터링 기준에 대해 레코드가 발견되지 않아 `None`을 반환하는 경우에 사용할 수 있습니다.
- `assertIsInstance`: 이 어설션은 제공된 객체가 제공된 유형의 인스턴스로 평가되는지 확인하는 데 사용됩니다. 예를 들어, 이 어설션은 메서드가 반환하는 값이 실제로 `List`, `Dict`, `Tuple` 등과 같은 특정 유형인지 확인하는 데 사용할 수 있습니다.

- `assertEqual`: This is a very basic function that takes in two arguments and checks whether the arguments provided to it are equal in value or not. This can be useful when you plan to compare the values of data structures that do not guarantee ordering.
- `assertRaises`: This method is used to validate whether the name of the method provided to it, when called, raises a specified exception or not. This is helpful when we are writing test cases where a code path that raises an exception needs to be tested. As an example, this kind of assertion can be useful when we want to make sure an exception is raised by a method performing a database query if the database connection is not yet established.

These were just a small set of useful assertions that we can make in our test cases. The `unittest` module, on top of which Django's testing library is built, provides a lot more assertions that can be tested for.

Ever wondered what to do when our unit tests require some lists to be pre-populated or databases to be pre-initialized? The next section will cover how to pre-test this setup and how it can help us avoid introducing repetitive code that must be executed before every test case.

Performing pre-test setup and cleanup after every test case run

Sometimes, while writing test cases, we may need to perform some repetitive tasks, such as setting up some variables that will be required for the test. Once the test is over, we will want to clean up all the changes we've made to the test variables so that any new test starts with a fresh instance.

Luckily, the `unittest` library provides us with a useful way to automate our repetitive efforts of setting up the environment before every test case runs and cleaning it up after the test case is finished. This can be achieved using the two methods that we can implement in `TestCase`.

The first is the `setUp()` method. This method is called before the execution of every test method inside the `TestCase` class. It implements the code required to set up the test case's environment before the test executes. This method can be a good place to set up any local database instance or test variables that may be required for the test cases.

Note

The `setUp()` method is only valid for test cases written inside the `TestCase` class.

- `assertEqual`: 이것은 두 개의 인수를 받아들이고 제공된 인수가 값이 같은지 여부를 확인하는 매우 기본적인 함수입니다. 이는 순서를 보장하지 않는 데이터 구조의 값을 비교할 계획일 때 유용할 수 있습니다.
- `assertRaises`: 이 메서드는 제공된 메서드의 이름이 호출될 때 지정된 예외를 발생시키는지 여부를 검증하는 데 사용됩니다. 이는 예외를 발생시키는 코드 경로를 테스트해야 하는 테스트 케이스를 작성할 때 유용합니다. 예를 들어, 데이터베이스 연결이 아직 설정되지 않은 경우 데이터베이스 쿼리를 수행하는 메서드가 예외를 발생시키는지 확인하고 싶을 때 이러한 종류의 주장이 유용할 수 있습니다.

이들은 테스트 케이스에서 만들 수 있는 유용한 주장의 작은 집합에 불과합니다.

Django의 테스트 라이브러리가 구축된 `unittest` 모듈은 테스트할 수 있는 훨씬 더 많은 주장을 제공합니다.

단위 테스트가 일부 목록을 미리 채우거나 데이터베이스를 미리 초기화해야 할 때 어떻게 해야 할지 궁금한 적이 있습니까? 다음 섹션에서는 이 설정을 사전 테스트하는 방법과 이를 통해 모든 테스트 케이스 전에 실행해야 하는 반복적인 코드를 도입하는 것을 피할 수 있는 방법을 다룰 것입니다.

각 테스트 케이스 실행 후 사전 테스트 설정 및 정리 수행

때때로, 테스트 케이스를 작성하는 동안 테스트에 필요한 일부 변수를 설정하는 것과 같은 반복적인 작업을 수행해야 할 수 있습니다. 테스트가 끝나면 테스트 변수를 변경한 모든 내용을 정리하여 새로운 테스트가 새 인스턴스에서 시작되도록 하고 싶습니다.

다행히도, `unittest` 라이브러리는 각 테스트 케이스가 실행되기 전에 환경을 설정하고 테스트 케이스가 끝난 후 정리하는 반복적인 노력을 자동화하는 유용한 방법을 제공합니다. 이는 `TestCase`에서 구현할 수 있는 두 가지 메서드를 사용하여 달성할 수 있습니다.

첫 번째는 `setUp()` 메서드입니다. 이 메서드는 `TestCase` 클래스 내의 모든 테스트 메서드 실행 전에 호출됩니다. 이는 테스트가 실행되기 전에 테스트 케이스의 환경을 설정하는 데 필요한 코드를 구현합니다. 이 메서드는 테스트 케이스에 필요할 수 있는 로컬 데이터베이스 인스턴스나 테스트 변수를 설정하는 좋은 장소가 될 수 있습니다.

메모

`setUp()` 메서드는 `TestCase` 클래스 내에 작성된 테스트 케이스에만 유효합니다.

For example, the following example illustrates a simple definition of how the `setUp()` method is used inside a `TestCase` class:

```
class MyTestCase(unittest.TestCase):
    def setUp(self):
        # Do some initialization work
    def test_method_a(self):
        # code for testing method A
    def test_method_b(self):
        # code for testing method B
```

In the preceding example, when we try to execute the test cases, the `setUp()` method we defined here will be called every time before a test method executes. In other words, the `setUp()` method will be called before the execution of the `test_method_a()` call, and then it will be called again before the `test_method_b()` call is called.

The second is the `tearDown()` method. This method is called once the test function finishes execution and cleans up the variables and their values once the test case's execution has finished. This method is executed regardless of whether the test case evaluates to `True` or `False`. An example of using the `tearDown()` method is shown here:

```
class MyTestCase(unittest.TestCase):
    def setUp(self):
        # Do some initialization work
    def test_method_a(self):
        # code for testing method A
    def test_method_b(self):
        # code for testing method B
    def tearDown(self):
        # perform cleanup
```

In the preceding example, the `tearDown()` method will be called every time a test method finishes execution – that is, once `test_method_a()` finishes executing and again once `test_method_b()` finishes executing.

Now that we are aware of the different components of writing test cases. Let's look at how we can test the different aspects of a Django application using the provided test framework.

예를 들어, 다음 예시는 TestCase 클래스 내에서 setUp() 메서드가 어떻게 사용되는지를 간단히 정의한 것입니다:

```
class MyTestCase(unittest.TestCase):
    def setUp(self):
        # 초기화 작업 수행
    def test_method_a(self):
        # 메서드 A 테스트를 위한 코드
    def test_method_b(self):
        # 메서드 B 테스트를 위한 코드
```

앞서의 예에서, 테스트 케이스를 실행하려고 할 때, 여기서 정의한 setUp() 메서드는 테스트 메서드가 실행되기 전에 매번 호출됩니다. 다시 말해, setUp() 메서드는 test_method_a() 호출이 실행되기 전에 호출되고, 그 다음에 test_method_b() 호출이 호출되기 전에 다시 호출됩니다.

두 번째는 tearDown() 메서드입니다. 이 메서드는 테스트 함수의 실행이 끝난 후 호출되며, 테스트 케이스의 실행이 끝난 후 변수와 그 값들을 정리합니다. 이 메서드는 테스트 케이스가 True 또는 False로 평가되든 관계없이 실행됩니다. tearDown() 메서드를 사용하는 예는 다음과 같습니다:

```
class MyTestCase(unittest.TestCase):
    def setUp(self):
        # 초기화 작업 수행
    def test_method_a(self):
        # 메서드 A 테스트를 위한 코드
    def test_method_b(self):
        # 메서드 B 테스트를 위한 코드
    def tearDown(self):
        # 정리 수행
```

앞서의 예에서 tearDown() 메서드는 테스트 메서드가 실행을 마칠 때마다 호출됩니다. 즉, test_method_a()가 실행을 마친 후와 test_method_b()가 실행을 마친 후에 다시 호출됩니다.

이제 테스트 케이스를 작성하는 다양한 구성 요소를 알고 있으니, 제공된 테스트 프레임워크를 사용하여 Django 애플리케이션의 다양한 측면을 테스트하는 방법을 살펴보겠습니다.

Testing Django models

Models in Django are object-based representations of how data will be stored inside the database of an application. They provide methods that can help us to validate the data input provided for a given record, as well as to perform any processing on the data before it is inserted into the database.

It is as easy to test models in Django as it is to create them. Now, let's look at how Django models can be tested using the Django Test framework.

Exercise 14.02 – Testing Django models

In this exercise, we will create a new Django model and write test cases for it. The test case will validate whether your model can correctly insert and retrieve the data from the database.

These kinds of test cases that work on database models can turn out to be useful in cases where a team of developers is collaborating on a large project, and the same database model may be modified by multiple developers over time. Implementing test cases for the database models allows developers to pre-emptively identify potentially breaking changes that they may inadvertently introduce as part of their work. Let's get started with the steps:

1. Create a new application that you will use for the exercises in this chapter. For this, run the following command, which will set up a new application for your use case:

```
python manage.py startapp bookr_test
```

2. To make sure the `bookr_test` application behaves the same way as any other application in the Django project, add this application to the `INSTALLED_APPS` section of the `bookr` project. To do this, open the `settings.py` file in your `bookr` project and append the following code to the `INSTALLED_APPS` list:

```
INSTALLED_APPS = [  
    ... ,  
    ... ,  
    "bookr_test",  
]
```

3. Now, with the application setup complete, create a new database model that you will use for testing purposes. For this exercise, we are going to create a new model named `Publisher` that will store the details about the book publisher in our database. To

장고 모델 테스트

장고의 모델은 애플리케이션의 데이터베이스에 데이터가 어떻게 저장될지를 나타내는 객체 기반 idx_3101a102 표현입니다. 이들은 주어진 레코드에 대해 제공된 데이터 입력을 검증하는 데 도움이 되는 메서드와 데이터가 데이터베이스에 삽입되기 전에 처리할 수 있는 메서드를 제공합니다.

장고에서 모델을 테스트하는 것은 모델을 만드는 것만큼 쉽습니다. 이제 장고 모델을 장고 테스트 프레임워크를 사용하여 테스트하는 방법을 살펴보겠습니다.

연습 14.02 – 장고 모델 테스트

이 연습에서는 새로운 장고 모델을 만들고 이에 대한 테스트 케이스를 작성할 것입니다. 테스트 케이스는 귀하의 모델이 데이터베이스에서 데이터를 올바르게 삽입하고 검색할 수 있는지를 검증합니다. 데이터베이스 모델에서 작동하는 이러한 종류의 테스트 케이스는 개발자 팀이 대규모 프로젝트에서 협력할 때 유용할 수 있으며, 동일한 데이터베이스 모델이 시간이 지남에 따라 여러 개발자에 의해 수정될 수 있습니다. 데이터베이스 모델에 대한 테스트 케이스를 구현하면 개발자는 작업의 일환으로 우연히 도입할 수 있는 잠재적인 파손 변경 사항을 사전에 식별할 수 있습니다. 단계별로 시작해 보겠습니다:

1. 이 장의 연습에 사용할 새로운 애플리케이션을 만듭니다. 이를 위해 다음 명령을 실행하여 귀하의 사용 사례에 맞는 새로운 애플리케이션을 설정합니다:

```
python manage.py startapp bookr_test
```

2. Django 프로젝트의 다른 애플리케이션과 동일하게 bookr_test 애플리케이션이 작동하도록 하려면 이 애플리케이션을 bookr 프로젝트의 INSTALLED_APPS 섹션에 추가하세요. 이를 위해 bookr 프로젝트의 settings.py 파일을 열고 INSTALLED_APPS 목록에 다음 코드를 추가하세요:

```
INSTALLED_APPS = [
    ...,
    ...,
    "bookr_test",
]
```

3. 이제 애플리케이션 설정이 완료되었으므로 테스트 목적으로 사용할 새 데이터베이스 모델을 만듭니다. 이 연습에서는 데이터베이스에 책 출판사에 대한 세부 정보를 저장할 Publisher라는 새 모델을 만들 것입니다. To

create the model, open the `models.py` file under the `bookr_test` directory and add the following code to it:

```
from django.db import models

class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(
        max_length=50,
        help_text="The name of the Publisher.")

    website = models.URLField(
        help_text="The Publisher's website.")
    email = models.EmailField(
        help_text="The Publisher's email address.")

    def __str__(self):
        return self.name
```

In the preceding code snippet, you created a new class named `Publisher` that inherits from the `Model` class of Django's `models` module, defining the class as a Django model that will be used to store data about the publisher:

```
class Publisher(models.Model)
```

Inside this model, we have added three fields, which will act as the properties of the model:

- `name`: The name of the publisher
- `website`: The website belonging to the publisher
- `email`: The email address of the publisher

Once we did this, we created a class method called `__str__()` that defines what the string representation of the `Model` class will look like.

4. Now, with the model created, you need to migrate this model before you can run a test on it. To do this, run the following commands:

```
python manage.py makemigrations
python manage.py migrate
```

모델을 생성하려면 bookr_test 디렉토리 아래의 models.py 파일을 열고 다음 코드를 추가하세요:

```
from django.db import models

class Publisher(models.Model):
    """책을 출판하는 회사입니다."""
    name = models.CharField(
        max_length=50,
        help_text="출판사의 이름입니다.")

    website = models.URLField(
        help_text="출판사의 웹사이트입니다.")
    email = models.EmailField(
        help_text="출판사의 이메일 주소입니다.")

    def __str__(self):
        return self.name
```

앞서의 코드 조각에서는 Publisher라는 새 클래스를 생성하였으며, 이는 Django의 models 모듈의 Model 클래스를 상속받아 출판사에 대한 데이터를 저장하는 Django 모델로 정의됩니다:

```
class Publisher(models.Model)
```

이 모델 안에는 세 개의 필드를 추가하였으며, 이는 모델의 속성으로 작용합니다:

- name: 출판사의 이름
- 웹사이트: 발행자의 웹사이트 이메일: 발행자
- 의 이메일 주소

이 작업을 수행한 후, 우리는 Model 클래스의 문자열 표현이 어떻게 보일지를 정의하는 __str__()라는 클래스 메서드를 만들었습니다.

4. 이제 모델이 생성되었으므로, 이를 테스트하기 전에 이 모델을 마이그레이션해야 합니다. 이를 위해 다음 명령어를 실행하세요:

```
python manage.py makemigrations
python manage.py migrate
```

5. With the model now set up, write the test case with which you are going to test the model you created in *Step 3*. For this, open the `tests.py` file under the `bookr_test` directory and add the following code to it:

```
from django.test import TestCase

from .models import Publisher

class TestPublisherModel(TestCase):
    """Test the publisher model."""
    def setUp(self):
        self.p = Publisher(
            name='Packt',
            website='www.packt.com',
            email='contact@packt.com'
        )

    def test_create_publisher(self):
        self.assertIsInstance(self.p, Publisher)

    def test_str_representation(self):
        self.assertEqual(str(self.p), "Packt")
```

In the preceding code snippet, there are a couple of things worth exploring.

At the start, after importing the `TestCase` class from the Django test module, you imported the `Publisher` model from the `bookr_test` directory, which is going to be used for testing.

Once the required libraries have been imported, we must create a new class named `TestPublisherModel` that inherits the `TestCase` class and is used for grouping the unit tests related to the `Publisher` model:

```
class TestPublisherModel(TestCase):
```

Inside this class, we defined a couple of methods. First, we defined a new method named `setUp()` and added the `Model` object creation code to it so that the `Model` object is created every time a new test method is executed inside this test case. This `Model`

5. 모델이 이제 설정되었으므로, 3단계에서 생성한 `themodel youidx_dbd437bb`를 테스트할 테스트 케이스를 작성하십시오. 이를 위해 `bookr_test` 디렉토리 아래의 `tests.py` 파일을 열고 다음 코드를 추가하십시오:

```
from django.test import TestCase

from .models import Publisher

class TestPublisherModel(TestCase):
    """퍼블리셔 모델을 테스트합니다."""
    def setUp(self):
        self.p = Publisher(
            name='Packt',
            website='www.packt.com',
            email='contact@packt.com'
        )

    def test_create_publisher(self):
        self.assertIsInstance(self.p, Publisher)

    def test_str_representation(self):
        self.assertEqual(str(self.p), "Packt")
```

앞서 언급한 코드 조각에서는 탐구할 가치가 있는 몇 가지 사항이 있습니다.

시작할 때, Django 테스트 모듈에서 `TestCase` 클래스를 가져온 후, `bookr_test` 디렉토리에서 `Publisher` 모델을 가져왔습니다. 이는 테스트에 사용될 것입니다.

필요한 라이브러리를 가져온 후, `TestCase` 클래스를 상속받는 새로운 클래스 `TestPublisherModel`을 생성해야 하며, 이는 `Publisher` 모델과 관련된 단위 테스트를 그룹화하는 데 사용됩니다:

```
class TestPublisherModel(TestCase):
```

이 클래스 안에서 우리는 `idx_b2df8758a`라는 몇 가지 메서드를 정의했습니다. 먼저, `setUp()`이라는 새로운 메서드를 정의하고, 이 메서드에 모델 객체 생성 코드를 추가하여 이 테스트 케이스 내에서 새로운 테스트 메서드가 실행될 때마다 모델 객체가 생성되도록 했습니다. 이 모델

object is stored as a class member so that it can be accessed inside other methods without a problem:

```
def setUp(self):
    self.p = Publisher(
        name='Packt',
        website='www.packt.com',
        email='contact@packt.com'
    )
```

The first test case validates whether the `Model` object for the `Publisher` model was created successfully or not. To do this, we created a new method named `test_create_publisher()`, inside which we check whether the model object created points to an object of the `Publisher` type. If this `Model` object was not created successfully, your test will fail:

```
def test_create_publisher(self):
    self.assertIsInstance(self.p, Publisher)
```

If you check carefully, you will see that we are using the `assertIsInstance()` method of the `unittest` library here to assert whether the `Model` object belongs to the `Publisher` type or not.

The next test validates whether the string representation of the model is the same as we expected it to be. From the code definition, the string representation of the `Publisher` model should output the name of the publisher. To test this, we created a new method named `test_str_representation()` and checked if the generated string representation of the model matches the one we are expecting:

```
def test_str_representation(self):
    self.assertEqual(str(self.p), "Packt")
```

To perform this validation, we used the `assertEqual` method of the `unittest` library, which validates whether the two values provided to it are equal or not.

6. With the test cases now in place, you can run them to check what happens. To run these test cases, run the following command:

```
python manage.py test
```

객체는 다른 메서드 내에서 문제 없이 접근할 수 있도록 클래스 멤버로 저장됩니다:

```
def setUp(self):
    self.p = Publisher
        (name='Packt',
         website='www.packt.com',
         email='contact@
         packt.com')
```

첫 번째 테스트 케이스는 Publisher 모델의 모델 객체가 성공적으로 생성되었는지 여부를 검증합니다. 이를 위해 `test_create_publisher()`라는 새로운 메서드를 만들었고, 그 안에서 생성된 모델 객체가 Publisher 유형의 객체를 가리키는 지 확인합니다. 이 모델 객체가 성공적으로 생성되지 않았다면, 테스트는 실패할 것입니다:

```
def test_create_publisher(self):
    self.assertIsInstance(self.p, Publisher)
```

주의 깊게 살펴보면, 여기서 unittest 라이브러리의 `assertIsInstance()` 메서드를 사용하여 모델 객체가 Publisher 유형에 속하는지 여부를 주장하고 있음을 알 수 있습니다.

다음 테스트는 모델의 문자열 표현이 우리가 예상한 것과 동일한지 검증합니다. 코드 정의에 따르면, Publisher 모델의 문자열 표현은 출판사의 이름을 출력해야 합니다. 이를 테스트하기 위해 `test_str_representation()`라는 새로운 메서드를 만들고, 생성된 모델의 문자열 표현이 우리가 기대하는 것과 일치하는지 확인했습니다:

```
def test_str_representation(self):
    self.assertEqual(str(self.p), "Packt")
```

이 검증을 수행하기 위해, 우리는 unittest 라이브러리의 `assertEqual` 메서드를 사용했습니다. 이 메서드는 제공된 두 값이 같은지 여부를 검증합니다.

- 이제 테스트 케이스가 준비되었으므로, 실행하여 어떤 일이 발생하는지 확인할 수 있습니다. 이러한 테스트 케이스를 실행하려면 다음 명령을 실행하세요:

```
python manage.py test
```

Once the command finishes executing, you will see an output that resembles the one shown here:

```
% python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
..
-----
Ran 2 tests in 0.002s

OK
Destroying test database for alias 'default'...
```

As you can see from the preceding output, the test cases are executed successfully, hence validating that the operations, such as creating a new `Publisher` object and its string representation when fetched, are being done correctly.

With this exercise, we saw how we can write test cases for our Django models easily and validate their functionality by creating objects, retrieving them, and representing them.

Also, notice the following important line in the output from *Exercise 14.02 – Testing Django models*:

```
"Destroying test database for alias 'default'..."
```

This happens because when there are test cases that require the data to be persisted inside a database, instead of using the production database, Django creates a new empty database for the test cases, which it uses to persist the value for the test case.

Now that we know how to test models in Django, in the next section, we will learn how to test views in Django.

Testing Django views

Views in Django control the rendering of the HTTP response for users based on the URL they visit in a web application. In this section, we will learn how to test views inside Django.

Imagine you are working on a website where a lot of **application programming interface (API)** endpoints are required. An interesting question to ask is, how will you be able to validate every new endpoint? If done manually, you will have to first deploy the application every time a new endpoint is added, then manually visit the endpoint in the browser to validate whether it is working fine or not. Such an approach may work out when the number of endpoints is small, but it may become extremely cumbersome if there are hundreds of endpoints.

명령이 실행을 마치면, 여기 표시된 것과 유사한 출력을 볼 수 있습니다:

```
% python manage.py test
'default' 별칭에 대한 테스트 데이터베이스 생성
중... 시스템 점검에서 문제가 발견되지 않았습니다
(0 무시됨)...
```

```
-----
2개의 테스트를 0.002초에 실행했습니다
```

```
확인
```

```
'default' 별칭에 대한 테스트 데이터베이스 파괴 중...
```

앞서 나온 출력에서 볼 수 있듯이, 테스트 케이스가 성공적으로 실행되었으며, 따라서 새로운 Publisher 객체를 생성하고 이를 가져올 때의 문자열 표현과 같은 작업이 올바르게 수행되고 있음을 검증합니다.

이 연습을 통해 우리는 Django 모델에 대한 테스트 케이스를 쉽게 작성하고 객체를 생성하고 검색하며 이를 표현함으로써 기능을 검증하는 방법을 보았습니다.

또한, 연습 14.02 – Django 모델 테스트의 출력에서 다음의 중요한 줄을 주목하세요:

```
"기본값에 대한 테스트 데이터베이스를 파괴하는 중..."
```

이것은 데이터가 데이터베이스에 지속되어야 하는 테스트 케이스가 있을 때 발생합니다. 프로덕션 데이터베이스 대신 Django는 테스트 케이스를 위해 새로운 빈 데이터베이스를 생성하고, 이를 사용하여 테스트 케이스의 값을 지속합니다.

이제 Django에서 모델을 테스트하는 방법을 알았으므로, 다음 섹션에서는 Django에서 뷰를 테스트하는 방법을 배울 것입니다.

Django 뷰 테스트

Django의 뷰는 사용자가 웹 애플리케이션에서 방문하는 URL에 따라 HTTP 응답의 렌더링을 제어합니다. 이 섹션에서는 Django 내에서 뷰를 테스트하는 방법을 배울 것입니다. 많은 애플리케이션 프로그래밍 인터페이스(API) 엔드포인트가 필요한

idx_2a87488f 웹사이트에서 작업하고 있다고 상상해 보세요. 흥미로운 질문은, 새로운 idx_d0c2421a 엔드포인트를 어떻게 검증할 수 있을까요? 수동으로 수행할 경우, 새로운 엔드포인트가 추가될 때마다 애플리케이션을 먼저 배포한 다음, 브라우저에서 엔드포인트를 수동으로 방문하여 제대로 작동하는지 확인해야 합니다. 이러한 접근 방식은 엔드포인트 수가 적을 때는 효과적일 수 있지만, 수백 개의 엔드포인트가 있을 경우 매우 번거로워질 수 있습니다.

Django provides a very comprehensive way of testing application views. This happens by using a testing client class provided by Django's test module. This class can be used to visit URLs mapped to the views and capture the output generated by visiting the URL endpoint. Then, we can use the captured output to test whether the URLs are generating a correct response or not. This client can be used by importing the `Client` class from the Django test module and then initializing it, as shown in the following snippet:

```
from django.test import Client

c = Client()
```

The `Client` object supports several methods that can be used to simulate the different HTTP calls a user can make – namely, GET, POST, PUT, DELETE, and others. An example of making such a request looks like this:

```
response = c.get('/welcome')
```

The response generated by the view is then captured by the client and exposed as a response object, which can then be queried to validate the output of the view.

With this knowledge, let's look at how we can write test cases for our Django views.

Exercise 14.03 – Writing unit tests for Django views

In this exercise, we will use the Django test client to write a test case for our Django view, which will be mapped to a specific URL. These test cases will help you validate whether your View function generates the correct response when visited using its mapped URL. Let's get started with the steps:

1. For this exercise, you are going to use the `bookr_test` application that was created in *Step 1 of Exercise 14.02 – Testing Django models*. To get started, open the `views.py` file under the `bookr_test` directory and add the following code to it:

```
from django.http import HttpResponse

def greeting_view(request):
    """Greet the user."""
    return HttpResponse("Hey there, welcome to Bookr!"
                       "Your one stop place to review books.")
```

장고는 애플리케이션 뷰를 테스트하는 매우 포괄적인 방법을 제공합니다. 이는 장고의 테스트 모듈에서 제공하는 테스트 클라이언트 클래스를 사용하여 이루어집니다. 이 클래스는 뷰에 매핑된 URL을 방문하고 URL 엔드포인트를 방문하여 생성된 출력을 캡처하는 데 사용될 수 있습니다. 그런 다음 캡처된 출력을 사용하여 URL이 올바른 응답을 생성하는지 여부를 테스트할 수 있습니다. 이 클라이언트는 장고 테스트 모듈에서 Client 클래스를 가져와 초기화하여 사용할 수 있으며, 다음 코드 조각과 같이 사용할 수 있습니다:

```
from django.test import Client

c = Client()
```

Client 객체는 사용자가 만들 수 있는 다양한 HTTP 호출을 시뮬레이션하는 데 사용할 수 있는 여러 idx_037c4598 메서드를 지원합니다. 즉, GET, POST, PUT, DELETE 및 기타 메서드입니다. 이러한 요청을 만드는 예는 다음과 같습니다:

```
response = c.get('/welcome')
```

뷰에 의해 생성된 응답은 클라이언트에 의해 캡처되고 응답 객체로 노출되며, 이를 쿼리하여 뷰의 출력을 검증할 수 있습니다.

이 지식을 바탕으로 장고 뷰에 대한 테스트 케이스를 작성하는 방법을 살펴보겠습니다.

연습 14.03 – 장고 뷰에 대한 단위 테스트 작성

이 연습에서는 idx_14365c4a Django 테스트 클라이언트를 사용하여 idx_ebd6ba1b Django 뷰에 대한 테스트 케이스를 작성할 것입니다. 이 뷰는 특정 URL에 매핑됩니다. 이러한 테스트 케이스는 매핑된 URL을 사용하여 방문할 때 View 함수가 올바른 응답을 생성하는지 확인하는 데 도움이 됩니다. 단계별로 시작해 봅시다:

1. 이 연습을 위해, Exercise 14.02 – Django 모델 테스트의 1단계에서 생성된 bookr_test 애플리케이션을 사용할 것입니다. 시작하려면 bookr_test 디렉토리 아래의 views.py 파일을 열고 다음 코드를 추가하세요:

```
from django.http import HttpResponse

def greeting_view(request):
    """사용자에게 인사합니다."""
    return HttpResponse("안녕하세요, Bookr에 오신 것을  
환영합니다!""책을 리뷰할 수 있는 원스톱 장소입  
니다.")
```

Here, you have created a simple Django view that will be used to greet the user with a welcome message whenever they visit an endpoint mapped to the provided view.

2. Once the view has been created, you need to map this view to a URL endpoint that can then be visited in a browser or a test client. To do this, add a `urls.py` file under the `bookr_test` directory and include the following code with the mapping in the `urlpatterns` list:

```
from django.urls import path
from . import views

urlpatterns = [
    path('test/greeting', views.greeting_view,
        name='greeting_view'),
]
```

In the preceding code snippet, we mapped `greeting_view` to the `'test/greeting'` endpoint for the application by setting the path in the `urlpatterns` list.

3. Once this path has been set up, we need to make sure that it is also identified by our project. For this, we need to add this entry to the `bookr` project's URL mapping. To achieve that, open the `urls.py` file in the `bookr` directory and append the following highlighted line to the end of the `urlpatterns` list:

```
urlpatterns = [
    ...,
    ...,
    path('', include('bookr_test.urls')),
]
```

4. Once the view has been set up, validate that it works correctly. Do this by running the following command:

```
python manage.py runserver localhost:8080
```

Then, visit `http://localhost:8080/test/greeting` in your web browser. Once the page opens, you should see the following text you added in *Step 1* to the greeting view being displayed in the browser:

```
Hey there, welcome to Bookr! Your one stop place to review books.
```

여기에서 사용자가 제공된 뷰에 매핑된 엔드포인트를 방문할 때마다 환영 메시지로 인사하는 간단한 Django 뷰를 생성했습니다.

2. 뷰가 생성되면, 이 뷰를 브라우저나 테스트 클라이언트에서 방문할 수 있는 URL 엔드포인트에 매핑해야 합니다. 이를 위해 `bookr_test` 디렉토리 아래에 `urls.py` 파일을 추가하고 `urlpatterns` 목록에 매핑을 포함한 다음 코드를 추가하세요:

```
from django.urls import path
from . import views

urlpatterns = [
    path('test/greeting', views.greeting_view,
         name='greeting_view'),
]
```

앞서 언급한 코드 조각에서 우리는 `urlpatterns` 목록에 경로를 설정하여 애플리케이션의 'test/greeting' 엔드포인트에 `greeting_view`를 매핑했습니다.

3. 이 경로가 설정되면, 우리 프로젝트에서 이 경로를 인식하도록 해야 합니다. 이를 위해 `bookr` 프로젝트의 URL 매핑에 이 항목을 추가해야 합니다. 이를 달성하기 위해 `bookr` 디렉토리의 `urls.py` 파일을 열고 `urlpatterns` 목록의 끝에 다음 강조된 줄을 추가합니다:

```
urlpatterns = [
    ...,
    ...,
    path('', include('bookr_test.urls')),
]
```

4. 뷰가 설정되면, 제대로 작동하는지 확인하십시오. 다음 명령을 실행하여 확인하십시오:

```
python manage.py runserver localhost:8080
```

그런 다음 웹 브라우저에서 `http://localhost:8080/test/greeting`을 방문하세요. 페이지가 열리면, `idx_4d380bb8`에서 1단계에서 추가한 다음 텍스트가 `idx_da8c2d33`의 인사 보기에서 브라우저에 표시되는 것을 볼 수 있어야 합니다:

안녕하세요, Bookr에 오신 것을 환영합니다! 책을 리뷰할 수 있는 원스톱 장소입니다.

5. Now, we are ready to write the test cases for `greeting_view`. For this exercise, we are going to write a test case that checks whether, on visiting the `/test/greeting` endpoint, you get a successful result or not. To implement this test case, open the `tests.py` file under the `bookr_test` directory and add the following code to the end of the file:

```
from django.test import TestCase

class TestGreetingView(TestCase):
    """Test the greeting view."""
    def test_greeting_view(self):
        response = self.client.get('/test/greeting')
        self.assertEqual(response.status_code, 200)
```

In the preceding code snippet, we defined a test case that helps validate whether the greeting view is working fine or not.

This is done by first importing Django's test client, which allows you to test views mapped to the URLs by making calls to them and analyzing the generated response:

```
from django.test import TestCase, Client
```

Once the import has been done, we must create a new class named `TestGreetingView` that will group the test cases related to the greeting view we created in *Step 2*:

```
class TestGreetingView(TestCase):
```

Inside this test case, we defined two methods, `setUp()` and `test_greeting_view()`. The `test_greeting_view()` method implements your test case. Inside this, we first make an HTTP GET call to the URL that is mapped to the greeting view and store the response generated by the view inside the response object created:

```
response = self.client.get('/test/greeting')
```

Once this call finishes, you will have its HTTP response code, contents, and headers available inside the response variable. Next, with the following code, we make an

5. 이제 우리는 `greeting_view`에 대한 테스트 케이스를 작성할 준비가 되었습니다. 이 연습에서는 `/test/greeting` 엔드포인트를 방문할 때 성공적인 결과를 얻는지 확인하는 테스트 케이스를 작성할 것입니다. 이 테스트 케이스를 구현하기 위해 `bookr_test` 디렉토리 아래의 `tests.py` 파일을 열고 파일 끝에 다음 코드를 추가하세요:

```
from django.test import TestCase

class TestGreetingView(TestCase):
    """인사 보기 테스트."""
    def test_greeting_view(self):
        response = self.client.get('/test/greeting')
        self.assertEqual(response.status_code, 200)
```

앞선 코드 조각에서는 `greeting` 뷰가 제대로 작동하는지 확인하는 테스트 케이스를 정의했습니다.

이는 먼저 Django의 테스트 클라이언트를 가져오는 것으로 수행되며, 이를 통해 URL에 매핑된 뷰를 호출하고 생성된 응답을 분석할 수 있습니다:

```
from django.test import TestCase, Client
```

가져오기가 완료되면, 2단계에서 생성한 `greeting` 뷰와 관련된 테스트 케이스를 그룹화할 `TestGreetingView`라는 새 클래스를 만들어야 합니다:

```
class TestGreetingView(TestCase):
```

이 테스트 케이스 내에서, 우리는 두 개의 메서드인 `setUp()`과 `test_greeting_view()`를 정의했습니다. `test_greeting_view()` 메서드는 테스트 케이스를 구현합니다. 이 안에서 우리는 먼저 `greeting` 뷰에 매핑된 URL에 HTTP GET 호출을 하고, 뷰에서 생성된 응답을 `response` 객체에 저장합니다:

```
response = self.client.get('/test/greeting')
```

이 호출이 완료되면, `response` 변수 내에서 HTTP 응답 코드, 내용 및 헤더를 사용할 수 있습니다. 다음으로, 다음 코드를 사용하여 우리는 하나의

assertion, validating whether the status code generated by the call matches the status code for successful HTTP calls (HTTP 200):

```
self.assertEqual(response.status_code, 200)
```

With this, we are now ready to run the tests.

6. With the test case written, look at what happens when you run the test case. This can be done by running the following command:

```
python manage.py test
```

Once the command executes, you can expect to see an output like the one shown in the following snippet:

```
% python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
...
-----
Ran 3 tests in 0.006s

OK
Destroying test database for alias 'default'...
```

As you can see from the output, your test cases executed successfully, hence validating that the response generated by the `greeting_view()` method is as per your expectations.

In this exercise, we learned how we can implement a test case for a Django view function and use Django's `TestClient` to assert that the output generated by the view function matches the one the developer should see.

In the next section, we will learn how to test views that have authentication enabled.

주장, 호출에 의해 생성된 상태 코드가 성공적인 HTTP 호출에 대한 상태 코드 (HTTP 200)와 일치하는지 검증하기:

```
self.assertEqual(response.status_code, 200)
```

이제 우리는 테스트를 실행할 준비가 되었습니다.

6. 테스트 케이스가 작성되었으므로, 테스트 케이스를 실행할 때 어떤 일이 발생하는지 살펴보세요. 이는 다음 명령어를 실행하여 수행할 수 있습니다:

```
python manage.py test
```

명령어가 실행되면, 다음 스니펫에 표시된 것과 같은 출력을 볼 수 있을 것입니다:

```
% python manage.py test
'default'에 대한 테스트 데이터베이스 생성 중...
시스템 점검에서 문제가 발견되지 않았습니다 (0개 무
시됨)...

-----
3개의 테스트를 0.006초에 실행했습니다

확인

'default'에 대한 테스트 데이터베이스 파괴 중...
```

idx_74059af0의 출력에서 볼 수 있듯이, 테스트 케이스가 성공적으로 실행되었으므로 idx_19cb9d93 응답이 `greeting_view()` 메서드에 의해 생성된 것이 귀하의 기대에 부합함을 검증합니다.

이번 연습에서는 Django 뷰 함수에 대한 테스트 케이스를 구현하는 방법과 Django의 `TestClient`를 사용하여 뷰 함수에 의해 생성된 출력이 개발자가 보아야 할 것과 일치하는지 확인하는 방법을 배웠습니다.

다음 섹션에서는 인증이 활성화된 뷰를 테스트하는 방법을 배울 것입니다.

Testing views with authentication

In the previous example, we looked at how we can test views inside Django. An important part to be highlighted about this view is that the view we created could be accessed by anyone and is not protected by any authentication or login checks. Now, imagine a case where a view should only be accessible if the user is logged in. For example, consider implementing a view function that renders a profile page of a registered user of our web application. To make sure that only logged-in users can view the profile page for their account, you might want to restrict the view to logged-in users only.

With this, we now have an important question: *How can we test views that require authentication?*

Luckily, Django's test client provides this functionality, through which we can log in to our views and then run tests over them. This result can be achieved using Django's test client `login()` method. When this method is called, Django's test client performs an authentication operation against the service. If the authentication succeeds, it stores the login cookie internally, which it can then use for further test runs. The following code snippet shows how you can set up Django's test client to simulate a logged-in user:

```
login = self.client.login(username='testuser',
                           password='testpassword')
```

The `login` method requires a username and password for the test user that we are going to test with, as will be shown in the next exercise. So, let's take a look at how we can test a flow that requires user authentication.

Exercise 14.04 – Writing test cases to validate authenticated users

In this exercise, we will write test cases for views that require the user to be authenticated. As part of this, we will validate the output generated by the view method when a user who is not logged in tries to visit the page and when a user who is logged in tries to visit the page mapped to the view function. Let's get started with the steps:

1. For this exercise, we are going to use the `bookr_test` application that you created in *Step 1 of Exercise 14.02 – Testing Django models*. To get started, open the `views.py` file under the `bookr_test` application and add the following code to it:

```
from django.http import HttpResponseRedirect
from django.contrib.auth.decorators import \
    login_required
```

인증이 있는 뷰 테스트

이전 예제에서는 장고 내에서 뷰를 테스트하는 방법을 살펴보았습니다. 이 뷰에 대해 강조해야 할 중요한 부분은 우리가 만든 뷰가 누구나 접근할 수 있으며 인증이나 로그인 검사가 보호하지 않는다는 것입니다. 이제 사용자가 로그인했을 때만 접근할 수 있는 뷰의 경우를 상상해 보십시오. 예를 들어, 우리 웹 애플리케이션의 등록된 사용자의 프로필 페이지를 렌더링하는 뷰 함수를 구현한다고 가정해 보겠습니다. 로그인한 사용자만 자신의 계정에 대한 프로필 페이지를 볼 수 있도록 하려면 뷰를 로그인한 사용자로 제한하고 싶을 것입니다.

이제 우리는 중요한 질문이 있습니다: 인증이 필요한 뷰를 어떻게 테스트할 수 있을까요?

다행히도 장고의 테스트 클라이언트는 이 기능을 제공하여, 우리는 뷰에 로그인한 다음 그 위에서 테스트를 실행할 수 있습니다. 이 결과는 장고의 테스트 클라이언트의 `login()` 메서드를 사용하여 달성할 수 있습니다. 이 메서드가 호출되면 장고의 테스트 클라이언트는 서비스에 대해 인증 작업을 수행합니다. 인증이 성공하면 내부적으로 로그인 쿠키를 저장하며, 이후의 테스트 실행에 사용할 수 있습니다. 다음 코드 스니펫은 로그인한 사용자를 시뮬레이션하기 위해 장고의 테스트 클라이언트를 설정하는 방법을 보여줍니다:

```
login = self.client.login(username='testuser',
                           password='testpassword')
```

로그인 메서드는 우리가 테스트할 테스트 사용자에게 대한 사용자 이름과 비밀번호를 요구합니다. 다음 연습에서 보여줄 것입니다. 그러니 사용자 인증이 필요한 흐름을 테스트하는 방법을 살펴보겠습니다.

연습 14.04 – 인증된 사용자를 검증하기 위한 테스트 케이스 작성

이 연습에서는 사용자가 인증되어야 하는 뷰에 대한 테스트 케이스 `idx_11796d78`를 작성할 것입니다. 이의 일환으로, 로그인하지 않은 사용자가 페이지를 방문하려고 할 때와 로그인한 사용자가 뷰 함수에 매핑된 페이지를 방문하려고 할 때 뷰 메서드에 의해 생성된 출력을 검증할 것입니다. 단계에 따라 시작해 보겠습니다:

1. 이 연습을 위해, 우리는 Exercise 14.02 – Django 모델 테스트의 1단계에서 생성한 `bookr_test` 애플리케이션을 사용할 것입니다. 시작하려면, `bookr_test` 애플리케이션 아래의 `views.py` 파일을 열고 다음 코드를 추가하세요:

```
from django.http import HttpResponse
from django.contrib.auth.decorators import \
    login_required
```

2. Once the preceding code snippet has been added, create a new function called `greeting_view_user()` at the end of the file, as shown in the following code snippet:

```
@login_required
def greeting_view_user(request):
    """Greeting view for the user."""
    user = request.user
    return HttpResponse(f"Welcome to Bookr! {user}")
```

Here, we have created a simple Django view that will be used to greet the logged-in user with a welcome message whenever they visit an endpoint mapped to the provided view.

3. Once this view has been created, we need to map this view to a URL endpoint that can then be visited in a browser or a test client. To do this, open the `urls.py` file under the `bookr_test` directory and add the highlighted code to it:

```
from django.urls import path
from . import views

urlpatterns = [ ...
    path('test/greet_user', views.greeting_view_user,
         name='greeting_view_user'),
]
```

In the preceding code snippet, we mapped `greeting_view` to the `'test/greet_user'` endpoint for the application by setting the path in the `urlpatterns` list:

```
path('test/greet_user', views.greeting_view_user,
     name='greeting_view_user')
```

If you followed the previous exercises, this URL should already be set up for detection in the project, and no further steps are required to configure the URL mapping.

4. Once the view has been set up, the next thing we need to do is validate whether it works correctly. To do this, run the following command:

```
python manage.py runserver localhost:8080
```

Then, visit `http://localhost:8080/test/greet_user` in your web browser.

2. 앞서 언급한 코드 조각이 추가되면, 파일 끝에 `greeting_view_user()`라는 새 함수를 생성합니다. 다음 코드 조각과 같이:

```
@login_required
def greeting_view_user(request):
    """사용자를 위한 인사 보기."""
    user = request.user
    return HttpResponse(f"Bookr에 오신 것을 환영합니다! {user}")
```

여기에서 우리는 사용자가 제공된 뷰에 매핑된 엔드포인트를 방문할 때마다 로그인한 사용자에게 환영 메시지를 표시하는 간단한 Django 뷰를 만들었습니다.

3. 이 뷰가 `idx_305f3a87` 생성되면, 브라우저나 테스트 클라이언트에서 방문할 수 있는 URL 엔드포인트에 이 뷰를 매핑해야 합니다. 이를 위해 `bookr_test` 디렉토리 아래의 `urls.py` 파일을 열고 강조된 코드를 추가합니다:

```
from django.urls import path
from . import views

urlpatterns = [ ...
    path('test/greet_user', views.greeting_view_user,
         name='greeting_view_user'),
]
```

앞의 코드 조각에서 우리는 `urlpatterns` 목록에서 경로를 설정하여 애플리케이션의 `'test/greet_user'` 엔드포인트에 `greeting_view`를 매핑했습니다:

```
path('test/greet_user', views.greeting_view_user,
     name='greeting_view_user')
```

이전 연습을 따랐다면, 이 URL은 이미 프로젝트에서 감지를 위해 설정되어 있어야 하며, URL 매핑을 구성하기 위한 추가 단계는 필요하지 않습니다.

4. 뷰가 설정되면, 다음으로 해야 할 일은 제대로 작동하는지 검증하는 것입니다. 이를 위해 다음 명령어를 실행하세요:

```
python manage.py runserver localhost:8080
```

그런 다음, 웹 브라우저에서 `http://localhost:8080/test/greet_user`를 방문하세요.

If you are not logged in already, by visiting the preceding URL, you will be redirected to the login page for the project.

5. Now, write the test cases for `greeting_view_user`, which check whether, on visiting the `/test/greet_user` endpoint, you get a successful result or not. To implement this test case, open the `tests.py` file under the `bookr_test` directory and add the following code to it:

```
from django.contrib.auth.models import User

class TestLoggedInGreetingView(TestCase):
    """Test the greeting view for the authenticated
    users."""
    def setUp(self):
        test_user = User.objects.create_user(
            username='testuser',
            password='test@#628password')
        test_user.save()
        self.client = Client()

    def test_user_greeting_not_authenticated(self):
        response = self.client.get('/test/greet_user/')
        self.assertEqual(response.status_code, 302)

    def test_user_authenticated(self):
        login = self.client.login(username='testuser',
            password='test@#628password')
        response = self.client.get('/test/greet_user/')
        self.assertEqual(response.status_code, 200)
```

In the preceding code snippet, we implemented a test case that checks the views that have authentication enabled before their content can be seen.

Here, first, we imported the required classes and methods that will be used to define the test case and initialize a testing client:

```
from django.test import TestCase, Client
```

The next thing you require is the `User` model from Django's `auth` module:

```
from django.contrib.auth.models import User
```

이전 URL을 방문하여 idx_6736939balready 로그인하지 않은 경우, 프로젝트의 로그인 페이지로 리디렉션됩니다.

5. 이제 /test/greet_user 엔드포인트를 방문할 때 성공적인 결과를 얻는지 확인하는 `greeting_view_user`에 대한 테스트 케이스를 작성하세요. 이 테스트 케이스를 구현하려면 `bookr_test` 디렉토리 아래의 `tests.py` 파일을 열고 다음 코드를 추가하세요:

```
from django.contrib.auth.models import User

class TestLoggedInGreetingView(TestCase):
    """인증된 사용자를 위한 인사 보기 테스트."""

    def setUp(self):
        test_user = User.objects.create_user(
            username='testuser',
            password='test@#628password')
        test_user.save()
        self.client = Client()

    def test_user_greeting_not_authenticated(self):
        response = self.client.get('/test/greet_user/')
        self.assertEqual(response.status_code, 302)

    def test_user_authenticated(self):
        login = self.client.login(username='testuser',
                                   password='test@#628password')
        response = self.client.get('/test/greet_user/')
        self.assertEqual(response.status_code, 200)
```

앞선 코드 조각에서는 콘텐츠를 보기 전에 인증이 활성화된 뷰를 확인하는 테스트 케이스를 구현했습니다.

여기서 먼저, 테스트 케이스를 정의하고 테스트 클라이언트를 초기화하는 데 사용될 필수 클래스와 메서드를 가져왔습니다:

```
from django.test import TestCase, Client
```

다음으로 필요한 것은 Django의 `auth` 모듈에서 `User` 모델입니다:

```
from django.contrib.auth.models import User
```

This model is required because, for the test cases requiring authentication, we will need to initialize a new test user. Next up, we created a new class named `TestLoggedInGreetingView`, which wraps your tests related to the `greeting_user` view (which requires authentication). Inside this class, you defined three methods called `setUp()`, `test_user_greeting_not_authenticated()`, and `test_user_authenticated()`. The `setUp()` method is used to initialize a test user that you will use for authentication. This is a required step because a test environment inside Django is a completely isolated environment that doesn't use data from your production application, so all the required models and objects are to be instantiated separately inside the test environment.

Then, we created the test user and initiated the test client using the following code:

```
test_user = User.objects.create_user(
    username='testuser', password='test@#628password')
test_user.save()
self.client = Client()
```

Next, we wrote the test case for the `greet_user` endpoint when the user is not authenticated. Here, you should expect Django to redirect the user to the login endpoint. This redirect can be detected by checking the HTTP status code of the response, which should be set to HTTP 302, indicating a redirect operation:

```
def test_user_greeting_not_authenticated(self):
    response = self.client.get('/test/greet_user/')
    self.assertEqual(response.status_code, 302)
```

Next, we wrote another test case to check whether the `greet_user` endpoint renders successfully when the user is authenticated. To authenticate the user, you must first call the `login()` method of the test client and perform authentication by providing the username and password of the test user you created in the `setUp()` method, as follows:

```
login = self.client.login(username='testuser',
    password='test@#628password')
```

이 모델은 `idx_57703a58`이 필요합니다. 인증이 필요한 테스트 케이스의 경우, 새로운 테스트 사용자를 초기화해야 합니다. 다음으로, 인증이 필요한 `greeting.userview`와 관련된 테스트를 감싸는 `TestLoggedInGreetingView`라는 새로운 클래스를 만들었습니다. 이 클래스 안에서 `setUp()`, `test_user_greeting_not_authenticated()`, `test_user_authenticated()`라는 세 가지 메서드를 정의했습니다. `setUp()` 메서드는 인증에 사용할 테스트 사용자를 초기화하는 데 사용됩니다. 이는 Django 내의 테스트 환경이 프로덕션 애플리케이션의 데이터를 사용하지 않는 완전히 격리된 환경이기 때문에 필수적인 단계입니다. 따라서 모든 필요한 모델과 객체는 테스트 환경 내에서 별도로 인스턴스화되어야 합니다.

그런 다음, 다음 코드를 사용하여 테스트 사용자를 만들고 테스트 클라이언트를 초기화했습니다:

```
test_user = User.objects.create_user(
    username='testuser', password='test@#628password')
test_user.save()
self.client = Client()
```

다음으로, 사용자가 인증되지 않았을 때 `greet_user` 엔드포인트에 대한 테스트 케이스를 작성했습니다. 여기서 `idx_bebfe8cc`는 Django가 사용자를 로그인 엔드포인트로 리디렉션할 것으로 예상해야 합니다. 이 리디렉션은 응답의 HTTP 상태 코드를 확인하여 감지할 수 있으며, 이는 HTTP 302로 설정되어 리디렉션 작업을 나타냅니다:

```
def test_user_greeting_not_authenticated(self):
    response = self.client.get('/test/greet_user/')
    self.assertEqual(response.status_code, 302)
```

다음으로, 사용자가 인증되었을 때 `greet_user` 엔드포인트가 성공적으로 렌더링되는지 확인하기 위해 또 다른 테스트 케이스를 작성했습니다. 사용자를 인증하려면 먼저 테스트 클라이언트의 `login()` 메서드를 호출하고 `setUp()` 메서드에서 생성한 테스트 사용자의 사용자 이름과 비밀번호를 제공하여 인증을 수행해야 합니다, 다음과 같이:

```
login = self.client.login(username='testuser',
    password='test@#628password')
```

Once the login has been completed, we must make an HTTP GET request to the `greet_user` endpoint and validate whether the endpoint generates a correct result or not by checking the HTTP status code of the returned response:

```
response = self.client.get('/test/greet_user/')
self.assertEqual(response.status_code, 200)
```

6. With the test cases written, it's time to check how they run. For this, run the following command:

```
python manage.py test
```

Once the execution finishes, we can expect to see a response that resembles the following one:

```
% python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 5 tests in 0.366s

OK
Destroying test database for alias 'default'...
```

As we can see from the preceding output, our test cases have passed successfully, validating that the view we created generates the desired response of redirecting the user if the user is unauthenticated to the website and allowing the user to see the page if the user is authenticated.

In this exercise, we implemented a test case where we can test what output is generated by a view function based on whether the user is authenticated or not.

In the next section, we will explore how to create requests in Django unit tests without the use of a Django-provided test client, but by providing raw request data.

로그인이 완료되면, 우리는 `greet_user` 엔드포인트에 HTTP GET 요청을 해야 하며, 반환된 응답의 HTTP 상태 코드를 확인하여 엔드포인트가 올바른 결과를 생성하는지 여부를 검증해야 합니다:

```
response = self.client.get('/test/greet_user/')
self.assertEqual(response.status_code, 200)
```

6. 테스트 케이스가 작성되었으므로, 이제 그것들이 어떻게 실행되는지 확인할 시간입니다. 이를 위해 다음 명령어를 실행하세요:

```
python manage.py test
```

실행이 끝나면, 다음과 유사한 응답을 볼 수 있을 것으로 기대할 수 있습니다:

```
% python manage.py test
별칭 'default'에 대한 테스트 데이터베이스 생성
중... 시스템 검사가 문제를 발견하지 못했습니다 (0
무시됨).....

-----
5개의 테스트를 0.366초에 실행했습니다.

확인
별칭 'default'에 대한 테스트 데이터베이스를 파괴하는 중...
```

`idx_f5dacao0`의 이전 출력에서 볼 수 있듯이, 우리의 테스트 케이스는 성공적으로 통과하였으며, 우리가 생성한 뷰가 사용자가 인증되지 않은 경우 웹사이트로 리디렉션하고 사용자가 인증된 경우 페이지를 볼 수 있도록 하는 원하는 응답을 생성함을 검증합니다.

이번 연습에서는 사용자가 인증되었는지 여부에 따라 뷰 함수가 생성하는 출력을 테스트할 수 있는 테스트 케이스를 구현했습니다.

다음 섹션에서는 Django에서 제공하는 테스트 클라이언트를 사용하지 않고 원시 요청 데이터를 제공하여 Django 단위 테스트에서 요청을 생성하는 방법을 탐구할 것입니다.

Django RequestFactory

So far, we have been using Django's test client to test the views that we have created for our application. The test client class simulates a browser and uses this simulation to make calls to the required APIs. But what if we didn't want to use the test client and its associated simulation of being a browser, but rather wanted to test the view functions by directly bypassing the request parameter? How can we do that?

To help us in such cases, we can leverage the RequestFactory class provided by Django. The RequestFactory class helps us provide the request object that we can pass to our view functions to evaluate whether they are working. The following object for the RequestFactory class can be created by instantiating the class as follows:

```
factory = RequestFactory()
```

The factory object we have created only supports HTTP methods such as `get()`, `post()`, `put()`, and others to simulate a call to any URL endpoint. Now, let's learn how to modify the test case that we wrote in *Exercise 14.04 – Writing test cases to validate authenticated users*, so that it uses RequestFactory.

Exercise 14.05 – Using RequestFactory to test views

In this exercise, we will use RequestFactory to test view functions in Django:

1. For this exercise, we are going to use the existing `greeting_view_user` view function that we created earlier, in *Step 1 of Exercise 14.04 – Writing test cases to validate authenticated users*, which looks like this:

```
@login_required
def greeting_view_user(request):
    """Greeting view for the user."""
    user = request.user
    return HttpResponse(f"Welcome to Bookr! {user}")
```

장고 요청 팩토리

지금까지 우리는 우리 애플리케이션을 위해 만든 뷰를 테스트하기 위해 장고의 테스트 클라이언트를 사용해왔습니다. 테스트 클라이언트 클래스는 브라우저를 시뮬레이션하고 이 시뮬레이션을 사용하여 필요한 API에 호출을 합니다. 하지만 만약 테스트 클라이언트와 브라우저의 시뮬레이션을 사용하고 싶지 않고, 요청 매개변수를 직접 우회하여 뷰 함수를 테스트하고 싶다면 어떻게 해야 할까요?

이런 경우에 도움을 주기 위해, 우리는 장고에서 제공하는 RequestFactory 클래스를 활용할 수 있습니다. RequestFactory 클래스는 우리가 뷰 함수에 전달할 수 있는 요청 객체를 제공하는 데 도움을 줍니다. 다음과 같이 클래스를 인스턴스화하여 RequestFactory 클래스의 객체를 생성할 수 있습니다:

```
factory = RequestFactory()
```

우리가 생성한 factory 객체는 get(), post(), put()과 같은 HTTP 메서드만 지원하여 어떤 URL 엔드포인트에 대한 호출을 시뮬레이션합니다. 이제, 우리가 14.04 연습문제에서 작성한 테스트 케이스를 수정하여 RequestFactory를 사용하도록 배우겠습니다.

연습문제 14.05 – RequestFactory를 사용하여 뷰 테스트하기

이번 연습문제에서는 RequestFactory를 사용하여 장고의 뷰 함수를 테스트할 것입니다:

1. 이번 연습문제에서는 14.04 연습문제 1단계에서 이전에 만든 기존 `greeting_view_user` 뷰 함수를 사용할 것입니다. 이 함수는 다음과 같습니다:

```
@로그인_필수
def greeting_view_user(request):
    """사용자를 위한 인사 보기."""
    user = request.user
    return HttpResponse(f"Bookr에 오신 것을 환영합니다! {user}")
```

2. Next, modify the existing test case, `TestLoggedInGreetingView`, defined inside the `tests.py` file under the `bookr_test` directory. Open the `tests.py` file and make the following changes:

- First, we need to add the following import to use the `RequestFactory` class inside the test cases:

```
from django.test import (TestCase, Client
                        RequestFactory)
```

- The next thing we need is an import for the `AnonymousUser` class from Django's `auth` module and the `greeting_view_user` view method from the `views` module. This is required to test the view functions with a simulated user who is not logged in. This can be done by adding the following code:

```
from django.contrib.auth.models import (AnonymousUser,
                                       User)

from .views import greeting_view_user
```

3. Once the import statements have been added, modify the `setUp()` method of the `TestLoggedInGreetingView` class and change its contents so that it resembles the following:

```
def setUp(self):
    self.test_user = User.objects.create_user(
        username='testuser',
        password='test@#628password')
    self.test_user.save()
    self.factory = RequestFactory()
```

In this method, we first created a user object and stored it as a class member so that we can use it later in the tests. Once the user object was created, we instantiated a new instance of the `RequestFactory` class to test our view function.

4. With the `setUp()` method now defined, modify the existing tests so that they use the `RequestFactory` instance. For the test for a non-authenticated call to the view

2. 다음으로, bookr_test 디렉토리의 tests.py 파일 내에 정의된 기존 테스트 케이스인 TestLoggedInGreetingView를 수정합니다. tests.py 파일을 열고 다음과 같은 변경을 합니다:

- 먼저, 테스트 케이스 내에서 RequestFactory 클래스를 사용하기 위해 다음 import를 추가해야 합니다:

```
django.test에서 (TestCase, Client 가져오기
RequestFactory)
```

- 다음으로 필요한 것은 Django의 auth 모듈에서 AnonymousUser 클래스를 가져오고 views 모듈에서 greeting_view_user 뷰 메서드를 가져오는 것입니다. 이는 로그인하지 않은 사용자로 시뮬레이션된 뷰 함수를 테스트하기 위해 필요합니다. 다음 코드를 추가하여 이 idx_a919b581를 수행할 수 있습니다:

```
from django.contrib.auth.models import (AnonymousUser,
User)

from .views import greeting_view_user
```

3. 가져오기 문이 추가되면 TestLoggedInGreetingView 클래스의 setUp() 메서드를 수정하고 다음과 비슷하게 내용을 변경합니다:

```
def setUp(self):
    self.test_user = User.objects.create_user(
        username='testuser',
        password='test@#628password')
    self.test_user.save()
    self.factory = RequestFactory()
```

이 방법에서는 먼저 사용자 객체를 생성하고 이를 클래스 멤버로 저장하여 나중에 테스트에서 사용할 수 있도록 했습니다. 사용자 객체가 생성되면, RequestFactory 클래스의 새 인스턴스를 인스턴스화하여 우리의 뷰 함수를 테스트했습니다.

4. setUp() 메서드가 이제 정의되었으므로 기존 테스트를 수정하여 RequestFactory 인스턴스를 사용하도록 하십시오. 뷰에 대한 비인증 호출 테스트의 경우

function, modify the `test_user_greeting_not_authenticated` method so that it has the following contents:

```
def test_user_greeting_not_authenticated(self):
    request = self.factory.get('/test/greet_user')
    request.user = AnonymousUser()
    response = greeting_view_user(request)
    self.assertEqual(response.status_code, 302)
```

In this method, we first created a request object using the `RequestFactory` instance we defined in the `setUp()` method. Once we had done that, we assigned an `AnonymousUser()` instance to the `request.user` property. Assigning the `AnonymousUser()` instance to the property makes the view function think that the user making the request is not logged in:

```
request.user = AnonymousUser()
```

Once we did this, we made a call to the `greeting_view_user()` view method and passed to it the request object you created. Once the call is successful, we must capture the output of the method in the response variable using the following code:

```
response = greeting_view_user(request)
```

For the unauthenticated user, we expect to get a redirect response that can be tested by checking the HTTP status code of the response, as follows:

```
self.assertEqual(response.status_code, 302)
```

5. Now, go ahead and modify the other method, `test_user_authenticated()`, similarly by using the `RequestFactory` instance, as follows:

```
def test_user_authenticated(self):
    request = self.factory.get('/test/greet_user')
    request.user = self.test_user
    response = greeting_view_user(request)
    self.assertEqual(response.status_code, 200)
```

As you can see, most of the code resembles the code you wrote in the `test_user_greeting_not_authenticated` method, with a small change – in this

함수, `test_user_greeting_not_authenticated` 메서드를 다음 내용을 가지도록 수정하세요:

```
def test_user_greeting_not_authenticated(self):
    request = self.factory.get('/test/greet_user')
    request.user = AnonymousUser()
    response = greeting_view_user(request)
    self.assertEqual(response.status_code, 302)
```

이 메서드에서는 먼저 `setUp()` 메서드에서 정의한 `RequestFactory` 인스턴스를 사용하여 요청 객체를 생성했습니다. 그렇게 한 후, `request.user` 속성에 `AnonymousUser()` 인스턴스를 할당했습니다. `AnonymousUser()` 인스턴스를 속성에 할당하면 뷰 함수는 요청을 하는 사용자가 로그인하지 않은 것으로 생각합니다:

```
request.user = AnonymousUser()
```

이 작업을 수행한 후, 우리는 `greeting_view_user()` 뷰 메서드를 호출하고 당신이 생성한 요청 객체를 전달했습니다. 호출이 성공하면, 다음 코드를 사용하여 메서드의 출력을 `response` 변수에 캡처해야 합니다:

```
response = greeting_view_user(request)
```

인증되지 않은 사용자에게 대해서는, 다음과 같이 응답의 HTTP 상태 코드를 확인하여 테스트할 수 있는 리디렉션 응답을 받을 것으로 예상합니다:

```
self.assertEqual(response.status_code, 302)
```

5. 이제 `RequestFactory` 인스턴스를 사용하여 `test_user_authenticated()` 메서드도 유사하게 수정하세요:

```
def test_user_authenticated(self):
    request = self.factory.get('/test/greet_user')
    request.user = self.test_user
    response = greeting_view_user(request)
    self.assertEqual(response.status_code, 200)
```

보시다시피, 대부분의 `idx_bee856d4of` 코드는 `test_user_greeting_not_authenticated` 메서드에서 작성한 코드와 유사하며, 작은 변경 사항이 있습니다 – 이 부분에서

method, instead of using `AnonymousUser` for the `request.user` property, you are using the `test_user` that you created in the `setUp()` method:

```
request.user = self.test_user
```

With the changes done, it's time to run the tests.

6. To run the tests and validate whether `RequestFactory` works as expected or not, run the following command:

```
python manage.py test
```

Once the command executes, you can expect to see an output that resembles the one shown here:

```
% python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 6 tests in 0.248s

OK
Destroying test database for alias 'default'...
```

As we can see from the output, the test cases written by us have passed successfully, hence validating the behavior of the `RequestFactory` class.

With this exercise, we learned how we can write test cases for view functions by leveraging `RequestFactory` and passing the request object directly to the view function, rather than simulating a URL visit using the test client approach, and hence allowing for more direct testing.

In the next section, we will test class-based views.

Testing class-based views

In the previous exercise, we saw how we can test views defined as methods. But what about class-based views? How can we test those?

메서드에서 `request.user` 속성에 대해 `AnonymousUser`를 사용하는 대신, `setUp()` 메서드에서 생성한 `test_user`를 사용하고 있습니다:

```
request.user = self.test_user
```

변경이 완료되었으므로, 이제 테스트를 실행할 시간입니다.

6. 테스트를 실행하고 `RequestFactory`가 예상대로 작동하는지 확인하려면 다음 명령을 실행하세요:

```
python manage.py test
```

명령이 실행되면, 여기 표시된 것과 유사한 출력을 볼 수 있습니다:

```
% python manage.py test
'default'에 대한 테스트 데이터베이스 생성 중...
시스템 점검에서 문제가 발견되지 않았습니다 (0개 무
시됨).....

-----
6개의 테스트를 0.248초에 실행했습니다

확인

'default'에 대한 테스트 데이터베이스 파괴 중...
```

출력에서 `idx_14cd87fc`를 보면, 우리가 작성한 테스트 케이스가 `idx_195ab945`를 성공적으로 통과했으므로 `RequestFactory` 클래스의 동작을 검증합니다.

이번 연습을 통해 우리는 `RequestFactory`를 활용하여 요청 객체를 뷰 함수에 직접 전달함으로써 뷰 함수에 대한 테스트 케이스를 작성하는 방법을 배웠습니다. 이는 테스트 클라이언트 접근 방식을 사용하여 URL 방문을 시뮬레이션하는 것보다 더 직접적인 테스트를 가능하게 합니다.

다음 섹션에서는 클래스 기반 뷰를 테스트할 것입니다.

클래스 기반 뷰 테스트

이전 연습에서 우리는 메서드로 정의된 뷰를 테스트하는 방법을 보았습니다. 하지만 클래스 기반 뷰는 어떻게 테스트할 수 있을까요?

As it turns out, it is quite easy to test class-based views. For example, if we have a class-based view defined with the name `ExampleClassView(View)`, to test the view, all we need to do is use the following syntax:

```
response = ExampleClassView.as_view()(request)
```

It is as simple as that.

A Django application generally consists of several different components that can either work in isolation, such as models, and some other components that need to interact with the URL mapping and other parts of the framework to work. Testing these different components may require some steps that are common to only those components. For example, when testing a model, first, we may want to create certain objects of the `Model` class before we start testing, or for views, first, we may want to initialize a test client with user credentials.

As it turns out, Django also provides some other classes based on the `TestCase` class that can be used to write test cases of specific types about the type of component being used. Let's look at the different classes provided by Django.

Test case classes in Django

Beyond the base `TestCase` class provided by Django, which can be used to define a multitude of test cases for different components, Django also provides some specialized classes derived from the `TestCase` class.

These classes are used for specific types of test cases based on the capabilities they provide to the developer. Let's look at the different classes provided by Django.

The `SimpleTestCase` class

This class is derived from the `TestCase` class provided by Django's test module and should be used when writing simple test cases that test the view functions. Usually, this class is not preferred when your test case involves making database queries. The class also provides a lot of useful features, such as the following:

- Ability to check for exceptions raised by a view function
- Ability to test form fields
- An inbuilt test client
- Ability to verify a redirect by a view function
- Can match the equality of two HTML, JSON, or XML outputs generated by the view functions

결국, 클래스 기반 뷰를 테스트하는 것은 매우 쉽습니다. 예를 들어, `ExampleClassView` (View)라는 이름으로 정의된 클래스 기반 뷰가 있다면, 뷰를 테스트하기 위해 필요한 것은 다음 구문을 사용하는 것입니다:

```
response = ExampleClassView.as_view()(request)
```

그것은 그만큼 간단합니다.

Django 애플리케이션은 일반적으로 모델과 같이 독립적으로 작동할 수 있는 여러 가지 다른 구성 요소와 URL 매핑 및 프레임워크의 다른 부분과 상호 작용해야 하는 몇 가지 다른 구성 요소로 구성됩니다. 이러한 다양한 구성 요소를 테스트하려면 특정 구성 요소에만 공통적인 몇 가지 단계가 필요할 수 있습니다. 예를 들어, 모델을 테스트할 때는 먼저 테스트를 시작하기 전에 `Model` 클래스의 특정 객체를 생성하고, 뷰의 경우에는 먼저 사용자 자격 증명으로 테스트 클라이언트를 초기화할 수 있습니다.

결국, Django는 사용되는 구성 요소의 유형에 대한 특정 유형의 테스트 케이스를 작성하는 데 사용할 수 있는 `TestCase` 클래스를 기반으로 한 몇 가지 다른 클래스를 제공합니다. Django에서 제공하는 다양한 클래스를 살펴보겠습니다.

Django의 테스트 케이스 클래스

Django에서 제공하는 기본 `TestCase` 클래스를 넘어, 다양한 `idx_8734d329` 구성 요소에 대한 다수의 테스트 케이스를 정의하는 데 사용할 수 있는 Django는 `TestCase` 클래스에서 파생된 몇 가지 전문화된 클래스도 제공합니다.

이 클래스들은 개발자에게 제공하는 기능에 따라 특정 유형의 테스트 케이스에 사용됩니다. Django에서 제공하는 다양한 클래스를 살펴보겠습니다.

SimpleTestCase 클래스

이 클래스는 Django의 테스트 모듈에서 제공하는 `TestCase` 클래스에서 파생되었으며, 뷰 함수를 테스트하는 간단한 테스트 케이스를 작성할 때 사용해야 합니다. 일반적으로 이 클래스는 데이터베이스 쿼리를 포함하는 테스트 케이스에는 선호되지 않습니다. 이 클래스는 다음과 같은 유용한 기능을 많이 제공합니다:

- 뷰 함수에서 발생한 예외를 확인하는 기능 폼 필드
- 를 테스트하는 기능
- 내장된 테스트 클라이언트
- 뷰 함수에 의한 리디렉션을 검증하는 기능
- 두 개의 HTML, JSON 또는 XML 출력의 동등성을 뷰 함수에 의해 생성된 것과 일치시킬 수 있습니다.

Now that we have a basic idea of what a `SimpleTestCase` class is, let's move on and try to understand another type of test case class that helps in writing test cases about the interaction with databases.

The `TransactionTestCase` class

This class is derived from the `SimpleTestCase` class and should be used when writing test cases that involve interaction with the database, such as database queries, model object creations, and more.

The class provides the following added features:

- Ability to reset the database to a default state before a test case runs
- Can skip tests based on database features, which can be quite handy if the database being used for testing does not support all the features of your production database

The `LiveServerTestCase` class

This class is like the `TransactionTestCase` class, but with a small difference that the test cases written in the class, instead of using the default test client, use a live server created by Django.

This ability to run the live server for testing comes in handy when writing test cases that test for the rendered web pages and any interaction with them, which is not possible while using the default test client.

Such test cases can leverage tools such as Selenium, which can be used to build interactive test cases that modify the state of the rendered page by interacting with it.

Modularizing test code

In the previous exercises, we saw how we can write test cases for different components of our project. But an important aspect to note is that, till now, we have written the test cases for all the components in a single file.

This approach is okay when the application does not have a lot of views and models, but this can become problematic as our application grows because our `single_tests.py` file will be hard to maintain.

이제 SimpleTestCase 클래스가 무엇인지에 대한 기본적인 아이디어를 갖추었으니, 데이터베이스와의 상호작용에 대한 테스트 케이스를 작성하는 데 도움이 되는 또 다른 유형의 테스트 케이스 클래스를 이해해 보겠습니다.

TransactionTestCase 클래스

이 클래스는 SimpleTestCase 클래스에서 파생되며, 데이터베이스 쿼리, 모델 객체 생성 등과 같이 데이터베이스와의 상호작용이 포함된 테스트 케이스를 작성할 때 사용해야 합니다.

이 클래스는 다음과 같은 추가 기능을 제공합니다:

- 테스트 케이스가 실행되기 전에 데이터베이스를 기본 상태로 재설정할 수 있는 기능
- 테스트 중 사용되는 데이터베이스가 프로덕션 데이터베이스의 모든 기능을 지원하지 않는 경우, 데이터베이스 기능에 따라 테스트를 건너뛸 수 있습니다.

LiveServerTestCase 클래스

이 클래스는 TransactionTestCase 클래스와 비슷하지만, idx_073f9f78 클래스에 작성된 테스트 케이스가 기본 테스트 클라이언트를 사용하는 대신 Django에 의해 생성된 라이브 idx_6f64d22a 서버를 사용한다는 작은 차이가 있다.

테스트를 위해 라이브 서버를 실행할 수 있는 이 기능은 렌더링된 웹 페이지와 그와의 상호작용을 테스트하는 테스트 케이스를 작성할 때 유용하다. 기본 테스트 클라이언트를 사용할 때는 불가능하다.

이러한 테스트 케이스는 Selenium과 같은 도구를 활용할 수 있으며, 이를 통해 렌더링된 페이지의 상태를 수정하는 대화형 테스트 케이스를 구축할 수 있다.

테스트 코드 모듈화

이전 연습에서 우리는 프로젝트의 다양한 구성 요소에 대한 테스트 케이스를 작성하는 방법을 보았다. 그러나 중요한 점은 지금까지 모든 구성 요소에 대한 테스트 idx_d61ed817 케이스를 단일 파일에 작성했다는 것이다.

이 접근 방식은 애플리케이션에 많은 뷰와 모델이 없을 때는 괜찮지만, 애플리케이션이 성장함에 따라 단일 tests.py 파일을 유지 관리하기 어려워질 수 있다.

To avoid running into such scenarios, we should try to modularize our test cases so that the test cases for models are kept separately from test cases related to the views and other properties. To achieve this modularization, all we need to do is follow two simple steps:

1. Create a new directory named `tests` inside our application directory by running the following command:

```
mkdir tests
```

2. Create a new empty file named `__init__.py` inside the `tests` directory by running the following command:

```
touch __init__.py
```

This `__init__.py` file is required by Django to correctly detect the `tests` directory we created as a module and not a regular directory.

Once you have completed the preceding steps, you can go ahead and create new testing files for the different components in your application. For example, to write test cases for your models, you can create a new file named `test_models.py` inside the `tests` directory and add any associated code for your model testing inside this file.

Also, you don't need to take any other additional steps to run your tests. The same command will work perfectly fine for your modular testing code base as well:

```
python manage.py test
```

With this, we know how we can write test cases for our projects. So, how about we test this knowledge and write test cases for the *Bookr* project we are working on?

Running tests faster using parallelization

A well-built application has several test cases that provide high code coverage and test all the different code paths in the application. This also means that every time the application is built, all these tests need to be executed. This slows down the build process and often becomes a reason for developer frustration.

By default, when we run testing with Django, it runs one test at a time. However, this can be made faster. A very simple way Django allows us to decrease the build time is by allowing the developers to run the tests in parallel.

이러한 상황을 피하기 위해, 모델에 대한 테스트 케이스를 뷰 및 기타 속성과 관련된 테스트 케이스와 별도로 유지하도록 테스트 케이스를 모듈화하려고 해야 합니다. 이 모듈화를 달성하기 위해 우리가 해야 할 일은 두 가지 간단한 단계를 따르는 것입니다:

1. 다음 명령어를 실행하여 애플리케이션 디렉토리 내에 tests라는 새 디렉토리를 만듭니다:

```
mkdir tests
```

2. tests 디렉토리 안에 __init__.py라는 새 빈 파일을 생성하려면 다음 명령을 실행하세요. 다음 명령:

```
touch __init__.py
```

이 __init__.py 파일은 Django가 우리가 만든 tests 디렉토리를 일반 디렉토리가 아닌 모듈로 올바르게 인식하도록 하기 위해 필요합니다.

이전 단계를 완료하면 애플리케이션의 다양한 구성 요소에 대한 새로운 테스트 파일을 생성할 수 있습니다. 예를 들어, 모델에 대한 테스트 케이스를 작성하려면 tests 디렉토리 내에 test_models.py라는 새 파일을 만들고 이 파일 내에 모델 테스트와 관련된 코드를 추가할 수 있습니다.

또한 테스트를 실행하기 위해 다른 추가 단계를 수행할 필요가 없습니다. 동일한 명령이 모듈형 테스트 코드베이스에서도 완벽하게 작동합니다:

```
python manage.py test
```

이로써 우리는 프로젝트에 대한 테스트 케이스를 작성하는 방법을 알게 되었습니다. 그렇다면 우리가 작업 중인 Bookr 프로젝트에 대한 테스트 케이스를 작성하여 이 지식을 테스트해보는 것은 어떨까요?

병렬화를 사용하여 테스트를 더 빠르게 실행하기

잘 구축된 애플리케이션은 높은 코드 커버리지를 제공하고 애플리케이션의 다양한 코드 경로를 테스트하는 여러 테스트 케이스를 가지고 있습니다. 이는 애플리케이션이 빌드될 때마다 이러한 모든 테스트를 실행해야 함을 의미합니다. 이는 빌드 프로세스를 느리게 하고 종종 개발자의 불만의 원인이 됩니다.

기본적으로 Django로 테스트를 실행할 때 한 번에 하나의 테스트를 실행합니다. 그러나 이를 더 빠르게 만들 수 있습니다. Django가 빌드 시간을 줄일 수 있도록 허용하는 매우 간단한 방법은 개발자가 테스트를 병렬로 실행할 수 있도록 하는 것입니다.

The following command shows you how to run tests in Django in a parallel manner:

```
python manage.py test --parallel
```

When the preceding command is run, Django executes all the tests in the Django web application. The `--parallel` parameter causes Django to run N number of tests in parallel instead of in a serial manner. The number of tests that will run in parallel will be determined by the number of cores that are available on your development machine.

You can also explicitly tell Django how many tests it can run in parallel by mentioning the count just after the `--parallel` parameter, as shown here:

```
python manage.py test --parallel 4
```

This will tell Django to run only 4 test cases in parallel.

When the tests are being executed in parallel, every test case gets its own database and other environment variables to avoid problems that come up due to running code in a concurrent environment.

Now that we understand how to unit test our applications in Django, let's go ahead and implement our knowledge by implementing test cases of Bookr.

Activity 14.01 – Testing models and views in Bookr

In this activity, you will implement test cases for the *Bookr* project. You will implement test cases to validate the functioning of the models created inside the *reviews* application of the Bookr project, and then you will implement a simple test case for validating the *index view* inside the *reviews* application.

The following steps will help you work through this activity:

1. Create a directory named `tests` inside the *reviews* application directory so that all our test cases for the *reviews* application can be modularized.
2. Create an empty `__init__.py` file so that the directory is considered not as a general directory, but rather a Python module directory.
3. Create a new file named `test_models.py` for implementing the code for testing the models. Inside this file, import the models you want to test.
4. Inside the `test_models.py` file, create a new class that inherits from the `TestCase` class of the `django.test` module and implements methods to validate the process of creating and reading the `Model` objects.

다음 명령은 장고에서 테스트를 병렬로 실행하는 방법을 보여줍니다:

```
python manage.py test --parallel
```

앞서 언급한 명령이 실행되면, 장고는 장고 웹 애플리케이션의 모든 테스트를 실행합니다. `--parallel` 매개변수는 장고가 직렬 방식이 아닌 병렬로 N 개의 테스트를 실행하도록 합니다. 병렬로 실행될 테스트의 수는 개발 머신에서 사용 가능한 코어 수에 따라 결정됩니다.

`--parallel` 매개변수 바로 뒤에 수를 언급하여 장고에 병렬로 실행할 수 있는 테스트 수를 명시적으로 알려줄 수도 있습니다. 다음과 같이:

```
python manage.py test --parallel 4
```

이 명령은 장고에게 병렬로 4개의 테스트 케이스만 실행하라고 지시합니다.

테스트가 병렬로 실행될 때, 각 테스트 케이스는 데이터베이스와 기타 환경 변수를 각각 가져와서 동시 환경에서 코드 실행으로 인해 발생할 수 있는 문제를 피합니다.

이제 장고에서 애플리케이션을 단위 테스트하는 방법을 이해했으니, Bookr의 테스트 케이스를 구현하여 우리의 지식을 적용해 봅시다.

활동 14.01 – Bookr에서 모델 및 뷰 테스트하기

이 활동에서는 Bookr 프로젝트에 대한 test cases를 구현할 것입니다. Bookr 프로젝트의 리뷰 애플리케이션 내에서 생성된 모델의 기능을 검증하기 위한 testcases를 구현한 후, 리뷰 애플리케이션 내의 인덱스 뷰를 검증하기 위한 간단한 test case를 구현할 것입니다.

다음 단계는 이 활동을 진행하는 데 도움이 될 것입니다:

1. 리뷰 애플리케이션 디렉토리 내에 tests라는 이름의 디렉토리를 생성하여 리뷰 애플리케이션에 대한 모든 test cases를 모듈화할 수 있도록 합니다.
2. 디렉토리가 일반 디렉토리가 아닌 Python 모듈 디렉토리로 간주되도록 빈 `__init__.py` 파일을 생성합니다.
3. 모델을 테스트하기 위한 코드를 구현하기 위해 `test_models.py`라는 새 파일을 생성합니다. 이 파일 내에서 테스트할 모델을 가져옵니다.
4. `test_models.py` 파일 내에서 `django.test` 모듈의 `TestCase` 클래스를 상속받는 새 클래스를 생성하고 `Model` 객체를 생성하고 읽는 과정을 검증하는 메서드를 구현합니다.

5. To test the view function, create a new file named `test_views.py` inside the `tests` directory that was created in *Step 1*.
6. Inside the `test_views.py` file, import the `test Client` class from the `django.test` module and the index view function from the `reviews` application's `views.py` file.
7. Inside the `test_views.py` file you created in *Step 5*, create a new `TestCase` class and implement methods to validate the index view.
8. Inside the `TestCase` class you created in *Step 7*, create a new function called `setUp()`, inside which you should initialize an instance of `RequestFactory`, which will be used to create a request object that can be directly passed to the view function for testing.
9. Once you have completed the previous steps, write the test cases and run them by executing `python manage.py` to validate that the test cases pass.

Upon completing this activity, all test cases should pass successfully.

Summary

In this chapter, we learned how to write test cases for different components of our web application project with Django. We learned why testing plays a crucial role in the development of any web application and the different types of testing techniques that are employed in the industry to make sure the application code they ship is stable and bug-free.

Then, we looked at how we can use the `TestCase` class provided by Django's test module to implement unit tests, which can be used to test models as well as views. We also looked at how we can use Django's test client to test view functions, which require or do not require the user to be authenticated. We also glanced over another approach of using `RequestFactory` to test method views and class-based views.

We concluded this chapter by understanding the predefined classes provided by Django and where they should be used, while also looking at how we can modularize our testing code base to make it appear clean.

As we move on to the next chapter, we will learn how to deploy our Django application to a server. This will cover web server architecture, verifying a project is ready, and options available for deployment.

5. 뷰 함수를 테스트하기 위해, 1단계에서 생성된 `testsdirectory` 안에 `test_views.py`라는 새 파일을 만드세요.
6. `test_views.py` 파일 안에서 `django.testsmodule`의 `test Client` 클래스를 가져오고 `reviews` 애플리케이션의 `views.py` 파일에서 `index` 뷰 함수를 가져오세요. 5단계에서 생성한 `test_views.py` 파일 안에 새로운 `TestCase` 클래스를 만들고
7. `index` 뷰를 검증하는 메서드를 구현하세요.
8. 7단계에서 생성한 `TestCase` 클래스 안에 `setUp()`이라는 새로운 함수를 만들고, 그 안에서 `RequestFactory`의 인스턴스를 초기화해야 합니다. 이 인스턴스는 테스트를 위해 뷰 함수에 직접 전달할 수 있는 요청 객체를 생성하는 데 사용
9. 됩니다. 이전 단계를 완료한 후, 테스트 케이스를 작성하고 `python manage.py`를 실행하여 테스트 케이스가 통과하는지 검증하세요.

이 활동을 완료하면 모든 테스트 케이스가 성공적으로 통과해야 합니다.

요약

이번 장에서는 Django를 사용하여 웹 애플리케이션 프로젝트의 다양한 구성 요소에 대한 테스트 케이스를 작성하는 방법을 배웠습니다. 우리는 테스트가 모든 웹 애플리케이션 개발에서 중요한 역할을 하는 이유와 애플리케이션 코드를 안정적이고 버그가 없도록 보장하기 위해 산업에서 사용되는 다양한 테스트 기법에 대해 배웠습니다.

그런 다음, Django의 테스트 모듈에서 제공하는 `TestCase` 클래스를 사용하여 모델과 뷰를 테스트하는 데 사용할 수 있는 단위 테스트를 구현하는 방법을 살펴보았습니다. 또한 사용자가 인증을 받아야 하는 뷰 함수와 그렇지 않은 뷰 함수를 테스트하기 위해 Django의 테스트 클라이언트를 사용하는 방법도 살펴보았습니다. 우리는 또한 `RequestFactory`를 사용하여 메서드 뷰와 클래스 기반 뷰를 테스트하는 또 다른 접근 방식에 대해서도 간단히 살펴보았습니다.

우리는 Django에서 제공하는 미리 정의된 클래스와 그것들이 사용되어야 하는 위치를 이해함으로써 이 장을 마무리했으며, 테스트 코드 기반을 모듈화하여 깔끔하게 보이도록 만드는 방법도 살펴보았습니다.

다음 장으로 넘어가면서 우리는 Django 애플리케이션을 서버에 배포하는 방법을 배울 것입니다. 여기에는 웹 서버 아키텍처, 프로젝트 준비 확인 및 배포를 위한 옵션이 포함됩니다.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하세요 (또는 packtpub.com/unlock으로 가세요). 이 책의 이름으로 검색하고, 판을 확인한 다음 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

15

Deploying a Django Project

Now that we have surveyed the crucial aspects of Django application development, we can examine the Django deployment process. Django has important utilities for deploying a project, and there is a range of deployment options that we will examine. Finally, we will step through the deployment of an application to a platform-as-a-service provider.

In this chapter, we will cover the following topics:

- Server architecture
- The Django deployment checklist
- Configuring a project for deployment
- Deployment options for Django

We will commence with a brief discussion of server architectures.

Technical requirement

All the code files in the Chapter15 folder in this book's GitHub repo. To access the repository link, follow the steps in the *"Download the example code files"* section in the *Preface*.

Server architecture

So far, we have been using the Django development server to both execute the Python code for Bookr and serve the static and media files. We mentioned throughout the book that the Django development server is not suitable for use in production. It is not designed to run multiple processes or handle many users, and more importantly, it doesn't run when the `DEBUG` setting is set to `True`.

We will start this chapter by looking at the architecture used in a production web server – that is, where the functions of the Django development server are split and handled by two

15

장고 프로젝트 배포

이제 장고 애플리케이션 개발의 중요한 측면을 조사했으므로 장고 배포 프로세스를 살펴볼 수 있습니다. 장고는 프로젝트를 배포하기 위한 중요한 유틸리티를 가지고 있으며, 우리가 살펴볼 다양한 배포 옵션이 있습니다. 마지막으로, 플랫폼-서비스 제공업체에 애플리케이션을 배포하는 과정을 단계별로 진행할 것입니다.

이 장에서는 다음 주제를 다룰 것입니다: 서버

- 아키텍처
- 장고 배포 체크리스트 배포를 위한
- 프로젝트 구성 장고의 배포 옵션
-

서버 아키텍처에 대한 간단한 논의로 시작하겠습니다.

기술 요구 사항

이 책의 GitHub 리포지토리의 Chapter15 폴더에 있는 모든 코드 파일. 리포지토리 링크에 접근하려면 서문에 있는 "예제 코드 파일 다운로드" 섹션의 단계를 따르세요.

서버 아키텍처

지금까지 우리는 Django 개발 서버를 사용하여 Bookr의 Python 코드를 실행하고 정적 및 미디어 파일을 제공해 왔습니다. 우리는 책 전반에 걸쳐 Django 개발 서버가 프로덕션에서 사용하기에 적합하지 않다고 언급했습니다. 이 서버는 여러 프로세스를 실행하거나 많은 사용자를 처리하도록 설계되지 않았으며, 더 중요한 것은 DEBUG 설정이 True로 설정되어 있을 때 실행되지 않습니다.

이 장은 프로덕션 웹 서버에서 사용되는 아키텍처를 살펴보는 것으로 시작할 것입니다. 즉, Django 개발 서버의 기능이 두 개로 나뉘어 처리되는 방식입니다.

different applications. There is a frontend web server (for example, NGINX, `lighttpd`, or Apache), which receives the request from the browser, and the application server (for example, Gunicorn or `uWSGI`), which executes the Python code. The frontend web server decides how the request should be handled. If it is a request for a static or media file, then the frontend web server can handle the request itself; it can just read and send the file. If the request is for Python-specific code, then it will be forwarded to the application server to handle. The Django application then parses the URL and other HTTP data, generates an `HttpRequest` object using a view, and sends the response back to the frontend web server. The frontend web server then passes the response back to the browser.

The following figure elaborates on this process:

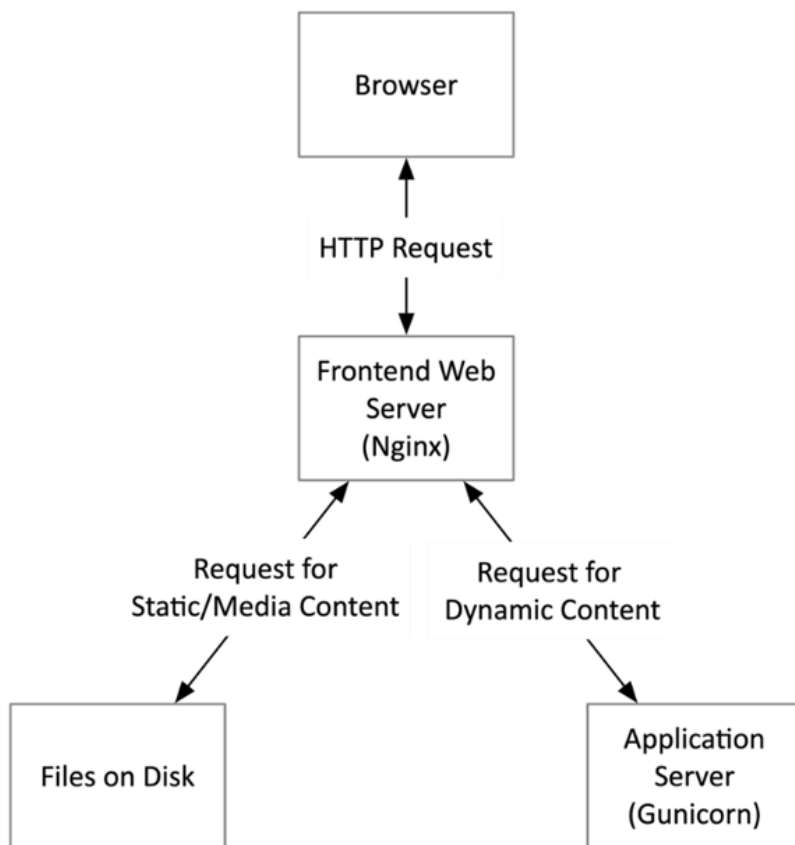


Figure 15.1: Request validation

다양한 애플리케이션. 프론트엔드 웹 서버(예: NGINX, lighttpd, 또는 Apache)가 브라우저로부터 요청을 받고, 애플리케이션 서버(예: Gunicorn 또는 uWSGI)가 Python 코드를 실행합니다. 프론트엔드 웹 서버는 요청을 어떻게 처리할지 결정합니다. 정적 파일이나 미디어 파일에 대한 요청이라면, 프론트엔드 웹 서버가 요청을 직접 처리할 수 있습니다. 파일을 읽고 전송하기만 하면 됩니다. 요청이 Python 전용 코드에 대한 것이라면, 애플리케이션 서버로 전달되어 처리됩니다. 그런 다음 Django 애플리케이션은 URL 및 기타 HTTP 데이터를 구문 분석하고, 뷰를 사용하여 HttpRequest 객체를 생성한 후, 응답을 프론트엔드 웹 서버로 다시 보냅니다. 프론트엔드 웹 서버는 응답을 브라우저로 다시 전달합니다.

다음 그림은 이 과정을 자세히 설명합니다:

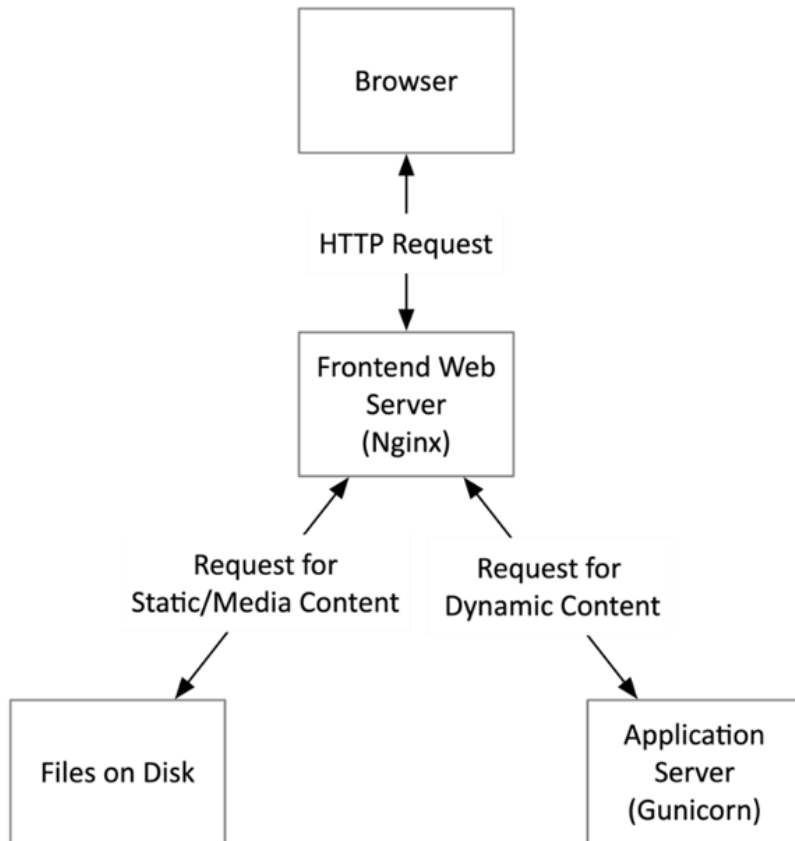


그림 15.1: 요청 검증

Gunicorn

Gunicorn is an easy-to-use application server. Its settings can be controlled with command-line flags or a configuration file. In its simplest usage, it can be run like this:

```
gunicorn bookr.wsgi:application
```

This starts Gunicorn using the Django WSGI.

Note

WSGI (often pronounced *wizz-gee*) stands for **Web Server Gateway Interface**. It describes a standard for Python to communicate with web servers. This means any WSGI Python script should be able to communicate with any WSGI server, and any combination of frontend server and application server can be used together, provided they both "speak WSGI." For example, instead of using Gunicorn, we could use uWSGI. Its configuration would be different, but it serves the same purpose.

Whitenoise

For a production environment with significant traffic, a frontend web server such as NGINX is required to handle the static and media content. However, if we are setting up a site with modest demand, then we can install Django middleware called Whitenoise that will fulfill this role, and an application server (such as Gunicorn) can be run without a frontend web server.

Note

Since 2019, Django has introduced the **Asynchronous Server Gateway Interface (ASGI)**, which is used to connect with asynchronous servers such as Uvicorn and Daphne. Asynchronous servers are frequently used for long-lived connections and are frequently seen in applications such as chatbots, site notifications, live update dashboards, and streaming services. ASGI is outside the scope of this book.

In the next section, we will examine the deployment checklist in preparation for a Gunicorn/Whitenoise deployment.

The deployment checklist

Before a Django application is deployed, it needs to be configured in a more robust way, as it is no longer just accessible by the developer but is exposed to the whole internet.

Gunicorn

Gunicorn은 사용하기 쉬운 애플리케이션 서버입니다. 설정은 명령줄 플래그나 구성 파일로 제어할 수 있습니다. 가장 간단한 사용법으로는 다음과 같이 실행할 수 있습니다:

```
gunicorn bookr.wsgi:application
```

이 명령은 Django WSGI를 사용하여 Gunicorn을 시작합니다.

참고

WSGI(종종 위즈기라고 발음됨)는 웹 서버 게이트웨이 인터페이스를 의미합니다. 이는 Python이 웹 서버와 통신하는 표준을 설명합니다. 이는 모든 WSGI Python 스크립트가 모든 WSGI 서버와 통신할 수 있어야 하며, 모든 프론트엔드 서버와 애플리케이션 서버의 조합을 함께 사용할 수 있음을 의미합니다. 두 서버 모두 "WSGI를 말할 수" 있는 경우에 한합니다. 예를 들어, Gunicorn 대신 uWSGI를 사용할 수 있습니다. 구성은 다르겠지만, 같은 목적을 수행합니다.

Whitenoise

상당한 트래픽이 있는 프로덕션 환경에서는 NGINX와 같은 프론트엔드 웹 서버가 정적 및 미디어 콘텐츠를 처리하는 데 필요합니다. 그러나 수요가 적은 사이트를 설정하는 경우, 이 역할을 수행할 Whitenoise라는 Django 미들웨어를 설치할 수 있으며, 애플리케이션 서버(예: Gunicorn)는 프론트엔드 웹 서버 없이 실행될 수 있습니다.

메모

2019년부터 Django는 비동기 서버인 Uvicorn 및 Daphne와 연결하는 데 사용되는 비동기 서버 게이트웨이 인터페이스(ASGI)를 도입했습니다. 비동기 서버는 장기 연결에 자주 사용되며 챗봇, 사이트 알림, 실시간 업데이트 대시보드 및 스트리밍 서비스와 같은 애플리케이션에서 자주 볼 수 있습니다. ASGI는 이 책의 범위를 벗어납니다.

다음 섹션에서는 Gunicorn/Whitenoise 배포를 준비하기 위한 배포 체크리스트를 살펴볼 것입니다.

배포 체크리스트

Django 애플리케이션이 배포되기 전에 더 강력한 방식으로 구성되어야 합니다. 이는 더 이상 개발자만 접근할 수 있는 것이 아니라 전체 인터넷에 노출되기 때문입니다.

The Django documentation gives a security checklist that should be followed before deploying an app to a production environment: <https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/>.

A development environment will typically fall short of the security needs of a production environment. We will discuss the tasks of the Django deployment checklist that are required to harden a Django project and then address them for the Bookr app in *Exercise 15.01 – Configuring the Bookr project for deployment*.

We will give a brief on the items in the checklist next.

Run `manage.py check --deploy`

When we start the Django development server with the `manage.py runserver` command, we might notice a line in the console that says this:

```
System check identified 0 issues (0 silenced).
```

This line is output from the `check` subcommand, which is used to detect and report on configuration issues in a Django project.

We can invoke it independently of the development server with this command:

```
$ python manage.py check
```

It can be used to assess the state of a Django application for production deployment by including the `--deploy` flag:

```
$ python manage.py check --deploy
```

This will alert us to security issues in our project configuration that we will need to address in a production environment before deployment, as seen in *Figure 15.2*.

장고 문서에서는 앱을 프로덕션 환경에 배포하기 전에 따라야 할 보안 체크리스트를 제공합니다: <https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/>.

개발 환경은 일반적으로 프로덕션 환경의 보안 요구 사항을 충족하지 못합니다. 우리는 장고 프로젝트를 강화하는 데 필요한 장고 배포 체크리스트의 작업에 대해 논의한 다음, 연습 15.01에서 Bookr 앱에 대해 다룰 것입니다 – Bookr 프로젝트를 배포를 위해 구성하기.

다음으로 체크리스트의 항목에 대해 간략히 설명하겠습니다.

manage.py check --deploy 실행

manage.py runserver 명령으로 장고 개발 서버를 시작할 때, 콘솔에 다음과 같은 줄이 표시될 수 있습니다:

```
시스템 점검에서 0개의 문제를 확인했습니다 (0개의 무시됨).
```

이 줄은 장고 프로젝트의 구성 문제를 감지하고 보고하는 데 사용되는 check 하위 명령의 출력입니다.

다음 명령으로 개발 서버와 독립적으로 호출할 수 있습니다:

```
$ python manage.py check
```

생산 배포를 위한 Django 애플리케이션의 상태를 평가하는 데 사용할 수 있으며, --deploy 플래그를 포함합니다:

```
$ python manage.py check --deploy
```

이것은 배포 전에 생산 환경에서 해결해야 할 프로젝트 구성의 보안 문제를 경고합니다. 이는 그림 15.2에서 볼 수 있습니다.

```
$ python manage.py check --deploy
System check identified some issues:

WARNINGS:
?: (security.W004) You have not set a value for the SECURE_HSTS_SECONDS setting.
   If your entire site is served only over SSL, you may want to consider setting a
   value and enabling HTTP Strict Transport Security. Be sure to read the document
   ation first; enabling HSTS carelessly can cause serious, irreversible problems.
?: (security.W008) Your SECURE_SSL_REDIRECT setting is not set to True. Unless y
   our site should be available over both SSL and non-SSL connections, you may want
   to either set this setting True or configure a load balancer or reverse-proxy s
   erver to redirect all connections to HTTPS.
?: (security.W009) Your SECRET_KEY has less than 50 characters, less than 5 uniq
   ue characters, or it's prefixed with 'django-insecure-' indicating that it was g
   enerated automatically by Django. Please generate a long and random value, other
   wise many of Django's security-critical features will be vulnerable to attack.
?: (security.W012) SESSION_COOKIE_SECURE is not set to True. Using a secure-only
   session cookie makes it more difficult for network traffic sniffers to hijack u
   ser sessions.
?: (security.W016) You have 'django.middleware.csrf.CsrfViewMiddleware' in your
   MIDDLEWARE, but you have not set CSRF_COOKIE_SECURE to True. Using a secure-only
   CSRF cookie makes it more difficult for network traffic sniffers to steal the C
   SRF token.
?: (security.W018) You should not have DEBUG set to True in deployment.
?: (security.W020) ALLOWED_HOSTS must not be empty in deployment.

System check identified 7 issues (0 silenced).
```

Figure 15.2: Deployment warnings

We can use this command to evaluate our efforts to secure a Django configuration and will reassess it after creating a robust production environment in *Exercise 15.01*.

Switch away from manage.py runserver

Django's internal runserver functionality is a useful development tool, but as a single-process server, it cannot be deployed in a production environment. A production deployment requires a high-concurrency web server. We will examine the use of `gunicorn` for our deployment.

Critical settings

These are the most crucial settings in a Django project that need to be addressed prior to deployment.

```
$ python manage.py check --deploy
System check identified some issues:

WARNINGS:
?: (security.W004) You have not set a value for the SECURE_HSTS_SECONDS setting.
   If your entire site is served only over SSL, you may want to consider setting a
   value and enabling HTTP Strict Transport Security. Be sure to read the document
   ation first; enabling HSTS carelessly can cause serious, irreversible problems.
?: (security.W008) Your SECURE_SSL_REDIRECT setting is not set to True. Unless y
   our site should be available over both SSL and non-SSL connections, you may want
   to either set this setting True or configure a load balancer or reverse-proxy s
   erver to redirect all connections to HTTPS.
?: (security.W009) Your SECRET_KEY has less than 50 characters, less than 5 uniq
   ue characters, or it's prefixed with 'django-insecure-' indicating that it was g
   enerated automatically by Django. Please generate a long and random value, other
   wise many of Django's security-critical features will be vulnerable to attack.
?: (security.W012) SESSION_COOKIE_SECURE is not set to True. Using a secure-only
   session cookie makes it more difficult for network traffic sniffers to hijack u
   ser sessions.
?: (security.W016) You have 'django.middleware.csrf.CsrfViewMiddleware' in your
   MIDDLEWARE, but you have not set CSRF_COOKIE_SECURE to True. Using a secure-only
   CSRF cookie makes it more difficult for network traffic sniffers to steal the C
   SRF token.
?: (security.W018) You should not have DEBUG set to True in deployment.
?: (security.W020) ALLOWED_HOSTS must not be empty in deployment.

System check identified 7 issues (0 silenced).
```

그림 15.2: 배포 경고

이 명령을 사용하여 장고 구성의 보안을 평가할 수 있으며, 연습 15.01에서 강력한 프로덕션 환경을 만든 후 다시 평가할 것입니다.

manage.py runserver에서 전환

장고의 내부 runserver 기능은 유용한 개발 도구이지만, 단일 프로세스 서버로서 프로덕션 환경에 배포할 수 없습니다. 프로덕션 배포는 높은 동시성을 지원하는 웹 서버가 필요합니다. 우리는 배포를 위해 gunicorn의 사용을 검토할 것입니다.

중요한 설정

이들은 배포 전에 해결해야 하는 장고 프로젝트의 가장 중요한 설정입니다.

SECRET_KEY

In the check management command, we saw the following warning:

```
?: (security.W009) Your SECRET_KEY has less than 50 characters, less than 5 unique
characters, or it's prefixed with 'django-insecure-' indicating that it was
generated automatically by Django. Please generate a long and random value,
otherwise many of Django's security-critical features will be vulnerable to
attack.
```

When a new Django project is created, a less secure SECRET_KEY is generated. Prior to deployment, it is necessary to create a more secure key for the production environment. Django comes with a function called `get_random_secret_key()` that produces a key that conforms to its current best practice standards. It can be invoked as follows:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

We also need to be mindful that this key is not present in the source code of the project, and we can enforce this using an environment variable, decouple, or a secrets file.

DEBUG

Setting DEBUG to True in a development environment is an invaluable way to receive useful information, such as tracebacks when encountering errors in web requests. However, this same information can be a security vulnerability if it is available in a production environment, as it can reveal snippets of source code, dependencies, and settings.

Environment-specific settings

ALLOWED_HOSTS

We have gotten this far leaving the value of ALLOWED_HOSTS as an empty list in `settings.py`. However, it is required that it is not empty in a production deployment:

```
?: (security.W020) ALLOWED_HOSTS must not be empty in deployment.
```

Indeed, when DEBUG is set to False, an empty list for ALLOWED_HOSTS will throw an exception:

```
$ python manage.py runserver
CommandError: You must set settings.ALLOWED_HOSTS if DEBUG is False.
```

비밀 키

idx_d7d01b35체크 관리 idx_144a17ac명령에서 다음 경고를 보았습니다:

```
?: (security.W009) 귀하의 비밀 키는 50자 미만이거나, 고유 문자가 5개 미만이거나,
'django-insecure-'로 시작하여 Django에 의해 자동으로 생성되었음을 나타냅니다. 긴 무작
위 값을 생성하십시오. 그렇지 않으면 Django의 보안에 중요한 많은 기능이 공격에 취약해질
것입니다.
```

새로운 Django 프로젝트가 생성될 때, 덜 안전한 비밀 키가 생성됩니다. 배포 전에 프로덕션 환경을 위한 더 안전한 키를 생성하는 것이 필요합니다. Django는 현재의 모범 사례 표준에 부합하는 키를 생성하는 `get_random_secret_key()`라는 함수를 제공합니다. 다음과 같이 호출할 수 있습니다:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

이 키가 프로젝트의 소스 코드에 포함되지 않도록 주의해야 하며, 환경 변수, `decouple` 또는 비밀 파일을 사용하여 이를 강제할 수 있습니다.

DEBUG

개발 환경에서 `DEBUG`를 `True`로 설정하는 것은 웹 요청에서 오류가 발생할 때 추적 정보를 포함한 유용한 정보를 받을 수 있는 귀중한 방법입니다. 그러나 이 정보는 프로덕션 환경에서 사용 가능할 경우 보안 취약점이 될 수 있으며, 소스 코드, 의존성 및 설정의 일부를 노출할 수 있습니다.

환경별 설정

ALLOWED_HOSTS

우리는 `settings.py`에서 `ALLOWED_HOSTS`의 값을 빈 목록으로 두고 이만큼 왔습니다. 그러나 프로덕션 배포에서는 비어 있지 않아야 합니다:

```
?: (security.W020) ALLOWED_HOSTS는 배포 시 비어 있으면 안 됩니다.
```

실제로 `DEBUG`가 `False`로 설정되면 `ALLOWED_HOSTS`에 대한 빈 목록은 예외를 발생시킵니다:

```
$ python manage.py runserver
CommandError: DEBUG가 False인 경우 settings.ALLOWED_HOSTS를 설정해야 합니다.
```

The `ALLOWED_HOSTS` setting was introduced into Django as a mechanism to stop host header poisoning attacks. This is where an HTTP request would contain a header to a malicious site, and it would be used to generate a subsequent false link in an HTML page or email to the malicious site.

If you are deploying to a server with a known domain, this is where you should add it:

```
ALLOWED_HOSTS=[ 'example.com' ]
```

If you are deploying to an environment where the domain is not yet known, a wildcard can be used, but this is less secure and should be replaced when the domain is apparent:

```
ALLOWED_HOSTS=[ '*' ]
```

CACHES

Web caching is the mechanism that allows the responses of frequently made requests to be stored for repeated use in order to avoid unnecessary recomputations that would impact the CPU, filesystem, or database resources.

While we haven't addressed Django's caching abilities in this book, caching is an important strategy for a web application that can be fine-tuned post-deployment. Further information can be found at the following link: <https://docs.djangoproject.com/en/6.0/topics/cache/>.

DATABASES

During development, it is satisfactory to work with a SQLite database, but in a production environment, where multiple HTTP requests will require their own database connections, we will need to scale up from a lightweight database to a production-ready deployment.

We should avoid leaving database connection parameters in a file such as `settings.py`. The database server should be configured to only allow connections from application servers and devices that host backups.

EMAIL_BACKEND and related settings

We discussed email configuration in *Chapter 10, Configuring an Email Server with Django*, and gave an example with a Gmail account. Email configuration is required for automating administration tasks (such as password resets) and application communications.

Static files

So far, we have been using the development server to serve static files. In a production environment, we will need to define a `STATIC_ROOT` directory where `collectstatic` will copy them during the deployment.

ALLOWED_HOSTS 설정은 호스트 헤더 중독 공격을 방지하기 위한 메커니즘으로 장고에 도입되었습니다. 이는 HTTP 요청이 악의적인 사이트에 대한 헤더를 포함하고, 이를 사용하여 HTML 페이지나 이메일에서 악의적인 사이트에 대한 잘못된 링크를 생성하는 경우입니다.

서버에 알려진 도메인으로 배포하는 경우, 여기에 추가해야 합니다:

```
ALLOWED_HOSTS=['example.com']
```

도메인이 아직 알려지지 않은 환경에 배포하는 경우, 와일드카드를 사용할 수 있지만, 이는 덜 안전하며 도메인이 명확해지면 교체해야 합니다:

```
ALLOWED_HOSTS=['*']
```

캐시

웹 캐싱은 자주 발생하는 요청의 응답을 저장하여 반복 사용을 위해 보관하는 메커니즘입니다. 이는 CPU, 파일 시스템 또는 데이터베이스 리소스에 영향을 미칠 불필요한 재계산을 피하기 위해 사용됩니다.

이 책에서는 장고의 캐싱 기능을 다루지 않았지만, 캐싱은 배포 후 조정할 수 있는 웹 애플리케이션에 중요한 전략입니다. 추가 정보는 다음 링크에서 확인할 수 있습니다: <https://docs.djangoproject.com/en/6.0/topics/cache/>.

데이터베이스

개발 중에는 SQLite 데이터베이스로 작업하는 것이 만족스럽지만, 여러 HTTP 요청이 각자의 데이터베이스 연결을 요구하는 프로덕션 환경에서는 경량 데이터베이스에서 프로덕션 준비가 된 배포로 확장해야 합니다.

settings.py와 같은 파일에 데이터베이스 연결 매개변수를 남기는 것을 피해야 합니다. 데이터베이스 서버는 애플리케이션 서버와 백업을 호스팅하는 장치에서만 연결을 허용하도록 구성되어야 합니다.

EMAIL_BACKEND 및 관련 설정

우리는 10장, Django로 이메일 서버 구성에서 이메일 구성에 대해 논의했으며, Gmail 계정을 사용한 예를 제시했습니다. 이메일 구성은 관리 작업(예: 비밀번호 재설정) 및 애플리케이션 통신을 자동화하는 데 필요합니다.

정적 파일

지금까지 우리는 정적 파일을 제공하기 위해 개발 서버를 사용해 왔습니다. 프로덕션 환경에서는 collectstatic이 배포 중에 복사할 STATIC_ROOT 디렉토리를 정의해야 합니다.